1. Loft Language Reference

A statically-typed scripting language with null safety and built-in parallel execution.

Version 2026.11.0

Every code example in this document is an executable part of the test suite.

Contents

1. Loft Language Reference 1
2. Getting Started 13
    - 2.0.1. Prerequisites 13
    - 2.0.2. Install from source 13
    - 2.0.3. Build locally 13
    - 2.0.4. Verify the installation 13
    - 2.0.5. Hello, World 13
    - 2.0.6. A slightly bigger program 14
    - 2.0.7. Command-line flags 14
    - 2.0.8. Running your program as a native binary 14
    - 2.0.9. Standard library 15
    - 2.0.10. Using libraries 15
    - 2.0.11. Editor setup 15
    - 2.0.12. Next steps 15
3. vs Rust 16
    - 3.0.1. Variables — no let or mut 16
    - 3.0.2. Null instead of Option<T> 16
    - 3.0.3. Parameter mutability — const and & instead of ownership 17
    - 3.0.4. ^ is XOR; ** or pow() for exponentiation 17
    - 3.0.5. No loop keyword — an infinite loop is while true 18
    - 3.0.6. Filtered loops — for ... if 18
    - 3.0.7. Loop attributes: #first, #count, #index 19
    - 3.0.8. Named loop break — x#break 19
    - 3.0.9. Methods by self name, not impl blocks 19
    - 3.0.10. Polymorphic enum dispatch 20
    - 3.0.11. Closures — same-scope capture works 21
    - 3.0.12. Generic functions — inferred, with structural interface bounds 22
    - 3.0.13. String formatting — embedded expressions 22
    - 3.0.14. Built-in parallel for-loops — par(...) 23
4. vs Python 24
    - 4.0.1. Variables — static types inferred at first assignment 24
    - 4.0.2. Null — structured absence, not a universal wildcard 24
    - 4.0.3. Structs vs classes and dicts 25
    - 4.0.4. while loop — yes, and while true; no while ... else 26
    - 4.0.5. Polymorphic enum dispatch vs isinstance 26
    - 4.0.6. String formatting — embedded expressions 27
    - 4.0.7. Collections — typed and built-in 28
    - 4.0.8. Closures — same-scope capture works 29
    - 4.0.9. No exception handling — file errors use FileResult 30
    - 4.0.10. Built-in parallel for-loops — par(...) 31
    - 4.0.11. Generic functions — interface bounds, no duck typing 32
    - 4.0.12. Function signatures — defaults and named args, but no *args 33

4.0.13. Ecosystem — minimal standard library 33
4.0.14. Exponentiation with ** or pow(); ^ is XOR 34
5. Keywords 36
5.0.1. Conditionals 36
5.0.2. if as an expression 36
5.0.3. Iteration 37
5.0.4. The other loop: while 37
5.0.5. Nested loops and break 38
5.0.6. Skipping an iteration: continue 38
5.0.7. Loop helpers: #first and #count 38
5.0.8. Formatting a loop inline 39
6. Texts 40
6.0.1. Joining and measuring text 40
6.0.2. Do not mix the two counts 40
6.0.3. Reading individual characters 41
6.0.4. Slicing text 41
6.0.5. Iterating over characters 41
6.0.6. Searching inside text 42
6.0.7. Escaping braces in format strings 42
6.0.8. Aligning text in a fixed-width field 42
7. Integers 43
7.0.1. Converting between numbers and text 43
7.0.2. Arithmetic and operator precedence 43
7.0.3. Division by zero — produces null and keeps running 44
7.0.4. Warning when you write a literal zero divisor 44
7.0.5. Embedding integers in text 44
7.0.6. Number format specifiers 44
7.0.7. Large integers 45
7.0.8. Common pitfall: integer overflow 45
8. Boolean 46
8.0.1. Logical operators: and / or 46
8.0.2. Null in boolean context 46
8.0.3. Bitwise operators 47
8.0.4. Negation 47
8.0.5. Formatting booleans 47
8.0.6. ‘&’ vs ‘and’ 47
9. Float 48
9.0.1. Undefined results are null (‘float?’) — but infinities are NOT undefined 48
9.0.2. Formatting Floats 49
9.0.3. Math Functions 49
10. Functions 51
10.0.1. Declaring Functions 51
10.0.2. Default arguments that involve other arguments 51

10.0.3. Named Arguments 51
10.0.4. Reference Parameters and Defaults 51
10.0.5. Early Return 52
10.0.6. Const and Reference Parameters 52
10.0.7. Type-Based Dispatch 52
10.0.8. Function References 53
10.0.9. Lambda Expressions 53
11. Vector 56
11.0.1. Transforming vectors: map, filter, reduce 56
11.0.2. Clearing 57
11.0.3. Slicing 58
11.0.4. Comprehensions 58
11.0.5. Reverse Iteration 58
11.0.6. Passing vectors to functions 59
11.0.7. Higher-order functions 59
12. Structs 61
12.0.1. Field Constraints 61
12.0.2. Methods 61
12.0.3. Defaults and Computed Fields 62
12.0.4. Storing many structs in a vector 62
12.0.5. Value structs 62
12.0.6. A struct that contains itself 63
12.0.7. A record with some fields changed 67
12.0.8. Comparing structs 67
12.0.9. sizeof 67
13. Enums 68
13.0.1. Enums that carry data 68
13.0.2. Variants that hold the enum itself 68
13.0.3. Methods that work on any variant 69
13.0.4. Enum methods 69
13.0.5. A method on the enum itself is the fallback 70
13.0.6. Match expressions on enums 71
13.0.7. Guard clauses 71
13.0.8. Scalar match 71
14. Sorted 73
14.0.1. Adding Elements 73
14.0.2. Looking Up by Key 73
14.0.3. Iterating in Order 74
14.0.4. Iterating in Reverse 74
14.0.5. Loop Helpers: #first, #count, #index, and #remove 74
14.0.6. Removing by Key 75
14.0.7. Iterating a Key Range 76
14.0.8. Iterating a Key Range 76

14.0.9. Filtering on something that is NOT the key 77
14.0.10. Open-ended Range (tail) 77
14.0.11. Open-ended Range (head) 77
14.0.12. Range outside the for (building a result vector) 77
15. Index 78
15.0.1. Adding and Looking Up Records 78
15.0.2. Iterating in Order 79
15.0.3. Range Queries 79
15.0.4. Loop Helpers: #first and #count 79
15.0.5. Removing by Key 79
16. Hash 81
16.0.1. Combining Hash and Vector 81
16.0.2. Basic Lookup 81
16.0.3. Hash + Vector Together 82
16.0.4. Removing a Key 82
16.0.5. Iterating a Hash 82
17. File 84
17.0.1. Inspecting the File System 84
17.0.2. Writing a Text or Binary File 85
17.0.3. Reading Back What You Wrote 85
17.0.4. Working with Vectors and Struct Data 86
17.0.5. Directories 87
17.0.6. Error handling 88
18. Lexer 90
18.0.1. Setting Up the Lexer 90
18.0.2. Reading Tokens 90
18.0.3. String Literals and Comments 91
18.0.4. Embedded Format Expressions 92
19. Parser 93
19.0.1. What the parser understands 93
19.0.2. Parsing a minimal function 94
19.0.3. Parsing a struct and function together 94
19.0.4. Parsing an enum 94
19.0.5. Parsing several statements in a body 94
19.0.6. Parsing assignments and operators 94
19.0.7. Parsing a for loop and arithmetic 94
19.0.8. Parsing while, match and lambdas 95
19.0.9. Practical use: validating user-supplied code 95
19.0.10. What a failed parse looks like 95
19.0.11. What it does not parse 95
20. Libraries 97
20.0.1. Constants and Enums 97
20.0.2. Struct Construction and Field Access 97

20.0.3. Calling Library Methods 97
20.0.4. Extending a Library Type 97
20.0.5. Package Layout and loft.toml 99
20.0.6. Wildcard and Selective Imports 100
20.0.7. Limitations 100
21. Store Locks 101
21.0.1. The two places ‘const’ can go 101
21.0.2. const local variables 102
21.0.3. Runtime store locks with #lock 102
21.0.4. When to use each approach 103
22. Parallel execution 104
22.0.1. Why parallel loops? 104
22.0.2. Syntax 104
22.0.3. Two call forms 104
22.0.4. What a worker may do 104
22.0.5. Order 105
22.0.6. Free function (Form 1) 106
22.0.7. Extra Arguments 106
22.0.8. Struct Return 106
22.0.9. Method call (Form 2) 107
22.0.10. Any for-source 107
22.0.11. Empty Vector 107
22.0.12. Sequential fallback 107
23. Logging 109
23.0.1. The four severity levels 109
23.0.2. Configuring the log destination 109
23.0.3. Production mode 110
23.0.4. Using format strings in log messages 110
23.0.5. Example log.txt output 110
23.0.6. Typical usage pattern 111
24. Time 112
24.0.1. Calendars are a library 112
24.0.2. Wall-clock time: now() 112
24.0.3. Elapsed time: ticks() 113
24.0.4. Measuring elapsed time 113
24.0.5. Common patterns 113
25. Safety 115
25.0.1. Null values — hidden reserved values 116
25.0.2. Parsing text to a number needs a fallback 117
25.0.3. Integer overflow yields null 117
25.0.4. Bitwise operators with zero 117
25.0.5. Float null compares uniformly with every other scalar 117
25.0.6. Text length counts characters; size counts bytes 118

25.0.7. `\#index` means different things on text and vectors 118
25.0.8. `??` evaluates the left side exactly once 119
25.0.9. Text indexing and slicing return different types 119
25.0.10. Format strings: braces are always interpreted 119
25.0.11. A hash iterates, and collections over one record type index one table 119
25.0.12. Mutation guard blocks appending during iteration 120
25.0.13. If-expression requires else when used as a value 120
25.0.14. Match guards do not prove a variant is handled 121
25.0.15. A parameter shares; a bind copies 121
25.0.16. Text file reading assumes UTF-8 121
25.0.17. A struct literal zeroes the fields it leaves out 121
25.0.18. A field only some variants declare answers null on the others 122
25.0.19. A yield inside an expression runs eagerly on –native 122
25.0.20. XOR is ^, not exponentiation 122
26. JSON 123
26.0.1. Serialisation — struct to JSON 123
26.0.2. Parsing — JSON to struct 123
26.0.3. Vectors 124
26.0.4. What went wrong — 'json_errors()' and '#errors' 124
26.0.5. Nested Structs 124
26.0.6. Reading a document whose shape you do not know 125
26.0.7. A missing field gets its own declared absence 126
26.0.8. The shapes on the wire 127
27. Generics 130
27.0.1. Declaring a generic function 130
27.0.2. Calling a generic function 130
27.0.3. Multiple parameters of the same type 130
27.0.4. Generic functions on vectors 130
27.0.5. Bounded generics with interfaces 131
27.0.6. Combining bounds 132
27.0.7. Your own types under a bound 133
27.0.8. Generic structs 134
27.0.9. Two type variables and a callback 134
27.0.10. Disallowed operations 135
28. Closures 136
28.0.1. Integer capture 136
28.0.2. Multiple integer captures 136
28.0.3. Text capture 136
28.0.4. Capture timing 136
28.0.5. Collections and structs are shared 136
28.0.6. Writing to a captured variable 136
28.0.7. Cross-scope closures 136
28.0.8. Closures with higher-order functions 136

28.0.9. Non-capturing lambdas with higher-order functions 136
28.0.10. Capturing a '&' parameter 136
28.0.11. What a closure cannot do 136
28.0.12. Integer capture 137
28.0.13. Multiple integer captures 137
28.0.14. Text capture 137
28.0.15. Capture timing 138
28.0.16. Collections and structs are shared 138
28.0.17. Writing to a captured variable 138
28.0.18. Cross-scope closures 139
28.0.19. Closures with higher-order functions 139
28.0.20. Non-capturing lambdas with higher-order functions 139
28.0.21. Capturing a '&' parameter 139
28.0.22. What a closure cannot do 140
29. Coroutines 141
29.0.1. Iterating with a for loop 141
29.0.2. The body really is suspended 141
29.0.3. Manual advance with next() and exhausted() 141
29.0.4. Generators with parameters 141
29.0.5. Delegation with yield from 141
29.0.6. Early termination with break 141
29.0.7. A generator runs once, and has no return 141
29.0.8. Iterating with a for loop 143
29.0.9. The body really is suspended 143
29.0.10. Manual advance with next() and exhausted() 144
29.0.11. Generators with parameters 144
29.0.12. Delegation with yield from 144
29.0.13. Early termination with break 145
29.0.14. A generator runs once, and has no return 145
30. Tuples 146
30.0.1. Creating tuples 146
30.0.2. Elements of different types 146
30.0.3. Accessing elements 146
30.0.4. Modifying elements 146
30.0.5. Tuples as function parameters 146
30.0.6. Returning tuples 146
30.0.7. Destructuring 146
30.0.8. Comparing tuples 146
30.0.9. Three or more elements 146
30.0.10. Creating tuples 147
30.0.11. Elements of different types 147
30.0.12. Modifying elements 148
30.0.13. Tuples as function parameters 148

30.0.14. Tuples as reference parameters (swap in place) 148
30.0.15. Returning a tuple from a function 148
30.0.16. Destructuring 149
30.0.17. Comparing tuples 149
30.0.18. Three or more elements 149
31. Match 150
31.0.1. Simple enum matching 150
31.0.2. Match as expression 151
31.0.3. Struct-enum destructuring 151
31.0.4. Patterns inside a field 151
31.0.5. Guards 152
31.0.6. Wildcard _ 152
31.0.7. Integer matching 153
31.0.8. Range patterns 153
31.0.9. Null patterns 153
31.0.10. Text matching 153
31.0.11. Multiple patterns with | 154
31.0.12. Tuple patterns 154
31.0.13. Slice arms that cover every length 154
31.0.14. When nothing matches 155
31.0.15. Match inside an arm 155
32. Formatting 156
32.0.1. Basic interpolation 156
32.0.2. Writing a real brace 157
32.0.3. Width and alignment 157
32.0.4. A fill character 157
32.0.5. The flags may be written in any order 158
32.0.6. Zero padding 158
32.0.7. Signed format 158
32.0.8. Hexadecimal, binary, octal 158
32.0.9. Float precision 159
32.0.10. Width + precision 159
32.0.11. JSON format 159
32.0.12. Vector format 159
32.0.13. Large integers and single-precision floats 159
32.0.14. Pretty-printing a struct 160
32.0.15. Null, and why formatting never fails 160
32.0.16. Character format 160
32.0.17. Boolean format 161
32.0.18. Building a value instead of text 161
33. Reference parameter forwarding 163
33.0.1. A scalar argument is copied 163
33.0.2. A struct or collection argument is SHARED 163

33.0.3. What & adds: REPLACEMENT ... 163
33.0.4. Forwarding ... 163
33.0.5. A scalar is copied ... 164
33.0.6. A struct or collection is shared — no & required ... 164
33.0.7. Replacement is the one thing that needs & ... 164
33.0.8. Forwarding carries it through ... 164
33.0.9. A worked example ... 165
34. Feature catalogue ... 166
34.0.1. Language features ... 166
34.0.2. Tooling and infrastructure ... 168
35. Running your program ... 170
35.0.1. Run a file ... 170
35.0.2. Give your program some arguments ... 170
35.0.3. Try something without making a file ... 170
35.0.4. Two ways to run, and why you usually ignore this ... 171
35.0.5. Stop a program that runs too long ... 171
35.0.6. Check your work without running it ... 171
35.0.7. When loft tells you something is wrong ... 171
36. Testing your code ... 173
36.0.1. Write a test ... 173
36.0.2. Run your tests ... 173
36.0.3. Run just one test ... 174
36.0.4. When a test fails ... 174
36.0.5. Read the line after the result ... 174
36.0.6. Once you have a project ... 174
37. Debugging ... 176
37.0.1. Stop on a line ... 176
37.0.2. Look at a value ... 176
37.0.3. Move one step at a time ... 176
37.0.4. Watch a value instead of watching for it ... 177
37.0.5. Change a value while it is running ... 177
37.0.6. Getting out ... 178
37.0.7. Typing, or feeding it a script ... 178
38. Projects and libraries ... 179
38.0.1. What a package looks like ... 179
38.0.2. The manifest ... 179
38.0.3. Testing a package ... 179
38.0.4. Running one test while you work ... 180
38.0.5. Check it compiles, without running anything ... 180
38.0.6. Coverage — what the tests did not reach ... 180
38.0.7. Using someone else's package ... 180
39. Call it yourself ... 182
39.0.1. What to try first ... 182

39.0.2. Numbers with a shape ... 182
39.0.3. Text goes in, and text comes back ... 183
39.0.4. A struct comes back ... 183
39.0.5. Pausing inside a run ... 183
39.0.6. When the panel says \<unavailable\> ... 184
40. Standard Library ... 185
40.1. Types ... 185
40.2. Interfaces ... 186
40.3. Math ... 188
40.4. exp / ln / log2 / log10 ... 191
40.5. min / max / clamp ... 192
40.6. Text ... 195
40.7. Collections ... 197
40.8. Output and Diagnostics ... 197
40.9. Vector aggregates ... 199
40.10. Assertions that report both sides ... 201
40.11. Field iteration support types ... 202
40.12. File System ... 203
40.13. Files and directories ... 204
40.14. Binary file I/O ... 207
40.15. Persistent stores ... 208
40.16. Lazy stores ... 210
40.17. Paged store loading ... 212
40.18. Environment ... 214
40.19. Optional C libraries ... 215
40.20. System directories ... 215
40.21. Time ... 216
40.22. Memory diagnostics ... 216
40.23. Path helpers ... 217
40.24. Text: case and classification ... 217
40.25. Text: bytes and code points ... 219
40.26. Stack traces ... 220
40.27. Coroutines ... 221
40.28. JSON ... 221
40.29. JSON: reading values ... 222
40.30. JSON: building values ... 223
40.31. JSON: the struct bridge ... 224
40.32. Type reflection ... 225
41. Roadmap ... 230
41.0.1. 0.8.4 (shipped 2026-04-24): Awesome Brick Buster ... 230
41.0.2. 0.8.5 (shipped 2026-06-07): Language maturity ... 230
41.0.3. 0.9.0 — Fully working loft language ... 231
41.0.4. 1.0.0 — Totally sure everything works ... 232

41.0.5. 1.1 and beyond 232
41.0.6. Following progress 232

2. Getting Started

Loft is a lightweight scripting language with null safety, built-in parallel execution, and a fast interpreter called **loft**. This page shows how to install loft and write your first program.

2.0.1. Prerequisites

Building from source requires the Rust toolchain. loft is an edition-2024 crate, so that means **Rust 1.96 or later** — the same version `--native` needs to compile your programs. Install it from rustup.rs if you do not already have it:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2.0.2. Install from source

Clone the repository and install it into `~/.local`, with no root needed:

```
git clone https://github.com/loft-lang/loft
cd loft
make install-user
```

This puts the `loft` binary in `~/.local/bin/` and the standard library it loads at runtime in `~/.local/share/loft/`, and tells you if `~/.local/bin` is not on your PATH. `make install` installs system-wide under `/usr/local` instead.

2.0.3. Build locally

To run loft from the checkout without installing it:

```
cargo build --release
./target/release/loft --version
```

The binary at `target/release/loft` uses the standard library in the checkout's `default/` directory, or `share/loft/` beside an installed `bin/`. A copy with neither beside it still runs, on the standard library built into it. `make install-user` (or `make install`) is the way onto your `PATH`.

2.0.4. Verify the installation

```
loft --version
```

2.0.5. Hello, World

Create a file called `hello.loft`:

```
fn main() {
    println("Hello, Loft!");
}
```

Run it:

```
loft hello.loft
```

```
Hello, Loft!
```

2.0.6. A slightly bigger program

Variables, loops, and string interpolation all work without any ceremony:

```
fn main() {
    // Variables are mutable by default; types are inferred.
    total = 0;
    for n in 1..=10 {
        total += n;
    }
    println("Sum 1..10 = {total}");

    // Vectors use square-bracket literals.
    words = ["loft", "is", "fast"];
    for w in words {
        print("{w} ");
    }
    println("");
}
```

2.0.7. Command-line flags

Run `loft --help` for the full list. The most commonly used flags are:

2.0.8. Running your program as a native binary

`loft myprogram.loft` already compiles through `rustc` when it can find it, and interprets when it cannot — you do not have to choose, and the answer is the same either way. Compiling takes a few seconds the first time and then runs compute-heavy code many times faster than interpreting (Performance measures native against hand-written Rust). Ask for one on purpose when you want to:

```
loft --native myprogram.loft
```

To see the generated Rust code without running it:

```
loft --native-emit myprogram.rs myprogram.loft
cat myprogram.rs
```

Native runs need `rustc` on your PATH — the same Rust you built loft with. Without it, `--native` says so and interprets instead. A downloaded release carries no native runtime, so it always interprets.

2.0.8.1. Compiling through C

`--native-c` writes your program as C and builds it with the C compiler already on your machine, so it needs no Rust at all: the C runtime is inside the `loft` binary. It answers what the interpreter answers; it is on its way to becoming what `--native` means. What each target needs:

When a tool is missing, loft says which and what to install on the machine it runs on.

2.0.9. Standard library

The standard library is a set of `.loft` files bundled alongside the loft binary. They are loaded automatically before your program runs — you do not need any `import` or `use` statement to call `println`, `assert`, or any other built-in function.

The full function reference is in the Standard Library section of these docs.

2.0.10. Using libraries

Place a library file (e.g. `lib/myutil.loft`) next to your program, then bring it into scope:

```
use myutil;

fn main() {
    result = myutil::compute(42);
    println("{result}");
}
```

See Libraries for the full search path and naming rules.

2.0.11. Editor setup

loft ships its own editor support, so you do not need to pretend `.loft` is Rust:

- **VS Code** — the extension in `editors/vscode/` of the repository gives syntax highlighting, snippets, and Run buttons for the interpreted (F5) and native (Ctrl+F5) paths. Build and install it with `cd editors/vscode && npm install && vsce package && code --install-extension loft-0.1.0.vsix`. It is not on the marketplace yet.
- **Any LSP editor** — `loft-lsp` is a language server built from this repository (`cargo build --release --bin loft-lsp`). It answers diagnostics, completion, go-to-definition, references, hover, document symbols, semantic tokens, inlay hints, rename and formatting over stdio. `loft-dap` is the matching debug adapter.
- **Vim / Neovim** — `editors/nvim/` adds filetype detection and syntax highlighting; point your LSP client at the `loft-lsp` binary for `*.loft` files.
- **In the browser** — the playground runs loft with nothing installed.

2.0.12. Next steps

- Read the language overview starting with Keywords and control flow.
- See vs Rust if you are coming from Rust.
- Browse the Standard Library reference for built-in functions.
- Check the Roadmap for what comes next.

3. vs Rust

Loft is inspired by Rust but designed for general-purpose scripting with a shorter learning curve and less ceremony. It trades some of Rust's safety guarantees and expressive power for a simpler mental model. This page documents the most important differences so that Rust programmers know exactly what they gain and what they give up.

3.0.1. Variables — no let or mut

```
x = 42;
x += 1;
const y = 42;   // opt-in: a write to it is a compile error
```

```
let mut x = 42;
x += 1;
let y = 42;     // immutable by default; compiler-enforced
```

Upside Less boilerplate. No need to decide upfront whether a variable is mutable. Variables are mutable by default, so refactoring is frictionless. Use const to say a value must not change — the compiler refuses every write it forbids.

Downside Immutability is opt-in, not the default. Rust makes variables immutable unless you write mut, so every binding is protected for free; in loft only the ones you mark const are.

3.0.2. Null instead of Option<T>

```
struct Point {
    label: text?,         // `?`: may be absent
    x: integer,           // never null — the default
}
p = Point { x: 1 };
assert(p.label == null, "absent");
```

```
struct Point {
    label: Option<String>,  // explicit absence
    x: i32,                 // always present
}
let p = Point { label: None, x: 1 };
assert!(p.label.is_none());
```

Upside Natural for database records where fields are often absent. No Some(x)/None wrapping. Conditionals on null are concise: if v == null.

Downside The absence is in the type, as in Rust, but nothing forces you to discharge it: label can be read straight and answers null, where Rust's Option makes every call site say what it does about None. What loft gives you instead is ?? and ? at the point of use, plus a warning when a nullable value flows into a slot that cannot hold one. (Older code may carry not null on a field; it is deprecated and has no effect, because a type is non-null by default now.)

3.0.3. Parameter mutability — const and & instead of ownership

```
// a plain collection parameter is already shared — the append reaches the caller
fn push_two(v: vector<integer>) {
    v += [1, 2];  // caller sees the change
}
// const: read-only (a write through it is a compile error)
fn count(v: const vector<integer>) -> integer {
    len(v)
}
// &: explicit write-back for primitives
fn add_to(n: &integer, delta: integer) {
    n += delta;
}
```

```
fn push_two(v: &mut Vec<i32>) {
    v.push(1);
    v.push(2);
}
fn count(v: &Vec<i32>) -> usize {
    v.len()
}
fn add_to(n: &mut i32, delta: i32) {
    *n += delta;
}
```

Upside No borrow-checker errors. No lifetime annotations. No ownership transfer to reason about. Struct field mutations through a parameter are always visible to the caller. A collection parameter is SHARED with no annotation at all — field writes, element writes, appends, removes and clears all reach the caller. & adds exactly one thing on top of that: whole-value replacement, v = [...], seen by the caller. A const parameter is read-only: a write through it, or passing it on to a parameter that is not const, is a compile error.

Downside The compile-time guarantees are much weaker. Rust's borrow checker statically proves no aliased mutations, no dangling references, and no data races — before the program ever runs. Loft's const covers the name it marks and nothing more; aliasing is otherwise unchecked at compile time, and the engine's Rust runtime is what truly enforces memory safety.

3.0.4. ^ is XOR; ** or pow() for exponentiation

```
sq   = 2 ** 10;          // ** exponentiation — integers: 1024
area = PI * r ** 2.0;    // floats too; pow(r, 2.0) also works
bits = a | b;            // bitwise OR
mask = a & b;            // bitwise AND
xor  = a ^ b;            // bitwise XOR
```

```
let sq   = 2i32.pow(10);   // 1024 — no ** operator in Rust
let area = PI * r.powi(2); // or r.powf(2.0)
let bits = a | b;          // bitwise OR
```

```
let mask = a & b;          // bitwise AND
let xor  = a ^ b;          // bitwise XOR
```

Upside Loft has a ** exponentiation operator that works on both integers (2 ** 10 == 1024) and floats (2.0 ** 3.0 == 8.0); pow(base, exp) is also available for float/single. ^ behaves exactly as Rust developers expect — it is bitwise XOR. Bitwise operator precedence matches Rust and C: | (loose) ⟶ ^ ⟶ & ⟶ shifts ⟶ arithmetic (tight).

Downside The ** operator is loft-specific — Rust has no exponentiation operator and reaches for methods instead (.pow(), .powi(), .powf()), so the muscle memory does not transfer. And ^ is XOR in both languages, never exponentiation — a trap for anyone reaching for it as a power operator out of habit.

3.0.5. No loop keyword — an infinite loop is while true

```
while !done() { step(); }  // while works

while true {           // infinite until a break
    if should_exit() { break; }
    step();
}
```

```
while !done() { step(); }

loop {           // truly infinite — no bound needed
    if should_exit() { break; }
    step();
}
```

Upside while condition {} works exactly as expected. Every for loop has an iteration variable, making it easy to add index tracking or a cycle limit without restructuring.

Downside There is no loop {} keyword: an unconditionally infinite loop is spelled while true {}, which reads the same and ends only at a break or a return.

3.0.6. Filtered loops — for … if …

```
for x in items if x > 0 {
    total += x;
}
```

```
for x in items.iter().filter(|&&x| x > 0) {
    total += x;
}
```

Upside Reads like natural language. No closure syntax, no double-reference in the filter predicate. v#remove inside a filtered loop also safely removes the current element while iterating — something that requires careful index management in Rust.

Downside The inline if filter is limited to the loop it lives in. For multi-stage pipelines, loft offers map(v, f), filter(v, pred), and reduce(v, init, f) as composable higher-order functions — see section 11. Rust's lazy iterator chain (.filter().map().take_while()) avoids intermediate allocations; loft's higher-order functions allocate a new vector at each stage.

3.0.7. Loop attributes: #first, #count, #index

```
for x in 1..=5 {
    if !x#first { b += ", "; }
    b += "{x#count}:{x}";
}
```

```
for (i, x) in (1..=5).enumerate() {
    if i != 0 { b.push_str(", "); }
    b.push_str(&format!("{i}:{x}"));
}
```

Upside No tuple destructuring needed. x#first reads as "is this the first x", which is self-explanatory. Eliminates helper counter variables for the common pattern of comma-separated output.

Downside The #attribute syntax is unique to loft and unfamiliar to everyone else. Rust's .enumerate() composes freely with other adapters; loft's attributes exist only on a for loop's own variable.

3.0.8. Named loop break — x#break

```
for x in 1..5 {
    for y in 1..5 {
        if x * y >= 6 { x#break; }
        pairs += 1;
    }
}
```

```
'outer: for x in 1..5 {
    for y in 1..5 {
        if x * y >= 6 { break 'outer; }
    }
}
```

Upside Reuses the existing loop variable name as the label. No separate 'label declaration at the loop head. The name x#break reads as "break out of the loop over x".

Downside Rust's lifetime-label syntax 'outer is explicit and visually separate from the loop body. x#break is easy to confuse with the loop attributes x#first and x#count, which have completely different semantics.

3.0.9. Methods by self name, not impl blocks

```
fn greet(self: Person) -> text {
    "Hello, {self.name}!"
}
```

```
fn name_len(self: const Person) -> integer {
    len(self.name)  // const: read-only access guaranteed
}
p.greet();      // dot syntax
greet(p);       // free-function call — identical
```

```
impl Person {
    fn greet(&self) -> String {
        format!("Hello, {}!", self.name)
    }
    fn name_len(&self) -> usize {
        self.name.len()
    }
}
p.greet();      // only dot syntax
```

Upside No impl block ceremony. Methods and free functions are the same thing — just a naming convention. Functions can be added to any type from any file without modifying the original struct definition, similar to extension methods. Mark the first parameter const to guarantee the method never modifies the receiver — analogous to Rust's &self.

Downside No consuming self. Plain (non-const) methods behave like &mut self — the caller's value is always mutably aliased.

3.0.10. Polymorphic enum dispatch

```
enum Shape {
    Circle { r: float },
    Rect   { w: float, h: float }
}
fn area(self: Circle) -> float { PI * pow(self.r, 2.0) }
fn area(self: Rect)   -> float { self.w * self.h }

s.area();   // dispatches on the runtime variant
```

```
enum Shape {
    Circle { r: f64 },
    Rect   { w: f64, h: f64 },
}
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle { r } => PI * r * r,
        Shape::Rect { w, h } => w * h,
    }
}
```

Upside Each variant's behaviour lives in its own small function — easy to read and extend. Adding a new type of shape only requires a new fn area(self: NewShape) without modifying the dispatch site. Feels like OOP method overriding without the inheritance.

Downside A variant needs its own implementation only once something calls the method on a value held as the enum — that call is refused until every variant has one or the enum has a fallback (fn area(self: Shape)) — so a missing one surfaces at the first such call, not where the variant is declared. An empty-body stub (fn area(self: NewShape) -> float { }) answers the type's default, 0.0, which a caller cannot tell from a computed one; Rust's match makes that decision visible at every site.

3.0.11. Closures — same-scope capture works

```
// Capture works when called in the same scope:
offset = 10;
shift = fn(x: integer) -> integer { x + offset };
assert(shift(5) == 15, "shift");

// Capturing and non-capturing lambdas both work with map/filter/reduce:
shifted = map([1, 2, 3], shift);   // carries `offset` with it
evens   = filter([1, 2, 3, 4], |x| { x % 2 == 0 });

// Cross-scope: returning a capturing lambda works:
fn make_adder(n: integer) -> fn(integer) -> integer {
    fn(x: integer) -> integer { x + n }
}
```

```
// Closure captures context freely, any scope:
let offset = 5;
let shift = |x| x + offset;
assert_eq!(shift(5), 10);

// Works as argument to higher-order functions:
let shifted: Vec<i32> = v.iter()
    .map(|x| x + offset)
    .collect();

// Returning a capturing closure:
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}
```

Upside Capture works for every type — scalars, text, structs and every collection kind — with no capture modes (move vs borrow), no Fn/FnMut/FnOnce trait bounds, and no lifetime annotations. The compiler picks the mode from the type: a scalar or text is copied at definition time, a struct or collection is shared with the enclosing scope, and a scalar the closure writes to is shared as well, so an accumulator needs no declaration. Both long-form (fn(x: integer) -> integer { x * 2 }) and short-form (|x| { x * 2 }) lambdas are supported. Named function references (a bare name: map(v, double)) are compile-checked.

Downside You do not choose the capture mode, so a case Rust expresses by picking one has no spelling here. Three limits have no Rust counterpart: a capturing closure cannot be stored in a collection (a struct field holds one fine), a closure cannot write a captured &integer or &text parameter (it holds a

copy), and a scalar the closure writes to may be captured by only one closure — the cure for the last two is a local the closure updates, or state held in a struct and captured.

3.0.12. Generic functions — inferred, with structural interface bounds

```
// Pass-through generics work:
fn identity<T>(x: T) -> T { x }
identity(42);          // integer
identity("hello");     // text

// Interface bounds work, and are satisfied structurally:
fn largest<T: Ordered>(a: T, b: T) -> T { if a > b { a } else { b } }
largest(3, 9);         // 9
largest("ada", "bob"); // "bob" — same function

// A per-type version is still what you write when the bound does not fit:
fn max_int(a: integer, b: integer) -> integer {
    if a > b { a } else { b }
}
```

```
fn identity<T>(x: T) -> T { x }

fn max<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}
// One function works for all comparable types.
```

Upside Basic generic functions (identity, pick_second, wrappers) work out of the box. Collections (vector<T>, hash<T>, sorted<T>) are generic at the engine level, covering the most common need. No dyn Trait, impl Trait, or associated types to learn.

Downside Interface bounds (<T: Ordered>, <T: Addable>, <T: Printable>) are structural and static-dispatch only — a type satisfies an interface implicitly by defining the required operators, and there are no trait objects (dyn Trait), associated types, or blanket impls. Rust's nominal traits and dynamic dispatch reach cases loft's static interfaces cannot.

3.0.13. String formatting — embedded expressions

```
msg  = "Hi {name}, score: {score:6.2}";   // width.precision
rank = "seed {n:+}";                      // sign shows on integers: "seed +42"
prec = "{value:.2}";                      // precision only
hex  = "{n:#x}";
list = "{for x in 1..4 {x*2}}";          // [2,4,6]
nest = "{\"hello\"}";                     // nested string literal — works
```

```
let msg  = format!("Hi {name}, score: {:6.2}", score);
let rank = format!("seed {:+}", n);
let prec = format!("{:.2}", value);
let hex  = format!("{:#x}", n);
```

```
// no in-string iteration; needs a separate collect step
// nested string literals in format args are fine in Rust
```

Upside Concise — no format!() call, no separate variable. Full format expressions (arithmetic, slices, for comprehensions) can appear directly inside {}. Specifiers mirror Rust: :width, :.precision, :width.precision, sign (+), radix (#x, #o, b), alignment (<, >, ^), and zero-padding (0N) all work. The sign (+) and zero-pad (0N) flags apply to floats too: {5.25:+} is +5.25 and {5.25:08} is 00005.25.

Downside Unknown radix letters in specifiers are compile-time errors (e.g. :5z or :5B are both rejected). Radix letters are case-sensitive: valid ones are b, o, x/X and d on an integer, and j/json on a struct, vector or enum — uppercase B or O produce an error, and there is no exponent form (e). Applying a numeric specifier to an incompatible type (such as :x or zero-padding on a text value) is a compile-time error.

3.0.14. Built-in parallel for-loops — par(...)

```
fn double(r: const Score) -> integer { r.value * 2 }

sum = 0;
for item in scores par(b=double(item), 4) {
    sum += b;    // b is double(item), computed in parallel
}                // results arrive in original order

// Method form:
for item in scores par(b=item.twice(), 4) { sum += b; }
```

```
use rayon::prelude::*;

fn double(r: &Score) -> i32 { r.value * 2 }

let sum: i32 = scores.par_iter()
    .map(double)
    .sum();      // order is not guaranteed without extra work

// rayon is an external crate; add to Cargo.toml first
```

Upside Built into the language — no external crate, no Cargo.toml edit. The compiler validates the worker function signature at the call site. Results are delivered in the original vector order. The thread count is set per call, making it easy to tune for the hardware at hand.

Downside A worker is a named function, not a closure: context from the calling scope reaches it only as extra scalar arguments, because each worker runs on its own copy of the heap and cannot reach a struct or vector of the caller's. Nor may it touch shared state (no println, no file access) — a rule the compiler does not check. For multi-stage transformations, use map() / filter() / reduce() sequentially — each stage allocates a new intermediate vector.

4. vs Python

Loft and Python share a similar surface syntax — assignment without type annotations, brace-free expressions, and names that just work. But loft is statically typed and compiled to bytecode, while Python is dynamically typed and interpreted. This page documents the most important differences so that Python programmers know exactly what they gain and what they give up.

4.0.1. Variables — static types inferred at first assignment

```
x = 42;          // integer — fixed at first assignment
x += 1;
name = "Bob";    // text — a different variable, not a rebind
// x = "oops";   // compile error: cannot assign text to integer
```

```
x = <span class="nm">42</span>           <span class="cm"># int inferred; can be
rebound to any type</span>
x += <span class="nm">1</span>
name = <span class="st">"Bob"</span>
x = <span class="st">"oops"</span>       <span class="cm"># fine — x is now a
str</span>
```

Upside Type errors are caught before the program runs. Renaming or reshaping data structures surfaces mismatches immediately. There is no equivalent of Python's runtime TypeError, AttributeError, or "NoneType has no attribute X" crash — those classes of bug are detected at compile time.

Downside Types are fixed at first assignment and cannot change. Python's dynamic typing genuinely speeds up prototyping: a function that accepts anything, a list that holds mixed types, or a variable that starts as an integer and becomes a float later are all natural in Python and impossible in loft. Adding a type annotation layer (mypy, pyright) gives Python static safety without giving up its flexibility.

4.0.2. Null — structured absence, not a universal wildcard

```
struct User {
    email: text?,            // `?`: may be absent
    age:   integer,          // never null — the default
}
u = User { age: 30 };
if u.email == null { print("no email\n"); }
u.age = null;  // warns: the slot holds null anyway
```

```
<span class="kw">from</span> dataclasses <span class="kw">import</span> dataclass
<span class="kw">from</span> typing <span class="kw">import</span> Optional

<span class="en">@dataclass</span>
<span class="kw">class</span> <span class="en">User</span>:
    age: <span class="bi">int</span>
    email: Optional[<span class="bi">str</span>] = <span class="kw">None</span>
<span class="cm"># explicit Optional</span>
```

```
u = <span class="fn-call">User</span>(age=<span class="nm">30</span>)
<span class="kw">if</span> u.email <span class="kw">is</span> <span
class="kw">None</span>:
    <span class="bi">print</span>(<span class="st">"no email"</span>)
u.age = <span class="kw">None</span>  <span class="cm"># allowed at runtime; mypy
catches this</span>
```

Upside Nullability is part of the type and readable at a glance, and the default is the safe one: a field is non-null unless it says ?, so absence is something you opt INTO. No Optional[T] wrapping is needed; the comparison v == null is natural, and ?? supplies a fallback at the point of use.

Downside The default is the safe one — a field is non-null unless its type says ? — but nothing forces you to discharge a nullable at the point of use, and writing null into a non-null slot is a warning rather than a refusal. Python with mypy and an Optional[T] annotation reports that write as an error, where loft only warns.

4.0.3. Structs vs classes and dicts

```
struct Point { x: float, y: float }
p = Point { x: 1.0, y: 2.0 };
p.x += 0.5;
// p.z;          // compile error: no field z on Point
// p.x = "hi";   // compile error: cannot assign text to float

fn distance(self: const Point) -> float {
    sqrt(self.x * self.x + self.y * self.y) ?? 0.0
}
```

```
<span class="kw">from</span> dataclasses <span class="kw">import</span> dataclass
<span class="kw">import</span> math

<span class="en">@dataclass</span>
<span class="kw">class</span> <span class="en">Point</span>:
    x: <span class="bi">float</span>
    y: <span class="bi">float</span>

    <span class="kw">def</span> <span class="fn-call">distance</span>(self) ->
<span class="bi">float</span>:
        <span class="kw">return</span> math.<span class="fn-call">sqrt</
span>(self.x**<span class="nm">2</span> + self.y**<span class="nm">2</span>)

p = <span class="fn-call">Point</span>(x=<span class="nm">1.0</span>, y=<span
class="nm">2.0</span>)
p.z        <span class="cm"># AttributeError at runtime (or dict: KeyError)</
span>
p.x = <span class="st">"hi"</span> <span class="cm"># allowed at runtime; mypy
catches this</span>
```

Upside Field access is checked at compile time — typos in field names are caught before any code runs. Struct memory layout is fixed and unboxed; integer and float fields live directly in memory with no heap allocation overhead. Methods are ordinary named functions — they can be added from any file, at any time, without modifying the struct definition.

Downside No inheritance, no properties or descriptors. Printing needs no __repr__ — "{p}" renders every field — and == compares every field with no __eq__ written, as @dataclass's generated one does (&a == &b asks what Python's is asks). A struct can overload operators by defining them (OpAdd for +, OpEq for ==, OpLt for <). Python also supports plain dicts as lightweight records, which is often more convenient for ad-hoc data.

4.0.4. while loop — yes, and while true; no while … else

```
while !ready() { step(); }  // works

while len(queue) > 0 {
    process(queue[0]);
    queue.remove(0);   // `#remove` is for a loop variable
}

// An infinite loop, left by a break:
while true {
    if should_exit() { break; }
    step();
}
```

```
<span class="kw">while</span> <span class="kw">not</span> <span class="fn-
call">ready</span>():
    <span class="fn-call">step</span>()

<span class="kw">while</span> queue:
    <span class="fn-call">process</span>(queue.<span class="fn-call">pop</
span>(<span class="nm">0</span>))

<span class="kw">while</span> <span class="kw">True</span>:          <span
class="cm"># truly infinite</span>
    <span class="kw">if</span> <span class="fn-call">should_exit</span>(): <span
class="kw">break</span>
    <span class="fn-call">step</span>()
```

Upside while condition { } works exactly as expected. Filtered loops (for x in v if pred(x)) and loop attributes (x#first, x#count) add expressive power to for without needing a separate loop form.

Downside No while … else (executes when the condition becomes false without a break), a Python pattern with no clean equivalent in loft.

4.0.5. Polymorphic enum dispatch vs isinstance

```
enum Shape {
    Circle { r: float },
```

```
    Rect   { w: float, h: float }
}
fn area(self: Circle) -> float { PI * pow(self.r, 2.0) }
fn area(self: Rect)   -> float { self.w * self.h }

s.area();   // dispatches on the runtime variant
```

```
<span class="kw">import</span> math
<span class="kw">from</span> dataclasses <span class="kw">import</span> dataclass

<span class="en">@dataclass</span>
<span class="kw">class</span> <span class="en">Circle</span>: r: <span
class="bi">float</span>
<span class="en">@dataclass</span>
<span class="kw">class</span> <span class="en">Rect</span>:   w: <span
class="bi">float</span>; h: <span class="bi">float</span>

<span class="kw">def</span> <span class="fn-call">area</span>(s):
    <span class="kw">if</span> <span class="bi">isinstance</span>(s, Circle):
<span class="kw">return</span> math.pi * s.r ** <span class="nm">2</span>
    <span class="kw">if</span> <span class="bi">isinstance</span>(s, Rect):
<span class="kw">return</span> s.w * s.h

<span class="cm"># Python 3.10+ structural pattern matching:</span>
<span class="kw">match</span> s:
    <span class="kw">case</span> <span class="en">Circle</span>(r=r): <span
class="kw">return</span> math.pi * r ** <span class="nm">2</span>
    <span class="kw">case</span> <span class="en">Rect</span>(w=w, h=h): <span
class="kw">return</span> w * h
```

Upside Each variant's behaviour lives in its own small, named function — easy to read, easy to navigate in an editor. Adding a new shape only requires a new fn area(self: NewShape) with no changes to existing code or a central dispatch function. Calling a method on a value held as the enum is refused until every variant has an implementation, or the enum has a fallback (fn area(self: Shape)).

Downside A missing implementation surfaces at the first call through the enum, not where the variant is declared, and an empty-body stub answers the type's default rather than anything like None. Python 3.10+ structural pattern matching (match/case) fully destructs the matched value and is exhaustive when case _: is present. Python also supports inheritance-based polymorphism (class Circle(Shape)) and abstract base classes, giving far richer dispatch options.

4.0.6. String formatting — embedded expressions

```
msg  = "Hi {name}, score: {score:8.2}";
sign = "seed {n:+}";              // sign shows on integers: "seed +42"
hex  = "{n:#x}";
list = "{for x in 1..4 {x*2}}";   // [2,4,6]
pad  = "{count:08}";              // zero-padded (integers)
```

```
msg  = <span class="st">f"Hi {name}, score: {score:8.2f}"</span>
sign = <span class="st">f"seed {n:+}"</span>
hex  = <span class="st">f"{n:#x}"</span>
<span class="cm"># no loop in f-string; use a join:</span>
lst  = <span class="st">","</span>.<span class="fn-call">join</span>(<span
class="bi">str</span>(x*<span class="nm">2</span>) <span class="kw">for</span> x
<span class="kw">in</span> <span class="bi">range</span>(<span class="nm">1</
span>, <span class="nm">4</span>))  <span class="cm"># "2,4,6"</span>
pad  = <span class="st">f"{count:08}"</span>
```

Upside All loft strings are implicitly format strings — no f prefix needed. Inline for loops inside {} produce a bracketed list ([2,4,6]) without a separate join. Format specifiers mirror Python's f-string mini-language: width, sign, alignment and zero-padding work on integers and floats alike, precision on floats, and radix (#x, #o, b) on integers: {5.25:+} is +5.25 and {5.25:08} is 00005.25.

Downside Both accept an expression inside the braces — a method call with a specifier ({obj.method():.2}), an if expression ({if ok {"yes"} else {"no"}}), a join. Python also supports conversion flags (!r for repr, !s for str, !a for ASCII) and the = debug specifier ({x=} prints x=42); loft has none of these.

4.0.7. Collections — typed and built-in

```
nums:   vector<integer> = [1, 2, 3];
lookup: hash<Entry[key]> = [];
lookup += [Entry { key: "key", value: "value" }];
scores: sorted<Score[user]> = [];
scores += [Score { user: "ann", points: 95 }];  // keyed insert: binary search,
kept in key order

// Element type is enforced at compile time:
// nums += ["x"];  // compile error: expected integer
```

```
nums   = [<span class="nm">1</span>, <span class="nm">2</span>, <span
class="nm">3</span>]          <span class="cm"># list — any element type</span>
lookup = {}                 <span class="cm"># dict — any key/value type</span>
lookup[<span class="st">"key"</span>] = <span class="st">"value"</span>

<span class="cm"># sorted dict requires an external package:</span>
<span class="kw">from</span> sortedcontainers <span class="kw">import</span>
SortedDict
scores = <span class="fn-call">SortedDict</span>()
scores[user] = <span class="nm">95</span>

nums.<span class="fn-call">append</span>(<span class="st">"x"</span>)
<span class="cm"># allowed at runtime</span>
```

Upside All collection types — vector, hash, and sorted map — are built in; no extra import or install needed. Element types are checked at compile time. The same [] indexing syntax works on all three.

for x in v if pred(x) { v#remove; } safely removes the current element while iterating — something Python requires careful index management for.

Downside Python's built-in dict and list are among the most heavily optimised data structures in any scripting runtime. Python also has set and frozenset (no loft equivalent), and ordered insertion semantics on dict (Python 3.7+), where a loft hash iterates in key order. Loft has comprehensions too ([for x in v if x > 0 { x * 2 }]), but they and the map() / filter() / reduce() higher-order functions build a whole vector at each stage, while Python's generators are lazy and allocation-free until consumed.

4.0.8. Closures — same-scope capture works

```
// Capture works when called in the same scope:
offset = 10;
shift = fn(x: integer) -> integer { x + offset };
assert(shift(5) == 15, "shift");

// Lambdas work with map/filter/reduce, capturing or not:
doubled = map([1, 2, 3], fn(x: integer) -> integer { x * 2 });
evens   = filter([1, 2, 3, 4], |x| { x % 2 == 0 });

// Capturing lambda with map:
offset = 10;
shifted = map(nums, fn(x: integer) -> integer { x + offset });
```

```
offset = <span class="nm">10</span>
shift = <span class="kw">lambda</span> x: x + offset  <span class="cm"># capture
works anywhere</span>
<span class="kw">assert</span> <span class="fn-call">shift</span>(<span
class="nm">5</span>) == <span class="nm">15</span>

nums = [<span class="nm">1</span>, <span class="nm">2</span>, <span
class="nm">3</span>, <span class="nm">4</span>, <span class="nm">5</span>]
doubled = <span class="bi">list</span>(<span class="bi">map</span>(<span
class="kw">lambda</span> x: x * <span class="nm">2</span>, nums))
evens   = <span class="bi">list</span>(<span class="bi">filter</span>(<span
class="kw">lambda</span> x: x % <span class="nm">2</span> == <span class="nm">0</
span>, nums))

<span class="cm"># capturing lambda passed to map:</span>
shifted = [x + offset <span class="kw">for</span> x <span class="kw">in</span>
nums]
```

Upside Capture works for every type — scalars, text, structs and every collection kind — with no capture modes and no scope surprises. The compiler picks the mode from the type, so an accumulator needs no declaration the way Python's nonlocal does. Both long-form (fn(x: integer) -> integer { x * 2 }) and short-form (|x| { x * 2 }) lambdas are supported. Named function references (a bare name: map(v, double)) are compile-checked. Higher-order functions (map, filter, reduce) accept both lambdas and fn-refs.

Downside You do not choose the capture mode, and it is not the same for every type. A scalar or text the lambda only READS is copied at definition time, so a later mutation of the original does not reach it — where Python closes over the name and would see the new value. Structs and collections share the way Python does, and so does a scalar the lambda writes to. Three limits have no Python counterpart: a capturing closure cannot be stored in a collection (a struct field holds one fine), a lambda cannot write a captured &integer or &text parameter (it holds a copy), and a scalar the lambda writes to may be captured by only one lambda. A comprehension is the short form in both languages: [for x in v { x + offset }] in loft, [x + offset for x in v] in Python.

4.0.9. No exception handling — file errors use FileResult

```
// Logic errors: assert aborts the program
fn divide(a: float, b: float) -> float {
    assert(b != 0.0, "division by zero");
    a / b ?? 0.0
}

// Reading: check f#exists before using content
f = file("data.txt");
if f#exists {
    data = f.content();
    print("{len(data)} characters\n");
} else {
    print("file not found\n");
}

// Mutating ops return a FileResult enum; match on the bare variant
result = delete("old.txt");
match result {
    Ok               => print("deleted\n"),
    NotFound         => print("not found\n"),
    PermissionDenied => print("permission denied\n"),
    IsDirectory      => print("is a directory\n"),
    _                => print("other error\n"),
}

// Or just check success:
if !move("a.txt", "b.txt").ok() { print("rename failed\n"); }
```

```
<span class="cm"># Logic errors: raise an exception</span>
<span class="kw">def</span> <span class="fn-call">divide</span>(a: <span
class="bi">float</span>, b: <span class="bi">float</span>) -> <span
class="bi">float</span>:
    <span class="kw">if</span> b == <span class="nm">0</span>:
        <span class="kw">raise</span> <span class="fn-call">ValueError</
span>(<span class="st">"division by zero"</span>)
    <span class="kw">return</span> a / b

<span class="kw">try</span>:
```

```
    result = divide(x, y)
except ValueError as e:
    print(f"error: {e}")

# File errors: catch specific exception types
try:
    with open("data.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("file not found")

try:
    os.remove("old.txt")
except FileNotFoundError:
    print("not found")
except PermissionError:
    print("permission denied")
except IsADirectoryError:
    print("is a directory")
```

Upside File errors are represented as a typed FileResult enum — Ok, NotFound, PermissionDenied, IsDirectory, NotEmpty, Other — so every failure case is named and exhaustively matchable. No accidental exception swallowing, no hidden exit paths, no stack-unwinding overhead. The control flow of every loft function is straightforward to read.

Downside Logic errors (assert, panic) abort the entire program — there is no try/except to catch them. Python's exception hierarchy supports retry logic, cleanup via finally, and graceful degradation from arbitrary errors anywhere in the call stack — patterns that are not expressible in loft.

4.0.10. Built-in parallel for-loops — par(...)

```
fn score(item: const Record) -> integer { item.value * 2 }

total = 0;
for item in records par(s=score(item), 4) {
    total += s;   // results arrive in original order
}                 // 4 worker threads, no GIL
```

```
from concurrent.futures import ProcessPoolExecutor

def score(item): return item.value * 2

# ProcessPoolExecutor: bypasses GIL via separate processes
with ProcessPoolExecutor(max_workers=4) as ex:
```

```
    results = <span class="bi">list</span>(ex.<span class="fn-call">map</span>(score, records))
total = <span class="bi">sum</span>(results)

<span class="cm"># ThreadPoolExecutor: simpler but GIL limits CPU-bound work</span>
```

Upside Built into the language — no import, no boilerplate. The GIL does not apply; worker threads run on separate OS threads inside the same process. Results arrive in the original vector order. The compiler validates the worker function signature at the call site. The thread count is set per call, making it easy to tune for the hardware.

Downside A worker is a named function, not a lambda, and context from the calling scope reaches it only as extra scalar arguments: each worker runs on its own copy of the heap, so it cannot reach a struct or vector of the caller's. Python's ProcessPoolExecutor works with any picklable object and gives full control over timeouts, cancellation, and error propagation.

4.0.11. Generic functions — interface bounds, no duck typing

```
// Pass-through generics work:
fn identity<T>(x: T) -> T { x }
identity(42);          // integer
identity("hello");     // text

// Interface bounds work, and are satisfied structurally:
fn largest<T: Ordered>(a: T, b: T) -> T { if a > b { a } else { b } }
largest(3, 9);         // 9
largest("ada", "bob"); // "bob" — same function

// A per-type version is still what you write when the bound does not fit:
fn max_int(a: integer, b: integer) -> integer {
    if a > b { a } else { b }
}
```

```
<span class="cm"># Duck typing: works for any T that supports ></span>
<span class="kw">def</span> <span class="fn-call">max_val</span>(a, b):
    <span class="kw">return</span> a <span class="kw">if</span> a > b <span class="kw">else</span> b

<span class="cm"># With type hints (Python 3.12+):</span>
<span class="kw">from</span> typing <span class="kw">import</span> TypeVar
T = <span class="fn-call">TypeVar</span>(<span class="st">"T"</span>)
<span class="kw">def</span> <span class="fn-call">max_val</span>(a: T, b: T) ->
T:
    <span class="kw">return</span> a <span class="kw">if</span> a > b <span class="kw">else</span> b
```

Upside Basic generic functions (pass-through, wrapping, forwarding) work out of the box. Collections (vector<T>, hash<T>, sorted<T>) are generic at the engine level, covering the most common need without any user-visible type parameter syntax.

Downside Operations on T require an explicit interface bound (<T: Ordered> for comparison, <T: Addable> for arithmetic, <T: Printable> for display); unbounded T stays opaque. Python's duck typing lets one function work on any type that supports the needed operation (len, >, +) with no annotation, and Python 3.12 TypeVar/Protocol add optional static checking on top.

4.0.12. Function signatures — defaults and named args, but no *args

```
fn connect(host: text, port: integer = 80, tls: boolean = true) -> text
{ "{host}:{port} tls={tls}" }

connect("localhost");                // uses defaults
connect("localhost", tls: false);    // named arg — skips port
connect("localhost", 443, false);    // positional — all explicit
```

```
def connect(host: str, port: int = 80, tls: bool = True):
    ...

connect("localhost") # uses defaults
connect("localhost", tls=False) # keyword arg — skips port

def log(*args, **kwargs): ...   # variadic
```

Upside Default parameter values and named arguments work similarly to Python. connect("host", tls: false) skips defaulted middle parameters. Every call site is readable without jumping to the function definition.

Downside No *args or **kwargs — variadic dispatch must use a vector parameter. Python's flexible argument syntax also enables decorator patterns and configuration-driven code that is harder to express in loft.

4.0.13. Ecosystem — minimal standard library

```
// `use mylib;` loads a library — a .loft file beside the script,
// or a package the manifest declares under [dependencies].
// The standard library ships inside the interpreter binary, and
// `loft install` fetches the rest from a signed registry.

// Built-in: text, math, file I/O, collections,
//           logging, threading, lexer/parser
```

```
<span class="kw">import</span> numpy <span class="kw">as</span> np          <span class="cm">
class="cm"># numerical arrays / SIMD</span>
<span class="kw">import</span> pandas <span class="kw">as</span> pd         <span class="cm">
class="cm"># dataframes</span>
<span class="kw">import</span> requests            <span class="cm">
span>
<span class="kw">import</span> flask               <span class="cm">
framework</span>
<span class="kw">import</span> sqlalchemy          <span class="cm">
ORM</span>
<span class="kw">import</span> scikit_learn        <span class="cm">
learning</span>
<span class="cm">
<span class="cm">
```

Upside The interpreter is a single self-contained binary with the standard library inside, so a program that needs nothing else deploys by copying one file, with no virtual environment. What it does need, loft.toml declares and loft install fetches from a signed registry. The built-in library covers text manipulation, math, file I/O, typed collections, parallel execution, and a full lexer/parser framework; image handling is the registry's imaging package.

Downside Python's ecosystem is its defining advantage. NumPy, pandas, scikit-learn, TensorFlow, requests, Flask, SQLAlchemy, pytest, and hundreds of thousands of other packages are not available to loft programs, whose registry holds a few dozen — games and graphics first, with web, crypto, regex and markdown beside them. Data science, and most integration and protocol work, still means writing what Python gets from a single pip install. For these domains, loft is the wrong tool.

4.0.14. Exponentiation with ** or pow(); ^ is XOR

```
area  = PI * r ** 2.0;      // ** exponentiation (like Python)
bits  = a | b;              // bitwise OR
xor   = a ^ b;              // bitwise XOR — not exponentiation
cube  = x ** 3;             // integer cube — exact: 2 ** 10 == 1024
```

```
<span class="kw">import</span> math
area  = math.pi * r ** <span class="nm">2</span>   <span class="cm">
exponentiation</span>
bits  = a | b              <span class="cm">
xor   = a ^ b              <span class="cm">
cube  = x ** <span class="nm">3</span>             <span class="cm">
cube — exact</span>
cube  = x ** (<span class="nm">1</span>/<span class="nm">3</span>)         <span class="cm">
class="cm"># cube root as float</span>
```

Upside Loft's ** exponentiation operator matches Python's — 2 ** 10 == 1024 on integers, 2.0 ** 3.0 == 8.0 on floats (pow(base, exp) is available too). ^ behaves as bitwise XOR in both loft and Python — no mismatch. Bitwise operator precedence matches: | (loose) ⟶ ^ ⟶ & ⟶ shifts ⟶ arithmetic (tight).

Downside Python's ** additionally handles complex numbers, and its three-argument pow(base, exp, mod) form computes modular exponentiation efficiently; loft has neither. Loft's pow() function itself operates on float/single only — use the ** operator for integer exponentiation.

5. Keywords

This page covers the building blocks that shape how your program makes decisions and repeats work: conditions, loops, breaks, and a few loop helpers that make common patterns shorter. Every example is verified: most with an assert naming the exact expected result, and the first with a panic that would fire if the rule it shows did not hold.

```
fn main() {
```

5.0.1. Conditionals

'if' runs a block only when its condition is true. If the condition is false the block is skipped entirely — nothing else happens. 'panic' stops the program immediately with a message. During development it is a clear way to mark situations that should never occur.

```
  if 2 > 5 {
    panic("Incorrect test");
  }
```

Combine conditions with 'and' / 'or' (or their symbol equivalents '&&' / '||'). Note that '&' is the bitwise AND operator — it works on the individual 1s and 0s inside a number, which is different from the logical 'and' keyword that works on true/false values. An 'if/else' chain picks exactly one branch to execute.

```
  a = 12;
  if a > 10 and a & 7 == 4 {
    a += 1;
  } else {
    a -= 1;
  }
```

Text enclosed in double quotes can contain expressions inside curly braces — the expression is evaluated and its result is inserted into the text. 'assert' checks that its first argument is true; if it is false the second argument is printed as an error message.

```
  assert(a == 13, "Incorrect value {a} != 13");
```

5.0.2. if as an expression

An 'if', a 'match', and a bare '{ }' block all produce a value — the last expression inside them. That means you can use 'if/else' on the right-hand side of an assignment, picking between two values based on a condition. No ternary operator needed. Loops are the exception: 'for' and 'while' are statements, so 'x = for i in 1..3 { i }' does not compile.

```
  b = if a == 13 {
    "Correct"
  } else {
    "Wrong"
```

```
};
assert(b == "Correct", "Logic expression");
```

5.0.3. Iteration

‘for x in a..b’ loops over the integers starting at a and stopping before b (exclusive upper bound). Use ‘..=’ to include the upper bound. Here we sum 1 + 2 + 3 + 4 + 5 = 15.

```
t = 0;
for i in 1..6 {
  t += i;
}
assert(t == 15, "Total was {t} instead of 15");
```

‘rev(range)’ reverses the direction of any range. ‘1..=5’ is inclusive, so it visits 5, 4, 3, 2, 1 in that order. Multiplying each digit into a running total builds the number 54321.

```
t = 0;
for i in rev(1..=5) {
  t = t * 10 + i;
}
assert(t == 54321, "Result was {t} instead of 54321");
```

A range with no upper bound, ‘a..’, counts up until the body leaves it with ‘break’ or ‘return’ — the first square above 50 is 64, found at i = 8.

```
first = 0;
for i in 1.. {
  if i * i > 50 {
    first = i;
    break;
  }
}
assert(first == 8, "First was {first} instead of 8");
```

5.0.4. The other loop: while

Loft has two loop statements: ‘for’, which walks a range or a collection, and ‘while’, which repeats as long as its condition holds. A ‘for’ over a BOUNDED range carries its own end, so it is the one to reach for; a ‘while’ does not, and ‘while true { }’ runs until something stops it. Because nothing bounds a ‘while’, its body must move the condition towards false — here ‘n += 1’ is what ends the loop. This is the same sum as the ‘for’ above, written the other way.

```
n = 1;
t = 0;
while n <= 5 {
  t += n;
  n += 1;
```

```
}
assert(t == 15, "while summed to {t} instead of 15");
```

5.0.5. Nested loops and break

'break' exits the nearest enclosing loop immediately. To exit an outer loop from inside an inner one, write 'outerVar#break'. Here x#break leaves both loops when the product x*y reaches 16.

```
b = "";
for x in 1..5 {
  for y in 1..5 {
    if y > x {
      break;
```

this breaks the inner y loop

```
    }
    if x * y >= 16 {
      x#break;
    }
    if len(b) > 0 {
      b += "; ";
    }
    b += "{x}:{y}";
  }
}
assert(b == "1:1; 2:1; 2:2; 3:1; 3:2; 3:3; 4:1; 4:2; 4:3", "Incorrect sequence
'{b}'");
```

5.0.6. Skipping an iteration: continue

'continue' is the sibling of 'break': instead of leaving the loop it skips the rest of the current iteration and moves on to the next value. Here we sum 1..=5 but skip 3, so the total is 1 + 2 + 4 + 5 = 12.

```
t = 0;
for i in 1..=5 {
  if i == 3 {
    continue;
  }
  t += i;
}
assert(t == 12, "continue skipped 3: total was {t} instead of 12");
```

5.0.7. Loop helpers: #first and #count

Inside a loop body you have access to some useful metadata about the current iteration. 'x#first' is true only for the very first element that passes the filter. 'x#count' is a zero-based index counting only the elements that were not filtered out. An 'if' clause directly after 'in range' filters which values enter the loop — here we skip every value where x % 3 == 1, keeping 2, 3, 5, 6, 8, 9.

```
  b = "";
  for x in 1..=9 if x % 3 != 1 {
    if !x#first {
      b += ", ";
    }
    b += "{x#count}:{x}";
  }
  assert(b == "0:2, 1:3, 2:5, 3:6, 4:8, 5:9", "Sequence '{b}'");
```

5.0.8. Formatting a loop inline

A for loop can appear inside a format string. The results are collected into a bracketed, comma-separated list. A format specifier after the closing brace is applied to every element — ':02' means at least 2 digits, zero-padded. This is handy for building compact representations on the fly.

```
  assert("a{for x in 1..7 {x*2}:02}b" == "a[02,04,06,08,10,12]b", "Format
range");
}
```

6. Texts

Text is one of the most frequently used types in any real program — for output, for reading input, for building messages. This page shows you how Loft stores and manipulates text, how to measure it, slice it, search it, and format it exactly the way you need.

The key thing to know upfront: Loft stores text as a sequence of bytes using UTF-8 encoding — the same format used on the web. Plain ASCII letters, digits, and punctuation each take exactly one byte. Accented letters, emoji, and many non-Latin scripts take 2–4 bytes. So there are two different counts, and which one you get depends on the operation: anything that takes a POSITION — indexing, slicing, 'find' — counts bytes, while 'len()' and iterating a text count characters. Positions in bytes are fast and simple; the thing to watch is mixing the two counts, which has its own section below.

```
fn main() {
```

6.0.1. Joining and measuring text

Use '+' to join two pieces of text into one. The result is a new value — neither of the originals is modified. Placing a value name inside curly braces inside a format string inserts the value as text. You can also control the width: '{a:4}' pads the value on the right to at least 4 characters.

```
    a = "1" + "2";
    assert("'{a:4}'" == "'12  '", "Formatting text");
    assert(len(a + "123") == 5, "Text length");
```

'len()' counts characters (Unicode code points), the human count: an emoji like '😃' is one character, a heart symbol '♥' is one, and each ASCII letter is one. For the UTF-8 byte length (bounds checks, byte indexing) use 'size()'.

```
    assert(len("😃") == 1, "Emoji is one character");
    assert(len("♥") == 1, "Heart is one character");
    assert(len("abc") == 3, "ASCII character count");
```

6.0.2. Do not mix the two counts

'len()' is a character count and 's[i]' is a byte index, so walking one with the other goes wrong on any text that is not pure ASCII: the loop runs too few times AND re-reads the bytes inside a multi-byte character. Over "Hi 😊!", 'for i in 0..len(s) { s[i] }' yields "Hi 😊😊" — the '!' is silently gone and the emoji appears twice. Nothing fails; the answer is just wrong. Iterate the characters with 'for c in text', or, when you really do mean bytes, walk '0..size(text)'. The compiler reports the mix for you as warning[text-index-char-bound].

```
    mixed = "Hi 😊!";
    assert(len(mixed) == 5, "five characters, not eight");
    assert(size(mixed) == 8, "eight bytes, not five");
```

6.0.3. Reading individual characters

Square brackets with a single byte index give you the character at that position. For plain ASCII text every byte is one character, so indexing by position is straightforward. Loft will return the full character even if it spans multiple bytes.

```
s = "ABCDE";
assert(s[0] == 'A', "Character at byte 0");
assert(s[2] == 'C', "Character at byte 2");
```

6.0.4. Slicing text

A slice ‘s[start..end]’ gives you the bytes from ‘start’ up to but not including ‘end’. You can omit the start to begin at 0, or omit the end to go all the way to the last byte. Loft snaps offsets to valid character boundaries so you can never accidentally cut a multi-byte character in half.

```
assert(s[0..2] == "AB", "Explicit start slice");
assert(s[..2] == "AB", "Open-start slice");
assert(s[1..4] == "BCD", "Sub-string by byte range");
assert(s[3..] == "DE", "Open-ended sub-string");
```

A negative end index counts backwards from the end of the text. ‘−1’ means “stop before the last byte”. Combined with a start offset this lets you trim a known suffix without knowing the exact length.

```
txt = "12😊🙃45";
assert(txt[2.. - 1] == "😊🙃4", "UTF-8 sub-string by byte range");
```

That counts BYTES, and the offset still snaps to a character boundary — so it trims a one-byte suffix like ‘5’ above, and leaves a multi-byte one whole. Here both −1 and −2 land inside the trailing emoji and snap back around it, returning the text unchanged. Trim by bytes only when you know the suffix is ASCII.

```
assert("ab😊"[.. - 1] == "ab😊", "a multi-byte suffix is not trimmed by -1");
assert("ab😊"[.. - 2] == "ab😊", "nor by -2");
```

6.0.5. Iterating over characters

A ‘for’ loop over a text value visits one character at a time, even when characters span multiple bytes. Two loop helpers give you position information without any extra code: ‘c#index’ is the byte offset where the current character starts. ‘c#next’ is the byte offset immediately after the current character ends. This makes it easy to build a byte-offset map or to collect characters starting from a specific position.

```
result = "";
positions = [];
nexts = [];
for c in "Hi 😊!" {
  positions += [c#index];
```

```
    nexts += [c#next];
    if c#index >= 3 { result += c; }
  }
  assert(result == "😊!", "Character iteration was {result}");
  assert("{positions}" == "[0,1,2,3,7]", "Character positions was {positions}");
  assert("{nexts}" == "[1,2,3,7,8]", "Character nexts was {nexts}");
```

6.0.6. Searching inside text

These built-in functions answer common “does this text contain…?” questions. ‘starts_with’ and ‘ends_with’ check the boundaries. ‘find’ returns the byte offset of the first match, or null if not found. ‘contains’ is true if the needle appears anywhere in the text. All positions are byte offsets, consistent with ‘size()’ and slicing.

```
  assert("something".starts_with("some"), "starts_with");
  assert("something".ends_with("thing"), "ends_with");
  assert("something".find("th") == 4, "find returns byte offset");
  assert("a longer text".contains("longer"), "contains");
```

‘trim()’ removes spaces and other whitespace from both ends of a text value. It is useful when reading user input or parsing text from a file.

```
  assert(trim("  hello  ") == "hello", "trim");
```

6.0.7. Escaping braces in format strings

Inside a format string ‘{‘ and ‘}’ are special: they introduce an interpolated expression. To include a literal brace in the output, double it: ‘{{‘ produces ‘{‘ and ‘}}’ produces ‘}’.

```
  brace_inner = "cd";
  assert("ab{{cd}}e" == "ab{{{brace_inner}}}e", "Escaping braces");
```

6.0.8. Aligning text in a fixed-width field

The ‘:’ specifier controls alignment and width. ‘<’ left-aligns the value and ‘>’ right-aligns it. The default depends on the TYPE: text is left-aligned (as ‘{a:4}’ and ‘{vr:6}’ here both show), while numbers are right-aligned, which is what lines a column of figures up. The width can be a constant or a small arithmetic expression — here ‘2+3’ evaluates to 5, giving a field of width 5.

```
  vr = "abc";
  assert("1{vr:<2+3}2{vr}3{vr:6}4{vr:>7}" == "1abc  2abc3abc   4    abc", "Text
alignment");
}
```

7. Integers

Numbers are at the heart of almost every program. This page covers the integer types Loft provides, the arithmetic and bitwise operations you can perform on them, how to convert between numbers and text, and what Loft does when something goes wrong — such as dividing by zero.

The default number type is 'integer': a 64-bit signed whole number, the same as i64 in Rust. It can hold values from about −9 quintillion to +9 quintillion (roughly ±9.2 × 10^18), so everyday counts and very large numbers both fit in the one type — there is no separate 'long'. For decimal (fractional) numbers, see the Float page.

```
fn main() {
```

7.0.1. Converting between numbers and text

Wrapping a value in '{...}' inside a string formats it as text. Going the other way, 'as integer?' parses a text value into a number. Write the '?': a text parse can fail, so a bare 'as integer' is refused at compile time ("a text parse `as integer` may fail, and a bare cast asserts it cannot"). The '?' is you accepting the failure case, and the result is null — not a crash — when the text does not parse.

```
  v = 4;
  assert("{v}" == "4", "Format integer as text");
  assert("123" as integer? == 123, "Parse text to integer");
  assert(!("abc" as integer?), "Unparseable text gives null");
```

7.0.2. Arithmetic and operator precedence

Loft follows standard mathematical precedence: '*' and '/' before '+' and '-'. Bitwise operators (<<, &, ^) have their own precedence — when mixing them with arithmetic, parentheses make your intent clear and avoid surprises. Note: '^' is XOR, not exponentiation. For powers use 'pow(base, exp)' — a FLOAT function, so 'pow(2, 3)' does not resolve: raise in floats and come back. It answers a NULLABLE float ('float?'), because a power can be uncomputable, so the trip back is a checked cast or a discharge:

```
pow(base, exp) as integer?           // the number, or null
(pow(base, exp) ?? 0.0) as integer   // your own default instead
```

A bare 'as integer' is refused ("cannot cast a possibly-null `float?` to the non-null `integer`") unless loft can prove the power is defined: a whole-number constant exponent, or a base it knows is not negative. That proof depends on the arguments, so it does not generalise — write the '?' and it works for any arguments.

```
  assert(1 + 2 * 4 == 9, "Multiplication before addition");
  assert(1 + 2 << 2 == 12, "Shift: (1+2) << 2 = 12");
  assert(0x0a8 & 15 == 8, "Bitwise AND masks low 4 bits");
  assert(42 ^ 0b111111 == 21, "Bitwise XOR");
  assert(105 % 100 == 5, "Modulus (remainder)");
```

Written with the '?' on purpose: it keeps compiling whatever numbers a reader puts in.

```
  pw_base = 2.0;
  assert(pow(pw_base, 3.0) as integer? == 8, "pow() for exponentiation");
  assert((pow(pw_base, 10.0) ?? 0.0) as integer == 1024, "…or discharge, then
cast");
```

‘abs()’ returns the absolute value — the distance from zero, always positive.

```
  assert(1 + abs(-2) == 3, "abs(-2) == 2");
```

7.0.3. Division by zero — produces null and keeps running

Most languages crash on division by zero. Loft does NOT: a divide (or modulo) by zero is uncomputable, so it produces null and execution CONTINUES — the spreadsheet model, where one bad cell shows an error but the rest still recalculate. This holds the same everywhere (development, test, and production) — one bad calculation never halts the run. Where the divisor is a literal zero in your source, loft can see it before the program runs and warns (next section). Where it is only zero at RUN time there is no warning — you get null, and ‘{12 / a}’ prints ‘null’. Recover with ‘??’, or test the result for null: nothing else tells you, so a divide whose divisor could be zero is one you handle rather than one you are reminded about.

```
  a = 2 * 2;
  a -= 4;
```

a is now 0

```
  assert(!(12 / a ?? null), "Division by zero gives null when defended with `??
null`");
```

7.0.4. Warning when you write a literal zero divisor

When the divisor is a literal 0 written directly in your source code, loft warns you while reading your code (before running it), because that is almost certainly a mistake:

```
n / 0   // warning: Division by constant zero
n % 0   // warning: Modulo by constant zero
```

Use a variable (like ‘a’ above) when you intentionally want null-on-zero division without a warning.

7.0.5. Embedding integers in text

```
  assert("a{12}b" == "a12b", "Integer in format string");
```

A full expression can appear inside ‘{…}’, not just a variable name.

```
  assert("a{1 + 2 * 3}b" == "a7b", "Expression in format string");
```

7.0.6. Number format specifiers

After a ‘:’ inside ‘{…}’ you can control how a number is displayed:

```
#x  — hexadecimal with '0x' prefix
o   — octal
b   — binary
+   — always show a sign (+ or -)
N   — minimum field width (space-padded on the left)
0N  — minimum field width (zero-padded on the left)
```

```
  assert("a{1+2+32:#x}b" == "a0x23b", "Hex format with 0x prefix");
  assert("{12:o}" == "14", "Octal");
  assert("{12:+4}" == " +12", "Sign and width");
  assert("{1:03}" == "001", "Zero-padded width");
  assert("{42:b}" == "101010", "Binary");
```

Hexadecimal literals in source code accept both lower and upper case digits.

```
  assert(0xff == 255, "Lowercase hex literal");
  assert(0xFF == 255, "Uppercase hex literal");
  assert(0x2A == 42, "Uppercase hex digit");
```

7.0.7. Large integers

Because ‘integer’ is 64-bit, values far past the 32-bit limit of 2 billion work directly — no separate type is needed. Arithmetic, comparisons, and the format specifiers above all behave the same at these magnitudes.

```
  big = 1000000000 * 5;
  assert(big == 5000000000, "Integers well past 2 billion");
  assert("{big}" == "5000000000", "Large integer formatted as text");
```

7.0.8. Common pitfall: integer overflow

A calculation whose result is too large to represent overflows — and in loft an overflow is uncomputable, so the result is null. It never wraps to a silently-wrong number, and it never crashes; execution continues with the null, exactly like divide-by-zero above. Check the result for null (or keep intermediate values in range) when multiplying or summing very large numbers.

A sized integer (‘u8’, ‘i16’, a field declared ‘integer limit(0, 255)’) has a smaller range, so it overflows sooner — and where it lands depends on one thing: whether the slot can hold null. A nullable one (‘u8?’) gets the null, the same as above. A non-nullable one has nowhere to put it, so it gets the type’s default instead — 0 for every range that includes zero. What you never get is a wrapped number.

```
  small: u8? = 250;
  small += 10;
  assert(small == null, "A nullable sized integer overflows to null");
  assert((small ?? 0) == 0, "…and '??' recovers from it, like any other null");
  fixed: u8 = 250;
  fixed += 10;
  assert(fixed == 0, "A non-nullable one takes its type's default instead");
}
```

8. Boolean

A boolean value is either ‘true’ or ‘false’. Booleans appear naturally wherever you make a decision: in ‘if’ conditions, loop filters, and comparisons. Declared as ‘boolean?’, a boolean gets a third state called ‘null’ — meaning “no value” — and any null behaves like false whenever a boolean is expected.

This design means you rarely need to write a separate null-check: ‘!x’ is true both when x is false and when x is null.

```
fn main() {
```

A comparison produces a boolean result directly. ‘!’ flips a boolean: true ⟶ false, false ⟶ true.

```
  assert(!(3 > 2 + 4), "3 is not greater than 6");
  assert(true as text == "true", "Convert boolean to text");
```

8.0.1. Logical operators: and / or

‘and’ (also ‘&&’) is true only when both sides are true. ‘or’ (also ‘||’) is true when at least one side is true. Both skip the right side when the left side already determines the result. For example, ‘false and X’ is always false, so X is never evaluated. This matters when the right side could produce null or has a side effect.

```
  assert(1 > 0 and 2 > 1, "Both conditions true");
  assert(1 > 2 or 3 > 2, "Second condition true");
  assert(!(1 > 2 && 3 > 2), "First false → whole 'and' is false");
  assert(!(1 > 2 || 3 > 4), "Both false → 'or' is false");
```

8.0.2. Null in boolean context

Division by zero (and other failed operations) produce null. Null in a boolean context is treated as false, so ‘!’ is true. This lets you write guard clauses without a separate null-check syntax:

```
if !result { ... handle missing value ... }
```

Even without `?? null`, a bare divide never halts: it yields null and execution continues, the same everywhere — development, test, and production.

The `?? null` is not what makes that safe, and it silences nothing. loft warns about a divisor that is a literal 0 in your source, while READING your code, and it warns whether or not you wrote `?? null`; a divisor that is only zero at RUN time is not warned about at all. Either way the result is null, and a null prints as ‘null’. Reach for `?? \<value\>` when you want a number — not to quiet a warning.

```
  zero = 0;
  assert(!(12 / zero ?? null), "null from division-by-zero is false-like when
defended");
  assert("{12 / zero}" == "null", "an undefended result prints as null");
  assert("{12 / zero ?? null}" == "null", "…and so does a defended one");
  assert(12 > 0, "positive integer is true");
```

8.0.3. Bitwise operators

While ‘and’/‘or’ work on true/false values, bitwise operators work on the individual bits of an integer. They are useful for flags, masks, and low-level data manipulation.

```
'&'  — keeps only bits set in BOTH operands (AND)
'|'  — keeps bits set in EITHER operand (OR)
'<<' — shift bits left N positions  (×2^N)
'>>' — shift bits right N positions (÷2^N)
```

Tip: ‘&’ binds more tightly than comparison operators, so ‘x & 15 == 8’ reads as ‘(x & 15) == 8’. Parentheses make that grouping visible to a reader when you mix bitwise and comparison expressions.

```
    assert((0x0f & 0xa8) == 8, "Bitwise AND: keeps only bits in both");
    assert((0xf0 | 0x0f) == 0xff, "Bitwise OR: combines bits from either");
    assert(1 << 4 == 16, "Left shift: 1 × 2^4 = 16");
    assert(256 >> 3 == 32, "Right shift: 256 ÷ 2^3 = 32");
```

8.0.4. Negation

‘!’ is the logical NOT operator. It flips any boolean expression.

```
    assert(!false, "not false is true");
    assert(!(1 > 2), "not false comparison is true");
```

8.0.5. Formatting booleans

Booleans can be embedded in a format string like any other value. The ‘^’ alignment specifier centres the value in a field of given width. ‘<’ aligns left, ‘>’ aligns right (a boolean aligns left by default, like text).

```
    assert("1{true:^7}2" == "1 true  2", "Centred boolean in field of width 7");
    assert("{false}" == "false", "Plain boolean format");
```

8.0.6. ‘&’ vs ‘and’

‘&’ is bitwise AND on integers; ‘and’ (or ‘&&’) is logical AND on booleans. Writing ‘a & b’ where both sides are booleans is a compile error — the compiler requires ‘and’ or ‘&&’ for boolean logic.

```
    flag_a = 3 > 1;
```

true

```
    flag_b = 4 > 2;
```

true

```
    assert(flag_a and flag_b, "Correct: logical AND on two booleans");
}
```

9. Float

A ‘float’ stores a number with a decimal point, like 3.14 or −0.001. It gives you about 15 significant digits of precision — enough for scientific work, games, and most real-world maths. When you write a number with a decimal point in Loft, it is automatically a float.

```
fn main() {
```

Write a decimal point in a literal and Loft treats the whole expression as a float. The usual arithmetic operators (+, -, *, /) all work the way you would expect.

```
  assert(1.5 + 0.5 == 2.0, "Float addition");
  assert(3.0 / 2.0 == 1.5, "Float division");
  assert(2.0 * 1.5 == 3.0, "Float multiplication");
```

9.0.1. Undefined results are null (‘float?’) — but infinities are NOT undefined

Some float operations have no real answer for some inputs: ‘sqrt’ of a negative, ‘ln’/‘log’ of a NEGATIVE, ‘asin’/‘acos’ outside −1..1, an out-of-domain ‘pow’, ‘%’ by zero, and 0.0/0.0. Rather than crash, Loft yields null for these — so ‘/’, ‘%’, ‘sqrt’, ‘ln’, ‘log’, ‘pow’, ‘asin’ and ‘acos’ each produce a NULLABLE float (‘float?’). The null then propagates: any further arithmetic on a null stays null, and ‘?? fallback’ discharges it when you need a plain float.

Read the boundary carefully, because it is not “anything unusual is null”. Floats keep IEEE INFINITY, and infinity is a value, not a missing one:

```
1.0 / 0.0   is  inf        0.0 / 0.0   is  null
```

−1.0 / 0.0 is -inf 1.0 % 0.0 is null

```
ln(0.0)     is -inf        ln(-1.0)    is  null
```

So a ‘?? fallback’ does NOT rescue a division by zero the way it rescues a sqrt of a negative — the infinity is not null, sails through the guard, and propagates through every later sum. When a zero divisor is possible, test the DIVISOR rather than defending the result.

```
  assert((sqrt(-1.0) ?? 0.0) == 0.0, "sqrt of a negative is null");
  assert(((sqrt(-1.0) + 5.0) ?? 0.0) == 0.0, "null propagates through
arithmetic");
  fz = 0.0;
  fp = 1.0;
  assert("{fp / fz}" == "inf", "a non-zero over zero is an infinity, not a
null");
  assert(((fp / fz) ?? -1.0) > 1000000.0, "…so `??` does not rescue it");
  assert(((fz / fz) ?? -1.0) == -1.0, "…while 0.0/0.0 really is null, and `??`
does");
```

The ‘f’ suffix selects single-precision floats, which take half the memory of a regular float but have fewer decimal digits of accuracy. This trade-off is common in graphics and audio code where exact decimal values matter less than speed or memory.

```
x = 0.1f + 2 * 1.0f;
assert(x == 2.1f, "Single precision float");
```

‘as float’ converts an integer to a float so you can mix them in calculations. Putting a float inside ‘{...}’ converts it to text for display. You can also go the other way: ‘“1.5” as float?’ parses the text back to a number. The ‘?’ is required — a bare ‘as float’ on text is refused for the same reason a bare ‘as integer’ is (“a text parse `as float` may fail, and a bare cast asserts it cannot”), because the text might not be a number.

```
assert(3 as float == 3.0, "Integer to float");
assert("1.5" as float? == 1.5, "Text to float");
assert("{1.5}" == "1.5", "Float formatted as text");
```

9.0.2. Formatting Floats

Put a colon after the value inside ‘{...}’ to control how it looks. ‘{value:width.precision}’ — ‘width’ is the minimum number of characters printed (padded with spaces on the left); ‘precision’ fixes the number of decimal places. You can use either part on its own: ‘{value:.2}’ just fixes decimal places, ‘{value:5}’ just sets the minimum width.

```
assert("{1.2:4.2}" == "1.20", "Float with width and precision");
assert("{334.1:.2}" == "334.10", "Float with precision only");
assert("{1.4:5}" == "  1.4", "Float with width only");
```

9.0.3. Math Functions

‘PI’ is a built-in constant (approximately 3.14159265358979). Multiplying it by 1000 and rounding gives 3142, which confirms the value is correct.

```
assert(round(PI * 1000.0) == 3142.0, "PI constant");
```

‘pow(base, exponent)’ raises a number to a power — this is exponentiation. Note: ‘^’ in Loft is bitwise XOR, NOT exponentiation. Always use pow() for powers. ‘log(value, base)’ is the inverse: log(x, b) answers “b to the what power equals x?” Here: 4^5 = 1024, and log(1024, 2) = 10 because 2^10 = 1024.

```
assert(log(pow(4.0, 5), 2) == 10.0, "log base 2 of 4^5");
```

‘sin’ and ‘cos’ work in radians, not degrees. A full circle is 2*PI radians; a half circle (180 degrees) is PI radians. sin(PI) should be exactly zero but computers use approximations for decimal numbers, so you may get something like 0.0000000001 instead. We use ceil() to round it up to 0. cos(PI) is exactly −1, so the whole expression ceil(0 + −1 * 1000) lands at −1000.

```
assert(ceil(sin(PI) + cos(PI) * 1000) == -1000.0, "sin and cos");
```

‘abs()’ returns the absolute value — the distance from zero, always non-negative. Useful any time you care about magnitude but not direction.

```
    assert(abs(-2.5) == 2.5, "Absolute value of float");
```

‘round()’ picks the nearest whole number; a half rounds away from zero (2.5 ⟶ 3, −2.5 ⟶ −3). ‘ceil()’ always rounds up to the next whole number, even for 2.01. ‘floor()’ always rounds down to the previous whole number, even for 2.99. All three give back a float, not an integer — so 3.0, not 3.

```
    assert(round(2.6) == 3.0, "round up");
    assert(round(2.4) == 2.0, "round down");
    assert(ceil(2.1) == 3.0, "ceil");
    assert(floor(2.9) == 2.0, "floor");
```

Single-precision floats work inside format strings just like regular floats.

```
    assert("a{0.1f + 2 * 1.0f}b" == "a2.1b", "Single-precision format");
}
```

10. Functions

Functions let you give a reusable piece of logic a name so you can call it from multiple places without copying code. This page covers: declaring functions, default argument values, reference parameters (so a function can modify the caller's variable), early return, const parameters, and type-based dispatch.

10.0.1. Declaring Functions

The keyword 'fn' starts a function definition. List parameters as 'name: type' separated by commas. '-> type' declares what the function gives back. The last expression in the body is returned automatically — no 'return' needed there. Default values let callers skip optional arguments.

```
fn greet(name: text, greeting: text = "Hello") -> text {
  greeting + ", " + name + "!"
}
```

10.0.2. Default arguments that involve other arguments

A default value can be any expression, not just a literal — including one that references earlier parameters, which are in scope by the time their default is evaluated. So 'end' can default directly to the length of the earlier 't'.

```
fn substring(t: text, start: integer = 0, end: integer = len(t)) -> text {
  t[start..end]
}
```

10.0.3. Named Arguments

When a function has several parameters with defaults, you can skip middle ones by naming the arguments you want to provide. Positional arguments come first; once you use a name, all remaining arguments must also be named.

```
fn connect(host: text, port: integer = 8080, tls: boolean = true) -> text {
  "{host}:{port}:{tls}"
}
```

10.0.4. Reference Parameters and Defaults

A function with no '-> type' is called for its side effects, not its result. Adding '&' before a parameter type makes it a reference: the function receives a direct link to the caller's variable and can read or write it. Without '&', a number, boolean or text is copied in, so reassigning it inside the function stays local. A struct or vector parameter still shares its contents with the caller: a field or element write inside the function reaches the caller too. Default values (written '= value' after the type) are substituted when the caller omits that argument.

```
fn add(a:  & integer, b: integer, c: integer = 0) {
  a += b + c;
}
```

10.0.5. Early Return

Use 'return value;' to exit a function before reaching the end. This is handy for guard clauses: handle the special cases first, then write the normal path without extra nesting.

```
fn classify(n: integer) -> text {
  if n < 0 {
    return "negative";
  }
  if n == 0 {
    return "zero";
  }
  "positive"
}
```

10.0.6. Const and Reference Parameters

Write 'const' with the TYPE, not before the name — 'fn f(s: const text)', not 'fn f(const s: text)', which does not parse. It tells the compiler "this function must never change this value", and the compiler enforces it: assigning to a const parameter is a compile error ("Cannot modify const parameter"). '&' is the opposite promise: "this function will modify this value." Declaring '&' without actually writing to the parameter is also a compile error ("Parameter 'a' has & but is never modified; remove the &"). These rules make it easy to read a function signature and know what it does to its inputs.

'const' earns its keep on a parameter the function COULD write through — a text, a vector, a struct. On a primitive you never modify it is reported as needless ('needless-const-parameter'), because a plain 'integer' parameter is already a copy and const adds nothing to it.

```
fn shout(s: const text) -> text {
  s + "!"
}
```

```
fn scale(a: integer, factor: integer) -> integer {
  a * factor
}
```

10.0.7. Type-Based Dispatch

You can define two functions with the same name as long as their parameter types differ. Loft picks the right one at compile time based on the type of the argument you pass.

```
fn describe_int(v: integer) -> text {
  "integer:{v}"
}
```

```
fn describe_text(v: text) -> text {
  "text:{v}"
}
```

10.0.8. Function References

Writing a function's NAME on its own creates a reference to it that you can store or pass around — 'f = double_it', with no 'fn' in front. ('fn double_it' is refused: "Use the function name directly, without 'fn' prefix".) The compiler checks that the name exists — a typo is a compile error. The result has type 'fn(param_types) -> return_type' and can be:

```
- stored in a variable,
- called directly with 'f(args)', and
- passed as a parameter to another function.
```

```
fn double_it(x: integer) -> integer {
  x * 2
}
```

```
fn negate_it(x: integer) -> integer {
  - x
}
```

```
fn apply_fn(f:  fn(integer) -> integer, x: integer) -> integer {
  f(x)
}
```

10.0.9. Lambda Expressions

A lambda is an anonymous (unnamed) function written right where you need it. Use the full form when you want to be explicit about types:

```
fn(param: type) -> return_type { body }
```

Use the short form (pipe-bar syntax) when the call site already knows the types:

```
|param| { body }
```

Both forms work anywhere a function reference is expected — especially with map, filter, and reduce. The short form does not allow type annotations; use the full fn(...) form when you need explicit types. Lambdas that need outer variables can capture them: see the Closures page.

```
fn main() {
```

Passing both arguments explicitly overrides the default.

```
  assert(greet("World", "Hi") == "Hi, World!", "Explicit argument");
```

Leaving out the second argument causes Loft to use the default "Hello".

```
  assert(greet("World") == "Hello, World!", "Default argument");
```

‘add’ takes ‘a’ by reference, so every call updates the original variable ‘v’. Watch how v accumulates: 1 -> 3 -> 8.

```
v = 1;
add(v, 2);
```

v = 1 + 2 = 3

```
add(v, 4, 1);
```

v = 3 + 4 + 1 = 8

```
assert(v == 8, "Reference parameter: {v}");
```

‘classify’ uses early returns to handle each case separately.

```
assert(classify(-5) == "negative", "Classify negative");
assert(classify(0) == "zero", "Classify zero");
assert(classify(3) == "positive", "Classify positive");
```

‘scale’ uses plain parameters (no ‘const’, no ‘&’): it only reads them to compute the product, so the caller’s values are never touched.

```
assert(scale(3, 7) == 21, "scale(3,7)");
```

‘shout’ promises not to change its text, and the compiler holds it to that.

```
assert(shout("hi") == "hi!", "const parameter: {shout(\"hi\")}");
```

Loft selects the right function based on the type of the argument.

```
assert(describe_int(42) == "integer:42", "Integer describe");
assert(describe_text("hi") == "text:hi", "Text describe");
```

A function reference is stored in a variable with type ‘fn(...) -> ...’. Calling it looks exactly like a regular function call.

```
f = double_it;
assert(f(5) == 10, "fn-ref stored and called: {f(5)}");
```

Pass function references as arguments to higher-order functions.

```
assert(apply_fn(double_it, 7) == 14, "fn-ref as arg (double)");
assert(apply_fn(negate_it, 3) == -3, "fn-ref as arg (negate)");
```

Lambdas with map, filter, and reduce. Use |...| short form — types are inferred from the call site.

```
  nums = [1, 2, 3, 4, 5];
  doubled = map(nums, |x| { x * 2 });
  assert(doubled[0] == 2, "lambda map first element");
  assert(doubled[4] == 10, "lambda map last element");
```

filter keeps only elements where the lambda returns true.

```
  evens = filter(nums, |x| { x % 2 == 0 });
  assert(evens[0] == 2, "filter evens first");
  assert(evens[1] == 4, "filter evens second");
```

reduce collapses the whole list into one value. The lambda receives the running total (acc) and the next element (x).

```
  total = reduce(nums, 0, |acc, x| { acc + x });
  assert(total == 15, "reduce sum: {total}");
```

Use fn(...) long form when you need explicit types — for example when storing a lambda in a variable or passing it without call-site context.

```
  negate = fn(x: integer) -> integer { -x };
  assert(negate(7) == -7, "stored lambda: {negate(7)}");
  assert(apply_fn(|x| { x * x }, 6) == 36, "lambda as arg: 6^2");
```

— Named arguments —

```
  assert(connect("example.com") == "example.com:8080:true", "all defaults");
  assert(connect("example.com", tls: false) == "example.com:8080:false", "skip
port");
  assert(connect(host: "example.com", tls: false) == "example.com:8080:false",
"all named");
  assert(connect("example.com", port: 443, tls: false) ==
"example.com:443:false", "mixed");
  assert(greet(name: "World") == "Hello, World!", "named with default greeting");
```

— Default argument that depends on another argument — ‘end’ defaults to len(t), computed from the earlier ‘t’ parameter.

```
  assert(substring("hello", 1, 3) == "el", "explicit start and end");
  assert(substring("hello", 2) == "llo", "end defaults to len(t)");
  assert(substring("hello") == "hello", "both defaults");
}
```

11. Vector

A vector is an ordered list of values that can grow and shrink while your program runs. Every element must have the same type — you cannot mix integers and text in one vector. Write a vector literal with square brackets: '[1, 2, 3]'. Loft automatically manages the storage, so vectors grow as you add elements without declaring a size up front.

11.0.1. Transforming vectors: map, filter, reduce

'map', 'filter', and 'reduce' each take a function and apply it to the vector. Pass the function by writing its NAME on its own — 'map(v, triple)', with no 'fn' in front of it.

```
map(v, f)          — apply f to every element; returns a new vector
filter(v, pred)    — keep only elements for which pred returns true
reduce(v, init, f) — combine all elements into a single value, starting from init
```

```
fn triple(x: integer) -> integer {
  x * 3
}
```

```
fn is_even_n(x: integer) -> boolean {
  x % 2 == 0
}
```

```
fn sum_acc(acc: integer, x: integer) -> integer {
  acc + x
}
```

```
fn mul_acc(acc: integer, x: integer) -> integer {
  acc * x
}
```

```
fn set_first_v(v: vector<integer>, x: integer) {
  v[0] = x;
}
```

```
fn append_one_v(v: vector<integer>, x: integer) {
  v += [x];
}
```

```
fn replace_all_plain(v: vector<integer>) {
  v = [7, 7];
}
```

```
fn replace_all(v: &vector<integer>) {
  v = [7, 7];
}
```

```
fn main() {
```

Create a vector with a literal and loop over it with ‘for’. Two special loop annotations help when you need to know where you are:

```
'v#first'  is true only on the very first iteration — useful for skipping
separators.
'v#index'  holds the zero-based position of the current element.
```

```
  x =[1, 3, 6, 9];
  b = "";
  for v in x {
    if !v#first {
      b += " ";
    }
    b += "{v#index}:{v}"
  }
  assert(b == "0:1 1:3 2:6 3:9", "result {b}");
```

‘+=’ appends another vector to the end of an existing one. You can filter and delete elements in a single pass:

```
Add 'if condition' after 'in vector' to visit only matching elements.
Write 'v#remove' inside the loop body to delete the current element.
```

Here we keep only multiples of 3 by removing everything else. Note: you cannot append to a vector (v += [...]) while iterating over it — that is a compile error because the loop would then visit the new elements too, which could loop forever.

```
  x +=[12, 14, 15];
  for v in x if v % 3 != 0 {
    v#remove;
  }
  assert("{x}" == "[3,6,9,12,15]", "result {x}");
```

11.0.2. Clearing

‘v.clear()’ removes all elements, setting the length to 0. The underlying storage is kept so appending afterwards is efficient.

```
  x.clear();
  assert(x.len() == 0, "clear empties the vector");
```

```
x += [99];
assert(x[0] == 99, "append after clear works");
```

11.0.3. Slicing

A slice gives you part of a vector as a new vector. 'v[a..b]' contains elements at positions a, a+1, ..., b-1 (b is excluded). 'v[a..]' goes from position a to the very last element. 'v[..b]' goes from the beginning up to (but not including) position b.

```
pows =[1, 2, 4, 8, 16];
assert("{pows[1..3]}" == "[2,4]", "Sub-vector");
assert("{pows[3..]}" == "[8,16]", "Open-ended sub-vector");
assert("{pows[..3]}" == "[1,2,4]", "Open-start sub-vector");
```

Accessing an index that does not exist is safe: Loft returns null instead of crashing. You can check for null with '!' (logical not) because null is falsy.

```
assert(!pows[10], "Out-of-bounds access returns null");
```

11.0.4. Comprehensions

A comprehension is a concise way to build a new vector from a formula. Syntax: '[for variable in range { expression }]' Add 'if condition' to include only elements that satisfy the condition. This is much shorter than creating an empty vector and appending in a loop.

```
evens =[for n in 1..10 if n % 2 == 0 {
  n
}];
assert("{evens}" == "[2,4,6,8]", "Filtered comprehension");
doubled =[for n in 1..6 {
  n * 2
}];
assert("{doubled}" == "[2,4,6,8,10]", "Doubled comprehension");
```

Embedding a for loop directly inside a format string '{...}' produces a formatted list. The format specifier after ':' is applied to every element in the result. ':02' means "at least 2 digits wide, padded with zeros on the left".

```
  assert("{for n in 1..7 {n*2}:02}" == "[02,04,06,08,10,12]", "Formatted vector
loop");
```

11.0.5. Reverse Iteration

Wrap a range in 'rev()' to step through elements from the last index to the first. 'rev(0..=3)' covers indices 0, 1, 2, 3 — the '=' makes the upper end inclusive. Here we visit pows[3]=8, pows[2]=4, pows[1]=2, pows[0]=1, building 8421 digit by digit.

```
c = 0;
for e in pows[rev(0..=3)] {
```

```
    c = c * 10 + e;
  }
  assert(c == 8421, "Reverse sub-vector iteration");
```

You can fill a vector with many copies of the same value using '; count' syntax:

```
[SomeStruct { field: value }; 16]
```

This creates 16 identical copies in one expression. See the Struct page for examples.

11.0.6. Passing vectors to functions

A vector parameter is a shared view of the caller's vector — no data is copied. So everything the function does to the CONTENTS is visible to the caller once it returns: writing an element, appending, removing, clearing. You do not need to mark the parameter for any of that.

Exactly one thing stays local: replacing the WHOLE value. Writing 'v = [7, 7]' inside the function gives that function a different vector and leaves the caller's alone. Mark the parameter '&' when you want that replacement to reach the caller as well.

```
fn replace_all(v: &vector<integer>) { v = [7, 7]; }   // caller sees the new
vector
```

So '&' means "I may replace this", not "I may add to it".

A SLICE is a different thing again. 'v[2..5]' builds a FRESH vector holding copies of those elements, so writing to the slice does not reach the original. A slice passes directly where a 'vector<T>' parameter is expected ('total(v[1..3])'); the function then works on that copy.

```
  passed: vector<integer> = [1, 2, 3];
  set_first_v(passed, 99);
  assert(passed[0] == 99, "a write to an existing element reaches the caller");
  grown: vector<integer> = [1, 2, 3];
  append_one_v(grown, 4);
  assert(len(grown) == 4, "…and so does an append, with no '&' needed");
  kept: vector<integer> = [1, 2, 3];
  replace_all_plain(kept);
  assert(len(kept) == 3, "replacing the whole value stays inside the function");
  swapped: vector<integer> = [1, 2, 3];
  replace_all(swapped);
  assert(swapped[0] == 7, "…unless the parameter is marked '&'");
  source: vector<integer> = [1, 2, 3, 4, 5];
  window = source[2..5];
  set_first_v(window, 77);
  assert(window[0] == 77, "the slice itself is written");
  assert(source[2] == 3, "…and the original is untouched, because a slice is a
copy");
```

11.0.7. Higher-order functions

'map' applies a function to every element and returns a new vector.

```
    nums =[1, 2, 3, 4, 5];
    tripled = map(nums,  triple);
    assert("{tripled}" == "[3,6,9,12,15]", "map triple: {tripled}");
```

‘filter’ keeps only elements for which the predicate returns true.

```
    evens2 = filter(nums,  is_even_n);
    assert("{evens2}" == "[2,4]", "filter evens: {evens2}");
```

‘reduce’ folds all elements into a single value, starting from an initial accumulator. Argument order: reduce(vector, initial_value, combiner).

```
    total = reduce(nums, 0,  sum_acc);
    assert(total == 15, "reduce sum: {total}");
    product = reduce(nums, 1,  mul_acc);
    assert(product == 120, "reduce product: {product}");
```

You can chain these calls: filter first, then map, then reduce.

```
    result = reduce(map(filter(nums,  is_even_n),  triple), 0,  sum_acc);
    assert(result == 18, "filter+map+reduce: {result}");
}
```

12. Structs

A struct groups related values under one name. Instead of keeping a product's name, price, and stock count in three separate variables that can drift out of sync, you bundle them together into a Product struct. Each piece of data inside the struct is called a field. You read and write fields using dot notation: 'product.price'. Here is a simple product record. All four fields are plain integers or text.

```
struct Product {
  name: text,
  price: integer,
  stock: integer
}
```

12.0.1. Field Constraints

You can restrict what values a field may hold. 'limit(min, max)' gives the field a smaller range. A value the compiler cannot prove fits is refused where you write it, and the error names the cures: a fallback ('?? <value>'), a 'u8?' destination so a value that does not fit reads null, or a mask ('& 255') that makes the value provably fit. It never stops a running program: a step the compiler cannot ask about, 'c.r += 100', takes the type's default when it leaves the range — 0 for a range that includes zero — and loft advises you ('narrow-fallback'). A nullable field takes null instead. Zero is an ordinary value here. A Colour of 0, 0, 0 is pure black and reads back as three zeros, with nothing extra to write. Fields you omit in a constructor receive zero (or null for nullable fields) by default.

You may meet 'not null' on a field in older code. It has no effect — a type is non-null by default — and the compiler advises removing it.

```
struct Colour {
  r: integer limit(0, 255),
  g: integer limit(0, 255),
  b: integer limit(0, 255)
}
```

12.0.2. Methods

A method is a function whose first parameter is named 'self'. Loft uses the type of 'self' to decide which struct the method belongs to. Call it with dot notation: 'c.to_hex()'. A method can read fields via 'self.field' and return any type.

```
fn to_hex(self: Colour) -> integer {
  self.r * 0x10000 + self.g * 0x100 + self.b
}
```

A method can also return a completely new struct value. Write '-> StructName' as the return type and construct the value in the body. The original struct is not modified — the caller gets a brand-new copy.

```
fn dimmed(self: Colour) -> Colour {
  Colour {r: self.r / 2 ?? 0, g: self.g / 2 ?? 0, b: self.b / 2 ?? 0 }
}
```

12.0.3. Defaults and Computed Fields

Write '= expression' after the type to set a stored default, filled at construction. Use 'computed(expression)' for a field that recalculates every time you read it — it takes no space in the record and always reflects the current state. Inside both, '$' refers to the struct.

```
struct Item {
  name: text,
  name_length: integer = len($.name)
}
```

```
struct Circle {
  radius: float,
  area: float computed(3.14159265 * $.radius * $.radius)
}
```

12.0.4. Storing many structs in a vector

'Area' uses smaller integer types to save memory: 'u16' holds values 0–65535, and 'u8' holds values 0–255. This matters when you have millions of records (such as tiles in a game map) and memory usage counts.

```
struct Area {
  height: u16,
  terrain: u8,
  water: u8,
  direction: u8
}
```

12.0.5. Value structs

Prefix a struct with 'value' to give it value (copy) semantics and zero heap overhead: it is stored inline inside a record or vector element, exactly like a plain integer. Reading one out of a field or element yields an independent COPY — writing to that copy never changes the original. Use a 'value struct' for small wrapper types (a point, a colour, a timestamp) that should be as cheap as the number they wrap.

A plain (reference) struct differs only when you read one OUT of somewhere. Two spellings look alike, so it is worth keeping them apart:

- Binding a whole value COPIES it, always. After 'b = a' the two names are

```
independent, and writing 'b' never changes 'a'. The copy goes all the way
down, including nested structs and vectors.
```

- Reading a struct-typed field or element gives you a VIEW: 'w = o.inner'

```
and 'c = v[i]' name the record in place, so 'w.n = 42' changes 'o.inner'
too. A view is not a copy — it points at the interior of what you read it
from. (A vector-typed field is a whole value, so 'av = bx.v' copies.)
```

- Write '&' when you want a live link where a bind would copy: 'b = &a' and

```
'av = &bx.v' both write through to the source.
```

- ⚠️ The same expression can COPY in one position and ALIAS in another. ‘av = bx.v’

```
copies, because binding a whole value always copies — but 'f(bx.v)' hands the
field itself to the function, and a heap parameter shares what it is given (see
the Vector page), so the function writes through to 'bx.v'. Binding is the copy;
passing is not. Read the position, not just the expression.
```

```
value struct Point {
  x: integer,
  y: integer
}
```

12.0.6. A struct that contains itself

A field may have the struct’s own type: a tree node holds its children. Write the field nullable — ‘left: Tree?’ — so a branch can end. A ‘Tree’ that must always hold another ‘Tree’ would never end, and the compiler refuses it: “‘Tree.left: Tree’ makes every Tree contain another Tree — make it ‘Tree?’”. A child is a value like any other field. Reading ‘t.left’ gives a view, writing it copies the new subtree in, ‘null’ removes the subtree, and a copy of the tree copies its children too. It prints, compares and converts to JSON as a nested record. When one node must be SHARED by two places, use ‘reference<Tree>’ instead — that field holds a link, not a copy. To test a child and pass it on, bind it to a local first: ‘l = t.left; if l { … }’. The test narrows ‘l’ to a present ‘Tree’.

```
struct Tree {
  key: integer,
  left: Tree?,
  right: Tree?
}
```

```
fn tree_sum(t: Tree?) -> integer {
  if !t {
    return 0;
  }
  t.key + tree_sum(t.left) + tree_sum(t.right)
}
```

```
fn insert(t: Tree, key: integer) {
  if key < t.key {
    l = t.left;
    if l { insert(l, key); } else { t.left = Tree { key: key }; }
  } else {
    r = t.right;
    if r { insert(r, key); } else { t.right = Tree { key: key }; }
  }
}
```

```
struct Holder8 {
  v: vector<integer>
}
```

```
fn touch8(w8: vector<integer>) {
  w8[0] = 99;
}
```

```
fn main() {
```

Constructing a struct: name the fields you want to set. Fields you leave out default to zero (or null for nullable fields).

```
  apple = Product {name: "Apple", price: 120, stock: 50 };
  assert(apple.price == 120, "price field: {apple.price}");
  assert(apple.name == "Apple", "name field: {apple.name}");
```

You can read and write individual fields after construction.

```
  apple.stock -= 1;
  assert(apple.stock == 49, "stock after one sale: {apple.stock}");
```

A field omitted from the constructor gets zero as its default. loft points this out ('omitted-field-zero'): the zero is the type's, and nothing in the declaration chose it. That is fine when zero is what you meant, and a trap when zero is a meaningful value in your data — an index where 0 is the first entry, say. Give the field a declared default when you want a different one, the way 'Item.name_length' does above.

```
  col = Colour {r: 128, b: 128 };
  assert(col.g == 0, "omitted green channel defaults to zero");
```

Zero is a real value in a limited field — pure black needs nothing extra.

```
  black = Colour {r: 0, g: 0, b: 0 };
  assert(black.r == 0 and black.g == 0 and black.b == 0, "pure black reads back
as zeros");
```

A value the compiler cannot prove in range is REFUSED unless you say what it becomes: `?? 0` makes the fallback explicit, and 300 does not fit, so the fallback is stored.

```
  over = 300;
  black.r = over ?? 0;
  assert(black.r == 0, "out of range takes the fallback you wrote: {black.r}");
```

Formatting a struct shows all fields compactly.

```
  assert("{col}" == "{{r:128,g:0,b:128}}", "Struct compact formatting");
```

':j' produces JSON output — a text format used by many web APIs and config tools.

```
  assert("{col:j}" == "{{\"r\":128,\"g\":0,\"b\":128}}", "JSON format");
```

Call a method on a variable using dot notation.

```
  purple = Colour {r: 128, b: 128 };
  assert("{purple.to_hex():x}" == "800080", "hex method result");
```

You can call a method directly on a constructor expression.

```
  assert(Colour {r: 255, g: 0, b: 0 }.to_hex() == 0xff0000, "method on
constructor");
```

'dimmed' returns a new Colour with each channel halved. The original variable is unchanged.

```
  dark = purple.dimmed();
  assert(dark.r == 64, "dimmed r: {dark.r}");
  assert(dark.b == 64, "dimmed b: {dark.b}");
  assert(dark.g == 0, "dimmed g: {dark.g}");
```

Stored defaults are filled once at construction time.

```
  it = Item {name: "hello" };
  assert(it.name_length == 5, "stored default name_length: {it.name_length}");
```

computed() fields recalculate on every access — changing radius updates area.

```
  ci = Circle { radius: 5.0 };
  assert(ci.area > 78.0, "computed area: {ci.area}");
  ci.radius = 10.0;
  assert(ci.area > 314.0, "recomputed area: {ci.area}");
```

Fill a vector with copies of the same struct using '; count' syntax. This creates 16 Area records, all with the same initial values.

```
  map =[Area {height: 0, terrain: 1, water: 1, direction: 1 };
  16];
  assert("{map[3]}" == "{{height:0,terrain:1,water:1,direction:1}}", "tile
record");
  map[3].height = 200;
  assert(map[3].height == 200, "individual tile update");
```

A value struct binding is a snapshot (copy), not a shared view.

```
  pts =[Point {x: 1, y: 2 }, Point {x: 3, y: 4 }];
  q = pts[0];
```

q is an independent COPY of element 0

```
q.x = 99;
assert(pts[0].x == 1, "value struct copy does not write back");
assert(q.x == 99, "the copy itself did change");
```

A PLAIN struct answers the same two questions differently, so here they are side by side. Reading one out of an element gives a view, and writing the view reaches the element.

```
shelf =[Product {name: "Pear", price: 90, stock: 10 }];
item = shelf[0];
item.stock = 7;
assert(shelf[0].stock == 7, "a plain struct element read is a view");
```

Binding the whole value copies it, so the original keeps its own stock.

```
spare = item;
spare.stock = 3;
assert(item.stock == 7, "a whole-value bind copies");
assert(spare.stock == 3, "and the copy moves on its own");
```

The same field expression, in the two positions: bound it copies, passed it does not. This pair is easy to generalise the wrong way in either direction.

```
bx8 = Holder8 { v: [1, 2, 3] };
av8 = bx8.v;
av8[0] = 42;
assert(bx8.v[0] == 1, "binding a vector field copies it");
cx8 = Holder8 { v: [1, 2, 3] };
touch8(cx8.v);
assert(cx8.v[0] == 99, "passing the same field shares it");
```

Write '&' to get a live link where a bind would otherwise copy.

```
linked = &item;
linked.stock = 1;
assert(item.stock == 1, "'&' binds a live link");
```

A tree: children are fields of the struct's own type, nullable so a branch can end.

```
root = Tree { key: 5, left: Tree { key: 3 } };
assert(root.left.key == 3, "a child reads like any field");
assert(!root.right, "an absent child is null");
insert(root, 8);
insert(root, 4);
assert(root.left.right.key == 4, "inserted below 3");
assert(tree_sum(root) == 20, "5 + 3 + 4 + 8 = {tree_sum(root)}");
```

Binding the whole tree copies every subtree with it.

```
  branch_copy = root;
  branch_copy.left.key = 30;
  assert(root.left.key == 3, "the copy's children are its own");
```

Writing null removes a subtree.

```
  root.right = null;
  assert(tree_sum(root) == 12, "without the right branch: {tree_sum(root)}");
  assert("{root}" == "{{key:5,left:{{key:3,left:null,right:
{{key:4,left:null,right:null}}}},right:null}}", "prints nested: {root}");
```

12.0.7. A record with some fields changed

‘Product { price: 90, ..apple }’ is a copy of ‘apple’ with ‘price’ written — exactly ‘sale = apple; sale.price = 90;’ in one expression. The base keeps its value, the named fields are written as assignments (each reads the BASE, never the half-written copy), and ‘..base’ is the last entry. The base must be a Product and present. A variant updates the same way when the base is known to be that variant: a local of the variant’s type, or ‘..(s as Circle)’ inside ‘if s is Circle’.

```
  sale = Product { price: 90, ..apple };
  assert(sale.price == 90 && sale.name == "Apple" && sale.stock == 49, "the copy
with price written: {sale.price} {sale.name} {sale.stock}");
  assert(apple.price == 120, "the base keeps its price: {apple.price}");
```

12.0.8. Comparing structs

‘==’ compares what two structs hold, field by field — a plain struct and a value struct alike, with no ‘OpEq’ to write. To ask whether two names are the SAME record, put ‘&’ on both sides.

```
  twin = Product {name: "Pear", price: 90, stock: 3 };
  assert(spare == twin, "equal fields make equal structs");
  assert(!(&spare == &twin), "…held in two different records");
  assert(&linked == &item, "a '&' link names the record it links to");
```

12.0.9. sizeof

‘sizeof(Type)’ returns the byte size used when the type is stored as a struct field or vector element: fields packed largest first, then padded so the size is a multiple of the largest field’s alignment. Range-constrained integer types like u8 and u16 report their packed size, not the 8-byte stack slot a local takes.

```
  assert(sizeof(integer) == 8, "integer: 8 bytes (i64 storage)");
  assert(sizeof(u8) == 1, "u8: 1 byte (packed)");
  assert(sizeof(u16) == 2, "u16: 2 bytes (packed)");
  assert(sizeof(Colour) == 3, "Colour: 3 × u8 = 3 bytes");
  assert(sizeof(Area) == 6, "Area: u16 + 3 × u8 = 5 bytes, padded to 6");
}
```

13. Enums

An enum defines a fixed set of named choices. Instead of using raw numbers like 0 = north, 1 = east — which nobody can read six months later — you give each option a name. The compiler then checks every comparison and conversion, so a typo becomes a compile error instead of a silent wrong answer. Plain enums are just names that can be compared, ordered, and converted to and from text. Their order matches their declaration order.

```
enum Direction {
  North,
  East,
  South,
  West
}
```

13.0.1. Enums that carry data

Sometimes different choices have different shapes of data. A 'Circle' needs one number (radius); a 'Rect' needs two (width and height). Struct enums let each variant carry its own fields, keeping all the shape kinds in one type while still letting you work with each variant naturally.

```
enum Shape {
  Circle {radius: float },
  Rect {width: float, height: float }
}
```

13.0.2. Variants that hold the enum itself

A variant's field may have the enum's own type. That is how an expression tree is written: an 'Add' holds two expressions, a 'Neg' one. No '?' is needed, because the other variants — here 'Num' — end the recursion. Each child is a value: it is copied when you assign it, compared by content and printed as a nested variant. A 'match' arm binds a child like any other field, so a function over the tree is one arm per variant.

```
enum Expr {
  Num {v: integer },
  Add {l: Expr, r: Expr },
  Neg {e: Expr }
}
```

```
fn eval(e: Expr) -> integer {
  match e {
    Num { v } => v,
    Add { l, r } => eval(l) + eval(r),
    Neg { e } => -eval(e)
  }
}
```

13.0.3. Methods that work on any variant

Write the same function name for each variant, each taking 'self' of that variant's type. Loft automatically picks the right version at runtime based on which variant is actually stored in the variable. This lets you call shape.area() without caring which kind of shape it is.

```
fn area(self: Circle) -> float {
  PI * pow(self.radius, 2)
}
```

```
fn area(self: Rect) -> float {
  self.width * self.height
}
```

Polymorphic methods can return text just as well as numbers. Each variant can produce its own human-readable description.

```
fn describe(self: Circle) -> text {
  "circle with radius {self.radius}"
}
```

```
fn describe(self: Rect) -> text {
  "rect {self.width} by {self.height}"
}
```

13.0.4. Enum methods

A plain enum can have methods too. The method is written once, for the whole enum: the 'self' parameter carries the current value, and a 'match' on it picks the case. Here 'opposite()' flips North↔South and East↔West.

```
fn opposite(self: Direction) -> Direction {
  match self {
    North => South,
    South => North,
    East => West,
    West => East
  }
}
```

```
fn main() {
```

Plain enum values are ordered by declaration position. North comes first (smallest), West comes last (greatest).

```
  d = Direction.East;
  assert(d == East, "Enum equality");
  assert(d != North, "Enum inequality");
```

```
  assert(d > North, "East declared after North, so it is greater");
  assert(Direction.West > South, "West declared last, so it is greatest");
```

Convert between text and enum in both directions.

```
  assert("{d}" == "East", "Format plain enum as text");
  assert("West" as Direction == West, "Parse text to enum");
```

Use a plain enum method like any other method.

```
  assert(d.opposite() == West, "opposite of East is West");
  assert(Direction.North.opposite() == South, "opposite of North is South");
```

Struct enum variants are constructed exactly like regular structs.

```
  c = Circle {radius: 1.0 };
  assert(c.radius == 1.0, "Circle radius field");
  r = Rect {width: 4.0, height: 5.0 };
  assert(r.width * r.height == 20.0, "Rect manual area");
```

Calling area() on a Circle runs the Circle version; on a Rect it runs the Rect version — chosen automatically at runtime.

```
  assert(round(c.area() * 1000) == round(PI * 1000), "Circle area = PI*r^2");
  assert(r.area() == 20.0, "Rect area = width*height");
```

describe() uses format strings with field access inside each variant's method.

```
  assert(c.describe() == "circle with radius 1", "describe circle:
{c.describe()}");
  assert(r.describe() == "rect 4 by 5", "describe rect: {r.describe()}");
```

An expression tree: 1 + -(2), built from nested variants and evaluated by one match.

```
  expr: Expr = Add {l: Num {v: 1 }, r: Neg {e: Num {v: 2 } } };
  assert(eval(expr) == -1, "1 + -2 = {eval(expr)}");
  assert("{expr:j}" == "{{\"Add\":{{\"l\":{{\"Num\":{{\"v\":1}}}},\"r\":
{{\"Neg\":{{\"e\":{{\"Num\":{{\"v\":2}}}}}}}}}}}", "JSON nests the variants");
  same: Expr = Add {l: Num {v: 1 }, r: Neg {e: Num {v: 2 } } };
  assert(expr == same, "two trees with the same content are equal");
```

13.0.5. A method on the enum itself is the fallback

Calling area() on a value held as a 'Shape' works like a match on its variant, and a match must cover every variant. So when only some variants have their own version, the call is refused unless the enum has one too:

```
fn area(self: Shape) -> float { 0.0 }
```

That version is the '_' arm: a variant with its own area() still runs it, and every other variant runs this one. A method only some variants have is fine as long as nothing calls it on a value held as the enum. An empty-body stub for one variant also covers it:

```
fn area(self: SomeVariant) -> float { }
```

Give the stub a real body — '{ 0.0 }' — when anything reads its result. An empty body hands back the type's default (0.0 for a float, 0 for an integer, "" for text), which is the same on both backends. That is a real value, not a marker: it is not null, so there is nothing to test for, and a caller cannot tell it from a computed 0.0.

13.0.6. Match expressions on enums

Match picks a code path based on the active variant. You must handle every variant, or include a _ wildcard arm that catches the rest.

```
axis = match d {
  North | South => "vertical",
  East | West => "horizontal"
};
assert(axis == "horizontal", "or-pattern: East matches East|West");
```

When a variant has fields, name them inside braces to use them in the arm body.

```
label = match c {
  Circle { radius } => "r={radius}",
  Rect { width, height } => "{width}x{height}"
};
assert(label == "r=1", "field destructuring in match arm");
```

13.0.7. Guard clauses

An arm can have an `if` guard after the pattern. If the guard fails, matching falls through to the next arm. Because the guard can fail, a guarded arm alone does not prove the variant is handled — you still need a wildcard _ or an unguarded arm for that variant.

```
area = match c {
  Circle { radius } if radius > 0.0 => PI * radius * radius,
  _ => 0.0
};
assert(round(area * 1000) == round(PI * 1000), "guarded match on Circle");
```

13.0.8. Scalar match

Match also works on integers, text, floats, booleans, and characters. Arms can be literals, ranges, `null`, or _.

```
grade = match 85 {
  90..=100 => "A",
  80..90   => "B",
```

```
    _         => "C"
  };
  assert(grade == "B", "range pattern 80..90 matches 85");
```

Or-patterns work on scalars too.

```
  kind = match 2 {
    1 | 2 | 3 => "low",
    _         => "high"
  };
  assert(kind == "low", "scalar or-pattern");
```

A `null` pattern matches when the value is absent — here a division by zero, which yields null (see the Integers page). The `?? null` turns the reasoned null into a plain one; it is the value that is being matched either way.

```
  zero = 0;
  check = match 1 / zero ?? null {
    null => "absent",
    _    => "present"
  };
  assert(check == "absent", "null pattern matches div-by-zero when defended");
```

Character literals work in match arms.

```
  vowel = match 'e' {
    'a' | 'e' | 'i' | 'o' | 'u' => true,
    _ => false
  };
  assert(vowel, "character or-pattern");
}
```

14. Sorted

A ‘sorted’ collection holds records in order by one or more fields you choose as the sort criteria (called “key fields”). You can look up a record by those fields instantly and iterate all records in that order. The collection stays sorted as you add new elements — no manual sorting needed. Name the key fields in square brackets after the record type, as in ‘sorted<Elm[-key]>’: ‘field’ sorts A⟶Z / 0⟶9 (ascending), ‘-field’ sorts Z⟶A / 9⟶0 (descending).

```
struct Elm {
  key: text,
  value: integer
}
```

```
struct Db {
  map: sorted < Elm[-key] >
}
```

```
fn main() {
```

14.0.1. Adding Elements

You initialise a sorted collection the same way as a vector. The elements are automatically placed in the right position as you add them.

```
  db = Db {map: [
  Elm {key: "One", value: 1 },
  Elm {key: "Two", value: 2 },
  Elm {key: "Three", value: 3 },
  Elm {key: "Four", value: 4 }
  ] };
```

14.0.2. Looking Up by Key

Use square brackets with the key value to find a record instantly. If the key is not present the result is null — you can check it with ‘!’.

```
  assert(db.map["Two"].value == 2, "Key lookup: Two");
  assert(db.map["Three"].value == 3, "Key lookup: Three");
  assert(!db.map["Five"], "Missing key returns null");
  assert(!db.map[null], "Null key returns null");
```

Append new elements with ‘+=’; they are placed in the correct sorted position.

```
  db.map +=[Elm {key: "Zero", value: 0 }];
  assert(db.map["Zero"].value == 0, "Newly added element");
```

14.0.3. Iterating in Order

A ‘for’ loop over a sorted collection visits elements in key order. Here the key is ‘-key’ (descending text), so the order is: Zero, Two, Three, One, Four.

```
sum = 0;
for v in db.map {
  sum = sum * 10 + v.value;
}
```

Zero(0), Two(2), Three(3), One(1), Four(4) ⟶ 0*10+2=2, *10+3=23, *10+1=231, *10+4=2314

```
assert(sum == 2314, "Sorted iteration total: {sum}");
```

14.0.4. Iterating in Reverse

Wrap the collection in rev() to visit elements from last to first key order. Here the collection is sorted by ‘-key’ (descending text), so the stored order is Zero, Two, Three, One, Four. rev() visits them in the opposite order.

```
rev_sum = 0;
for v in rev(db.map) {
  rev_sum = rev_sum * 10 + v.value;
}
```

Four(4), One(1), Three(3), Two(2), Zero(0) ⟶ 4*10+1=41, *10+3=413, *10+2=4132, *10+0=41320

```
assert(rev_sum == 41320, "Reverse iteration total: {rev_sum}");
```

14.0.5. Loop Helpers: #first, #count, #index, and #remove

‘v#first’ is true for the very first element visited. ‘v#count’ is a running index starting at 0. ‘v#index’ is a sequential counter over the visited elements, also starting at 0 — on a sorted collection it advances in step with ‘#count’. ‘v#remove’ removes the current element while iterating.

```
first_key = "";
labels = "";
index_sum = 0;
for v in db.map {
  if v#first {
    first_key = v.key
  }
  if !v#first {
    labels += ","
  }
```

‘#index’ matches ‘#count’ here: both run 0,1,2,3,4 over the five elements.

```
  assert(v#index == v#count, "index tracks count: {v#index} vs {v#count}");
  index_sum += v#index;
```

```
  labels += "{v#count}:{v.key}"
}
assert(first_key == "Zero", "First element: {first_key}");
assert(labels == "0:Zero,1:Two,2:Three,3:One,4:Four", "Labels: {labels}");
assert(index_sum == 10, "0+1+2+3+4 = {index_sum}");
```

Remove elements with value > 2 while iterating.

```
for v in db.map if v.value > 2 {
  v#remove
}
sum2 = 0;
for v in db.map {
  sum2 += v.value
}
```

Remaining: Zero(0), Two(2), One(1) ⟶ sum = 3

```
assert(sum2 == 3, "Sum after remove: {sum2}");
```

14.0.6. Removing by Key

Assigning null to a sorted subscript removes the element with that key. Removing a key that is not present is a safe no-op.

```
db2 = Db {map: [
Elm {key: "A", value: 10 },
Elm {key: "B", value: 20 },
Elm {key: "C", value: 30 }
] };
db2.map["B"] = null;
assert(!db2.map["B"], "B was removed");
assert(db2.map["A"].value == 10, "A still present");
assert(db2.map["C"].value == 30, "C still present");
db2.map["missing"] = null;
```

no-op; does not panic

```
sum3 = 0;
for v in db2.map {
  sum3 += v.value
}
assert(sum3 == 40, "Sum after key removal: {sum3}");
```

Note: ‘#index’ works on a sorted collection — it is a sequential counter that advances one per visited element (see the loop above). It is only ‘index<T>’ collections that reject ‘#index’ — use ‘#count’ there instead (see the Index page).

14.0.7. Iterating a Key Range

The range examples live in `main\_ranges` below; run them here, because a file with a `main` runs only its `main`.

```
  main_ranges();
}
```

```
struct Word {
  name: text,
  score: integer
}
```

```
struct Dict {
  words: sorted<Word[name]>
}
```

```
fn main_ranges() {
```

14.0.8. Iterating a Key Range

Give the collection a RANGE in square brackets and the loop starts and stops at the right places, without looking at the records outside it. This is the operation a sorted collection exists for.

```
c[lo..hi]   from lo up to but NOT including hi
c[lo..=hi]  from lo up to AND including hi
c[lo..]     from lo to the end
c[..hi]     from the start up to but not including hi
```

Mind the two spellings: '..' leaves the last key out, '..=' keeps it. That is the same rule as a vector slice.

```
  d = Dict { words: [
    Word { name: "apple",      score: 5 },
    Word { name: "banana",     score: 3 },
    Word { name: "cherry",     score: 8 },
    Word { name: "date",       score: 2 },
    Word { name: "elderberry", score: 1 }
  ] };
  slice_total = 0;
  for w in d.words["banana"..="date"] {
    slice_total += w.score
  }
  assert(slice_total == 13, "banana(3)+cherry(8)+date(2) = {slice_total}");
  half_open = 0;
  for w in d.words["banana".."date"] {
    half_open += w.score
  }
  assert(half_open == 11, "'..' leaves date out: {half_open}");
  head_total = 0;
```

```
  for w in d.words[.."cherry"] {
    head_total += w.score
  }
  assert(head_total == 8, "apple(5)+banana(3) = {head_total}");
```

14.0.9. Filtering on something that is NOT the key

An `if` guard inside the loop still works, and it is what you want when the condition is not about the key: the guard sees every element and the body runs only for the matching ones. Use a range when you are selecting BY key, and a guard when you are selecting by anything else.

```
  cheap = 0;
  for w in d.words if w.score < 4 {
    cheap += w.score
  }
  assert(cheap == 6, "banana(3)+date(2)+elderberry(1) = {cheap}");
```

14.0.10. Open-ended Range (tail)

All records from “cherry” onward to the end of the sorted order.

```
  tail_count = 0;
  for w in d.words if w.name >= "cherry" {
    tail_count += 1
  }
  assert(tail_count == 3, "cherry, date, elderberry = {tail_count}");
```

14.0.11. Open-ended Range (head)

All records up to and including “banana”.

```
  head_names = "";
  for w in d.words if w.name <= "banana" {
    head_names += w.name + " "
  }
  assert(head_names == "apple banana ", "apple and banana: '{head_names}'");
```

14.0.12. Range outside the for (building a result vector)

Collect matching records into a vector for later use.

```
  subset = [for w in d.words if w.name >= "banana" && w.name <= "cherry"
{ w.score }];
  assert(subset[0] == 3, "first in range: {subset[0]}");
  assert(subset[1] == 8, "second in range: {subset[1]}");
}
```

filling and finding values

15. Index

An ‘index’ lets you find records instantly by key and iterate over ranges of keys in order. It supports multi-part keys: you can sort by a primary key and break ties with a secondary key. Name the key fields in square brackets after the record type, as in ‘index<Elm[nr, -key]>’: ‘field’ sorts ascending, ‘-field’ descending.

Two keyed collections over the SAME element type are two ROUTES to one set of records, not two collections. Give records to either one and both see them; write through one and the other reads the change; remove through one and it is gone from both. That is the point of declaring a second one — a second way in, by a different key — and loft says so if a literal fills both (‘linked-group-double-fill’), because then one record set ends up holding everything they were given.

```
struct Elm {
  nr: integer,
  key: text,
  value: integer
}
```

```
struct Db {
  map: index < Elm[nr, -key] >
}
```

```
fn main() {
```

15.0.1. Adding and Looking Up Records

Fill the index just like a vector. Elements are automatically kept in key order. This index is sorted first by ‘nr’ (ascending), then by ‘key’ (descending) for ties.

```
  db = Db {map: [
  Elm {nr: 101, key: "One", value: 1 },
  Elm {nr: 92, key: "Two", value: 2 },
  Elm {nr: 83, key: "Three", value: 3 },
  Elm {nr: 83, key: "Four", value: 4 },
  Elm {nr: 83, key: "Five", value: 5 },
  Elm {nr: 63, key: "Six", value: 6 }
  ] };
```

Provide all key fields together inside brackets to find a record. Supply fewer fields to match all records that share a prefix.

```
  assert(db.map[101, "One"].value == 1, "Key lookup");
  assert(!db.map[12, ""], "Missing key returns null");
  assert(!db.map[83, "One"], "Wrong secondary key returns null");
```

Fewer fields than the key has: every record sharing that prefix.

```
prefix_sum = 0;
for r in db.map[83] {
  prefix_sum += r.value
}
assert(prefix_sum == 12, "the three nr=83 records: {prefix_sum}");
```

15.0.2. Iterating in Order

A 'for' loop visits all elements in key order.

```
total = 0;
for r in db.map {
  total += r.value;
}
assert(total == 21, "Sum of all values: {total}");
```

15.0.3. Range Queries

Provide a range in the first key position to visit only the matching slice. '[83..92, "Two"]' means: nr from 83 up to (not including) 92, and key from "Two" down (since the key is sorted descending, "Two" is the start of that descending segment).

```
sum = 0;
for v in db.map[83..92, "Two"] {
  sum = sum * 10 + v.value;
}
```

Elements matched: (83, Three, 3), (83, Four, 4), (83, Five, 5)

```
assert(sum == 345, "Range iteration result: {sum}");
```

15.0.4. Loop Helpers: #first and #count

'r#first' is true for the very first element visited. 'r#count' is a running counter starting at 0. Note: '#index' is a compile error on index collections — use '#count' instead.

```
first_nr = 0;
count_total = 0;
for r in db.map {
  if r#first {
    first_nr = r.nr
  }
  count_total = r#count
}
assert(first_nr == 63, "First element by key: {first_nr}");
assert(count_total == 5, "Total elements: {count_total}");
```

15.0.5. Removing by Key

Assigning null to an index subscript removes the element with that exact key combination. Removing a key that is not present is a safe no-op.

```
  db.map[92, "Two"] = null;
  assert(!db.map[92, "Two"], "element (92, Two) was removed");
  assert(db.map[101, "One"].value == 1, "element (101, One) still present");
  db.map[999, "Z"] = null;
```

no-op; does not panic

```
  total2 = 0;
  for r in db.map {
    total2 += r.value
  }
  assert(total2 == 19, "Sum after key removal: {total2}");
}
```

16. Hash

A ‘hash’ lets you find a record by its key in the same time whether you have 10 records or 10 million — there is no searching through a list, just a direct jump to the answer. Unlike ‘sorted’ and ‘index’, a hash does not keep its records in order — that is exactly what makes the lookup fast. You can still iterate one and get a predictable order (ascending key), but the hash has to sort a snapshot to do it; see ‘Iterating a Hash’ at the bottom of this page. List the key fields in brackets: ‘hash<Type[field]>’. Combine a hash with a vector when you want both fast lookup and a stable iteration order.

```
struct Keyword {
  name: text
}
```

```
struct Data {
  h: hash < Keyword[name] >
}
```

16.0.1. Combining Hash and Vector

Pairing a hash with a vector on the same record type gives you the best of both worlds: the hash for instant lookup by key, and the vector for iterating in insertion order.

```
struct Count {
  t: text,
  v: integer
}
```

```
struct Counting {
  entries: vector < Count >,
  lookup: hash < Count[t] >
}
```

```
fn fill(c: Counting) {
  c.entries =[
 {t: "One", v: 1 },
 {t: "Two", v: 2 },
 {t: "Three", v: 3 },
 {t: "Four", v: 4 }
  ]
}
```

```
fn main() {
```

16.0.2. Basic Lookup

Fill a hash the same way you fill a vector. Look up by key with square brackets. A found record is truthy; a missing key returns null.

```
d = Data { };
d.h =[ {name: "one" }, {name: "two" }];
d.h +=[ {name: "three" }, {name: "four" }];
assert(d.h["three"], "Key exists");
assert(!d.h["None"], "Missing key returns null");
```

16.0.3. Hash + Vector Together

Here 'entries' is the vector (keeps insertion order) and 'lookup' is the hash (fast access). Both fields point to the same records — adding to one automatically updates the other.

```
c = Counting { };
fill(c);
assert(c.lookup["Three"].v == 3, "Hash lookup: Three");
assert(c.lookup["One"].v == 1, "Hash lookup: One");
assert(!c.lookup["Five"], "Missing key returns null");
```

Iterate in insertion order via the vector; look up by name via the hash. The vector also supports #first, #count, and #remove inside the loop.

```
add = 0;
for item in c.entries {
  add += item.v;
}
assert(add == 10, "Sum via vector iteration: {add}");
```

16.0.4. Removing a Key

Assigning null to a hash subscript removes that element. Removing a key that is not present is a safe no-op.

```
d.h["three"] = null;
assert(!d.h["three"], "three was removed");
assert(d.h["one"], "one still present");
d.h["missing"] = null;
assert(d.h["one"], "removing a key that is not there leaves the others alone");
```

16.0.5. Iterating a Hash

`for e in h { ... }` walks the hash in ascending key order. The loop variable has the same shape as a `sorted\<T\>` iteration — a `reference\<T\>` with `\#index`, `\#count`, and `\#first` available. Multi-field keys sort lexicographically.

Under the hood the compiler builds a one-shot sorted snapshot of the hash's record-numbers and walks it, so iteration cost is O(n) + O(n log n) per `for`, not amortised across calls. For a hot loop, pair the hash with a `vector\<T\>` or `sorted\<T\[k\]\>` on the same record type and iterate the companion collection.

`e\#remove` is rejected at compile time — the snapshot is detached from the hash, so removing from it would not actually remove from the hash. Use `h\[key\] = null` to remove an entry. Adding the

values up cannot show you the order — a sum is the same whichever way you walk. Collect the keys instead: they come out alphabetically, not in the order they went in.

```
  sum = 0;
  order = "";
  for e in c.lookup { sum += e.v; order += "{e.t} "; }
  assert(sum == 10, "Sum via hash iteration: {sum}");
  assert(order == "Four One Three Two ", "hash walks in key order: {order}");
}
```

17. File

A file handle lets you read and write files without worrying about when the OS opens or closes them. The file opens on the first read or write and closes automatically when the handle goes out of scope.

17.0.1. Inspecting the File System

`file(path)` creates a File handle without opening anything yet. `f\#format` is the mode the handle will read and write in. Two of its values answer what the path IS — Directory, and NotExists for a path with nothing at it. The other three are modes you choose: TextFile (UTF-8 text, and the default for any file that exists), LittleEndian (binary, bytes stored least-significant first — common on most PCs) and BigEndian (binary, most-significant first — common in network protocols). It does not inspect content: a PNG you have not set a format on reads as TextFile, so `\#format` cannot tell you a file is text. `lines()` reads a text file and returns it as a vector of lines. `exists(path)` is a convenience shorthand for checking that a path is not NotExists. `delete(path)` removes a file and returns a `FileResult` enum; use `.ok()` to check success. Clean up any leftover files from a previous interrupted run. `\#cwd` opts this program into cwd-relative paths (CLI-tool semantics): a relative path resolves against the working directory, not the program's own directory. Without it, relative paths re-home next to the program (the portable-bundle default).

```
#cwd
fn cleanup() {
  delete("test.bin");
  delete("test2.bin");
  delete("buffer.bin");
}
```

```
struct Buffer {
  size: i32,
  data: vector < single >
}
```

```
fn main() {
  cleanup();
```

Asking for the format of a directory path returns Directory, not a file format. `lines()` reads the named text file and splits it on newlines.

```
  ex = file("tests/example");
  assert(ex#format == Directory, "example is a directory");
  c = file("tests/example/config/terrain.txt").lines();
  assert(c[1] == "    terrain = [", "Line was '{c[1]}'");
```

`exists` and `delete` are safe to call when the file is absent. Nothing crashes: `delete` answers `FileResult.NotFound`, whose `.ok()` is false.

```
  assert(!exists("test.bin"), "File should not exist before the test.");
  assert(!delete("nonexistent_xyz.bin").ok(), "delete on missing file not ok");
```

17.0.2. Writing a Text or Binary File

Wrapping the file handle in a block ensures the file closes the moment the block ends. Set `f\#format` to LittleEndian or BigEndian before writing binary data. Use `f += value` to append the raw bytes of any scalar value: u8, i8, u16, i16, u32, i32, integer, single, float, boolean, or text. Cast to the width you mean — an uncast `integer` writes 8 bytes, which is rarely the record width a binary format wants, so the compiler asks you to say it. Write `as integer` where the 8 bytes are deliberate.

```
{f = file("test.bin");
 assert(f#format == NotExists, "File should not exist yet.");
 f#format = BigEndian;  // BigEndian: most-significant byte written first.
 f += 0 as u8;
 f += 1 as u8;
 f += 0x203 as u16;
 f += 0x4050607 as integer;
 f += 0x8090a0b0c0d0e0f as integer;
```

`f\#size` returns the total number of bytes written so far. An integer is 8 bytes, so the total is 1 + 1 + 2 + 8 + 8 = 20.

```
  assert(f#size == 20, "Should have written 20 bytes.");
```

Text is written as raw UTF-8 bytes with no length prefix.

```
  f += "Hello world!";
  assert(f#size == 32, "Size should be 32 bytes (20 + 12).");
}  // The file closes when the handle goes out of scope.
```

`move(src, dst)` renames a file. It refuses if the destination already exists.

```
  assert(exists("test.bin"), "File should exist after writing.");
  assert(move("test.bin", "test2.bin").ok(), "Could not move the file.");
```

17.0.3. Reading Back What You Wrote

When you open an existing file the default format is TextFile. Set `f\#format` to match the format used when writing before you read any bytes. `f\#read(n) as T` reads exactly n bytes and interprets them as type T. `f\#index` is the byte offset where the last read started. `f\#next` is the current read position; assign to it to seek anywhere.

```
{f = file("test2.bin");
 assert(f#format == TextFile, "The default format is TextFile.");
 f#format = LittleEndian;
```

The file was written as BigEndian, so reading the same 4 bytes as LittleEndian produces a byte-swapped value — that is intentional here and confirms that the bytes were stored in the order you chose.

```
  v = f#read(4) as i32;
  assert(v == 0x3020100, "BigEndian bytes 0..3 read as LittleEndian i32.");
  assert(f#index == 0, "Last read started at byte 0.");
  assert(f#next == 4, "Next read starts at byte 4.");
```

Seek to byte 20 to skip past the integers and read the text directly. (The u8+u8+u16+integer+integer header is 1+1+2+8+8 = 20 bytes, so the text starts there.)

```
  f#next = 20;
  s = f#read(5) as text;
  assert(s == "Hello", "Partial text read from offset 20.");
  assert(f#index == 20, "Last read started at byte 20.");
  assert(f#next == 25, "Next position after 5-byte text read.");
```

Requesting more bytes than remain simply returns whatever is left.

```
  rest = f#read(100) as text;
  assert(rest == " world!", "Read continues to end of file.");
  assert(f#next == f#size, "Position should be at end of file.");
```

You can seek back to any position and re-read.

```
  f#next = 0;
  assert(f#read(4) as i32 == 0x3020100, "Seek-and-reread matches original.");
}
  assert(delete("test2.bin").ok(), "Could not remove the test file.");
```

17.0.4. Working with Vectors and Struct Data

`f += vector\<T\>` writes every element in sequence as raw bytes, which is the fastest way to dump a whole collection to disk. Setting `f\#size = n` truncates or zero-extends the file to exactly n bytes — useful for discarding the tail after you have finished writing.

```
{f = file("buffer.bin");
 f#format = LittleEndian;
 ints =[1, 2, 3, 4];
 f += ints;
 assert(f#size == 32, "Four integers = 32 bytes (8 each)");
```

Truncate to the first two integers.

```
  f#size = 16;
  assert(f#size == 16, "Truncated to 16 bytes");
```

Growing works the same way, and the bytes you gain are zero.

```
  f#size = 24;
  f#next = 16;
  assert(f#read(8) as integer == 0, "The extension reads as zero bytes.");
 }
  assert(delete("buffer.bin").ok(), "Could not remove buffer.bin after vector
write test.");
```

Write a count followed by individual float values. `sizeof(T)` returns the byte width of a type, so you can compute the correct read length without hard-coding magic numbers.

```
 {buf = file("buffer.bin");
  buf#format = LittleEndian;
  buf += 4 as i32;
  buf += 1.1f;
  buf += 1.2f;
  buf += 2.1f;
  buf += 2.2f;
 }
```

Read the count, then use it to read exactly that many floats directly into a struct field. This pattern lets you serialise and deserialise structs with variable-length data cleanly. A sized `f\#read` answers a raw byte buffer, so name the type you are reading it back as: the `as` is what turns those bytes into elements.

```
 {b = Buffer { };
  f = file("buffer.bin");
  f#format = LittleEndian;
  n = f#read(4) as i32;
  floats = f#read(n * sizeof(single)) as vector<single>;
  b.data = floats;
  assert(len(b.data) == 4, "Read four floats back into the field.");
  assert((b.data[0] ?? 0.0f) == 1.1f, "First float survives the round-trip.");
 }
  assert(delete("buffer.bin").ok(), "Could not remove buffer.bin.");
```

17.0.5. Directories

`is\_dir(path)` and `is\_file(path)` answer the question directly, without going through a handle. `files()` lists a directory as File handles, so every entry carries its own path and format; `list\_dir(path)` gives you just the names. Both are sorted by name, so an index means the same entry on every machine — but they disagree on a path that is not a directory: `files()` answers an empty list and `list\_dir` answers null, so discharge that one with `?? \[\]`.

```
  dir = file("tests/example");
  assert(is_dir("tests/example"), "the example directory is a directory");
  assert(!is_file("tests/example"), "a directory is not a file");
  entries = dir.files();
  assert(len(entries) > 0, "the example directory is not empty");
```

```
  assert((entries[0] ?? file("")).path == "tests/example/config", "first entry
path");
```

`mkdir(path)` creates one level and `mkdir\_all(path)` creates every missing level. `mkdir\_all` is idempotent — a directory that already exists is Ok, which is what makes it safe to call on the way into a run.

```
  assert(mkdir_all("tests/example").ok(), "mkdir_all on an existing directory is
Ok");
  assert(!mkdir("tests/example").ok(), "mkdir on an existing directory is not
Ok");
```

`delete` is a file operation and answers FileResult.IsDirectory when you point it at a directory. `rmdir` is the one that removes a directory, and only an EMPTY one — it answers NotEmpty while entries remain. A recursive removal is yours to write: list_dir() the children, delete() the files, rmdir() the directory, deepest first.

17.0.6. Error handling

Filesystem operations that can fail return a `FileResult`, and it is an enum with six variants, not a boolean. `.ok()` is the shorthand for the Ok one; match when you need to tell the failures apart. The program does not crash on failure — you decide what to do.

```
  result = delete("this_file_does_not_exist.txt");
  assert(!result.ok(), "delete missing file returns not-ok");
  why = match result {
    Ok => "deleted",
    NotFound => "there was nothing there",
    PermissionDenied => "the OS refused",
    IsDirectory => "that is a directory",
    NotEmpty => "the directory still has entries in it",
    Other => "it failed for another reason"
  };
  assert(why == "there was nothing there", "delete on a missing file is
NotFound");
```

A handle to a path with nothing at it is safe to hold and safe to read. Nothing crashes: the format is NotExists and `lines()` is empty. A sized `f\#read` on it answers null rather than a value you might mistake for data, and says so on stderr.

```
  ghost = file("no_such_file.txt");
  assert(ghost#format == NotExists, "non-existent file has NotExists format");
  assert(len(ghost.lines()) == 0, "lines() on a missing file is empty");
```

`move` reports on the FILES, not on the shape of the path: a missing source is NotFound whether or not the path contains ...

```
  assert(!move("any.txt", "../escape.txt").ok(), "move with a missing source
fails");
```

One asymmetry to know: `f += value` is a statement and reports nothing. If the path cannot be opened for writing, the bytes are dropped and the run carries on with a message on stderr. Use `f.write(text)` when you need the write itself to answer a FileResult, and check `.ok()` after `delete`, `move`, `mkdir`, and `mkdir\_all`.

```
}
```

18. Lexer

(`lexer` is a piece of the loft-in-loft front end that lives in loft's own `lib/`, @F120; it is not a registry library yet, so a program outside this tree cannot `use` it.) The lexer library breaks a text into tokens so your program can understand its structure. Tokens are the smallest meaningful pieces: numbers, names (like variable and function names), operators, and quoted strings. You tell the lexer which multi-character sequences count as one token and which words are reserved keywords, and it handles the rest.

18.0.1. Setting Up the Lexer

Create a `lexer::Lexer` and register your language's rules before you parse anything. `set\_tokens` ensures operators like `+=` or `\>\>` are scanned as one token instead of two separate characters. `set\_keywords` prevents reserved words from being treated as ordinary names — the lexer will report them exactly as written so your parser can treat them specially.

```
use lexer;
fn main() {
  l = lexer::Lexer { };
  l.set_tokens(["+=", "*=", "-=", "<=", ">=", "!=", "==", ">>", "<<", "->", "=>",
">>>", "..", "..=", "&&", "||"]);
  l.set_keywords(["for", "in", "if", "else", "fn", "pub", "use", "struct",
"enum", "match", "and", "or"]);
```

18.0.2. Reading Tokens

`parse\_string(name, source)` feeds source text into the lexer. The name is used in error messages and position reports. After that, call the typed reader functions one by one to consume tokens in order.

`int()` consumes and returns the next integer token, or null if the current token is not an integer. `long\_int()` does the same for integers suffixed with `l`. `matches(s)` consumes the next token only when it equals s and returns true; otherwise it leaves the token in place and returns false. `peek()` returns the next token as text without consuming it. `position()` returns the current location as `file:line:col`.

```
  l.parse_string("Tokens", "12 += -2 * 3 >> 4");
  assert(l.int() == 12, "Integer");
  assert(!l.matches("+"), "Incorrect plus");
  assert(l.peek() != "+", "Incorrect plus");
  assert(l.matches("+="), "Incorrect plus_is");
  assert(l.int() == -2, "Second integer");
  assert(l.matches("*"), "Incorrect multiply");
  assert(l.int() == 3, "Third number");
  assert(l.position() == "Tokens:1:14", "Incorrect position {l.position()}");
  assert(!l.matches(">"), "Incorrect higher");
  assert(l.matches(">>"), "Incorrect logical shift");
  assert(l.position() == "Tokens:1:17", "Incorrect position {l.position()}");
```

`long\_int()` reads an integer written with an `l` suffix. The suffix is part of the token, not part of the value.

```
  l.parse_string("Long", "9000000000l");
  assert(l.long_int() == 9000000000, "Long integer");
```

A registered keyword is reported exactly as written, so a parser can tell it from an ordinary name without keeping a second list of its own.

```
  l.parse_string("Keywords", "for name");
  assert(l.peek() == "for", "A keyword reads back as itself");
  assert(l.matches("for"), "and matches by that spelling");
  assert(l.peek() == "name", "while an ordinary name is just a name");
```

18.0.3. String Literals and Comments

`constant_text()` reads a double-quoted string and decodes the escapes inside it: `\n` becomes one newline, `\t` one tab, `\r` one return, and `\\` and `\"` stand for a single backslash and quote, and `\xNN` is the character with that two-digit hex code — `\x41` is `A`. An escape the lexer does not recognise stands for its own character, so `\q` is just `q`, and an `\x` without two hex digits after it is treated the same way rather than eating what follows.

```
  l.parse_string("Texts", "\"123\" + '4'");
  assert(l.constant_text() == "123", "Incorrect text literal");
  assert(l.matches("+"), "Incorrect add");
```

`constant_character()` reads a single-quoted literal and returns a `character`, not text — so compare it against a character.

```
  assert(l.constant_character() == '4', "Incorrect character literal");
```

An escape counts as ONE character, in a text and in a character literal alike. `size()` is what shows it: two source characters, one value.

```
  l.parse_string("Escapes", "\"a\\nb\" '\\t'");
  nl = l.constant_text() ?? "";
  assert(size(nl) == 3, "'a\\nb' is three characters, not four: {size(nl)}");
  assert(l.constant_character() == '\t', "An escaped character literal decodes
too");
```

The lexer collects `//` comments automatically as it scans. You do not need to handle them yourself. `last_comment()` returns the accumulated comment text since the last consumed token. When multiple comment lines appear in a row they are joined with newlines into a single string. `comment_behind()` is true when the comment appeared on the same line as the preceding token rather than on its own line above. `is_finished()` returns true once every token has been consumed.

```
  l.parse_string("Comments", "// starting comments\n123 // same line comment\n//
extra comment\n4");
  assert(!l.comment_behind(), "Initial comment not behind");
  assert(l.last_comment() == "starting comments", "Initial comment");
```

```
  assert(l.int() == 123, "Content integer");
  assert(l.comment_behind(), "Second comment is behind");
  assert(l.last_comment() == "same line comment\nextra comment", "Second
comment");
  assert(!l.is_finished(), "Not Ready");
  assert(l.int() == 4, "Second integer");
  assert(l.last_comment() == "", "No remaining comment");
  assert(l.is_finished(), "Ready");
```

18.0.4. Embedded Format Expressions

Loft string literals can embed expressions with `{expr}`. The lexer exposes a protocol that lets you parse these yourself. When `constant\_text()` reaches a `{`, it returns the literal text before it and sets `is\_formatting()` to true. At that point call `set\_formatting(false)` and parse the embedded expression normally using the usual token readers. When the expression is done, call `set\_formatting(true)` and consume the closing `}`. Then `constant\_text()` continues with the next segment of the string. The example below writes that brace as "}}" because a literal brace inside a loft string is doubled — the TOKEN the lexer matches is the single `}`.

```
  l.parse_string("Formatting", "\"abc{{12 + 34}}def\"");
  assert(l.constant_text() == "abc", "Before formatting");
  assert(l.is_formatting(), "Formatting");
  l.set_formatting(false);
  assert(l.int() == 12, "First integer");
  assert(l.matches("+"), "Incorrect plus");
  assert(l.int() == 34, "Second integer");
  l.set_formatting(true);
  assert(l.matches("}}"), "Incorrect closing brace");
  assert(l.constant_text() == "def", "After formatting");
  assert(!l.is_formatting(), "Formatting");
}
```

19. Parser

(`parser` is a piece of the loft-in-loft front end that lives in loft's own `lib/`, @F121; it is not a registry library yet, so a program outside this tree cannot `use` it.) The 'parser' library lets a Loft program read and validate other Loft source code at runtime. This is useful when you want to:

- validate configuration files written in the Loft syntax
- build tools that inspect or transform Loft source
- write a test that checks whether a generated code snippet is syntactically correct

Together the 'lexer' and 'parser' libraries are designed as a reusable foundation. You can use them to build entirely new languages: feed the lexer your own token rules, then layer a custom parser on top. This keeps the tokeniser and grammar logic separate from the language runtime so you only bring in what you need.

'parse(name, source)' is the single entry point. It takes a display name (used in error messages) and a Loft source string. The name does not need to correspond to a real file — it is only used when reporting parse errors.

parse() answers how many errors the source contained — '0' for a snippet that parsed cleanly. The diagnostics themselves go to the log as it reads; the count is what a caller can branch on.

19.0.1. What the parser understands

The parser handles most of Loft's syntax:

- 'struct Name { field: type [= default] }' — data containers with named fields
- 'enum Name { Variant [{ field: type }] }' — named choices, each with optional data
- 'fn name(params) [-> type] { body }' — functions with a block body
- 'fn name(params) [-> type]; #rust "template"' — operator templates backed by Rust
- 'use module;' — module imports
- Statements: 'for' and 'while' loops, 'if' / 'else if' / 'else', 'return', typed

```
locals ('c: (text?, integer) = ("c0", 3);') and expressions; a statement that
ends
in a block needs no ';'
```

- Expressions: binary operators with precedence, function calls, field access

```
('p.x', 'pair.0'), attributes ('f#size'), 'match' with guards and patterns, both
lambda forms ('|x| { x * 2 }' and 'fn(x: integer) -> integer { x + 1 }'), tuples,
vectors, struct literals and blocks — each also as a value ('y = match x { … }')
```

- Type expressions: plain and qualified names ('lexer::Lexer'), generic types like

```
'vector<T>', keyed collections (sorted/hash/index), tuples, 'fn(…) -> T' and
integer ranges with 'integer limit(min, max)' — in parameter, field, and return
position
```

It reads an index expression ('v[0]'), the null-coalescing operator ('a ?? 0') and a formatted string literal ('"v={n}"') as the compiler does.

Outside its grammar, so a snippet using one answers a non-zero count although the compiler accepts it: ‘const’ fields and top-level ‘const’, ‘**’, ‘/=’, hex and ‘f’-suffixed literals, generic functions (‘fn f<T>’), filtered loops (‘for e in v if e > 1’), ‘is’ patterns, slices (‘v[1..]’) and backtick strings.

```
use parser;
fn main() {
```

19.0.2. Parsing a minimal function

The double braces ‘{{‘ and ‘}}’ produce literal ‘{‘ and ‘}’ inside a Loft format string — needed here because the source code itself contains braces.

```
  assert(parser::parse("hello", "fn main() {{ println(\"Hello World\"); }}") ==
0,
         "a minimal function parses cleanly");
```

19.0.3. Parsing a struct and function together

The parser validates the entire source snippet as one unit. Both the struct definition and the function body are checked.

```
  assert(parser::parse("point", "struct Point {{ x: integer, y: integer }} fn
origin() -> Point {{ Point {{ x: 0, y: 0 }} }}") == 0,
         "a struct and a function in one unit parse cleanly");
```

19.0.4. Parsing an enum

Both a plain variant and a struct variant (with fields) are recognised.

```
  assert(parser::parse("shape", "enum Shape {{ Circle {{ radius: float }},
Rectangle {{ w: float, h: float }} }}") == 0,
         "both a plain and a struct variant parse cleanly");
```

19.0.5. Parsing several statements in a body

A body is a sequence of statements; each call below is one.

```
  assert(parser::parse("body", "fn m() {{ println(\"a\"); println(\"b\"); }}") ==
0,
         "a body of several statements parses cleanly");
```

19.0.6. Parsing assignments and operators

Binary operators parse with their normal precedence, and so does assignment.

```
  assert(parser::parse("assign", "fn f() {{ t = 0; t += 1; }}") == 0,
         "assignment and compound assignment parse cleanly");
```

19.0.7. Parsing a for loop and arithmetic

This exercises the expression parser, range syntax, and a tail expression after a loop body — the shape where a speculative struct-literal parse has to give its tokens back so the loop can claim them.

```
  assert(parser::parse("loop", "fn sum(n: integer) -> integer {{ t = 0; for i in
1..=n {{ t += i; }} t }}") == 0,
         "a for loop over a range parses cleanly");
```

19.0.8. Parsing while, match and lambdas

Each of these is a value as well as a statement: a 'match' or a lambda can stand on the right of '='.

```
  assert(parser::parse("while", "fn f() {{ n = 0; while n < 3 {{ n += 1; }} }}")
== 0,
         "a while loop parses cleanly");
  assert(parser::parse("match", "fn f(x: integer) -> integer {{ match x {{ 0 =>
1, n if n > 9 => 2, 2 | 3 => 3, _ => 0 }} }}") == 0,
         "a match with a guard and alternatives parses cleanly");
  assert(parser::parse("lambdas", "fn f() {{ g = |x| {{ x * 2 }}; h = fn(x:
integer) -> integer {{ x + 1 }}; }}") == 0,
         "both lambda forms parse cleanly");
```

19.0.9. Practical use: validating user-supplied code

If your application lets users write Loft snippets (for scripting or configuration), you can parse them before executing:

```
snippet = read_user_input();
if parser::parse("user_code", snippet) > 0 {
  // The snippet was invalid — the count is how many errors it had.
}
```

Combine this with the lexer (see the Lexer page) when you need to extract individual tokens from the source rather than validate all of the syntax.

19.0.10. What a failed parse looks like

The count is the whole point: a snippet that does not parse answers a NON-zero number, so a caller can tell the two apart. Watching the call not crash cannot.

```
  assert(parser::parse("broken", "fn main() {{ println(\"hi\");") > 0,
         "an unterminated body reports at least one error");
```

A missing operand is an error too, and so is text after the last definition.

```
  assert(parser::parse("no_value", "fn f() {{ x = ; }}") > 0, "a missing value is
reported");
  assert(parser::parse("stray", "fn f() {{ }} }}") > 0, "a stray brace is
reported");
```

19.0.11. What it does not parse

Each construct in the list at the top of this page answers a non-zero count here.

```
  outside = [
    "const K = 3;",
    "struct S {{ const k: integer }}",
    "fn f() {{ x = 2 ** 3; }}",
    "fn f() {{ x = 6; x /= 2; }}",
    "fn f() {{ x = 0xff; }}",
    "fn f() {{ x = 1.5f; }}",
    "fn f<T>(x: T) -> T {{ x }}",
    "fn f(v: vector<integer>) {{ for e in v if e > 1 {{ }} }}",
    "enum Dir {{ North, South }} fn f(e: Dir) {{ if e is North {{ }} }}",
    "fn f(v: vector<integer>) {{ w = v[1..]; }}",
    "fn f() {{ x = `a`; }}"
  ];
  for snippet in outside {
    assert(parser::parse("outside", snippet) > 0, "{snippet} is outside the
grammar");
  }
  println("parser test passed");
}
```

20. Libraries

A library is a `.loft` file you can share across projects. Place it in a `lib/` directory and import it at the top of your file. `use name;` gives you the `name::` prefix, which reaches every `pub` item the library defines. `use name::\*;` also brings every `pub` name into your own namespace, so you can write them bare; `use name::(a, b);` brings in just those. A definition without `pub` is the library's own: neither the bare name nor `name::thing` reaches it from outside. `pub` is the library's surface. A name you import is yours to use, not to hand on: a file that imports YOURS does not receive it. Write `pub use name::\*;` to pass the names on as well. `use` statements must come before any `fn` or `struct` definition in your file. Putting a `use` after a definition is a syntax error.

```
use testlib::*;
```

20.0.1. Constants and Enums

A library constant reads like one of your own, and so does an enum variant. You can compare, order, and convert enum values exactly as you would with enums defined in the same file. The `libname::` prefix is always available and is what disambiguates a variant name two enums share.

20.0.2. Struct Construction and Field Access

A library struct is constructed like any other. For a `pub` type the prefix is optional — `testlib::Point {…}` and `Point {…}` build the same value — and it is what you reach for when a name of your own would otherwise win. Field access never takes a prefix.

20.0.3. Calling Library Methods

Methods defined in the library are called on the value directly — no prefix on the call site. A `pub` free function is callable bare as well; the prefix is how you say which library you meant.

20.0.4. Extending a Library Type

You can add new methods to a library type from your own file. Write the `self` parameter type with the `::` separator and the compiler treats the function as a method on that type.

```
fn shifted(self: testlib::Point, dx: float, dy: float) -> testlib::Point {
  testlib::Point {x: self.x + dx, y: self.y + dy }
}
```

```
fn main() {
```

A library constant, both ways: the prefix always works, and the bare name works because `use testlib::\*;` brought every `pub` name into this file.

```
  assert(testlib::MAX_SIZE == 100, "library constant MAX_SIZE");
  assert(MIN_SIZE == 1, "the same constant, written bare");
```

Library enum variants: comparison and ordering work as normal.

```
s = testlib::Ok;
assert(s == testlib::Ok, "enum equality");
assert(s < testlib::Error, "enum ordering");
assert("{s}" == "Ok", "enum to text: {s}");
assert("Warning" as testlib::Status == testlib::Warning, "text to enum");
```

Library enum values can be passed to library free functions.

```
assert(testlib::status_text(testlib::Error) == "err", "free fn with enum arg");
```

A struct built through the prefix, and read by plain field access.

```
p = testlib::Point {x: 3.0, y: 4.0 };
assert(p.x == 3.0, "field access: x={p.x}");
assert(p.y == 4.0, "field access: y={p.y}");
```

Methods defined in the library are called without a prefix.

```
dist = p.distance();
assert(round(dist) == 5.0, "method call: distance={dist}");
```

User-defined method on a library type (self: testlib::Point) works.

```
assert(p.shifted(1.0, 2.0).x == 4.0, "shifted x");
assert(p.shifted(0.0, 2.0).y == 6.0, "shifted y");
```

Scalar fields can be mutated directly.

```
b = testlib::Bag {label: "test", count: 0 };
b.label = "updated";
assert(b.label == "updated", "direct scalar field mutation");
```

Methods that mutate scalar fields work.

```
b.bump();
assert(b.count == 1, "method scalar mutation: count={b.count}");
b.bump();
assert(b.count == 2, "method scalar mutation: count={b.count}");
```

Free functions: a `pub` one answers to either spelling.

```
assert(testlib::add(3, 4) == 7, "free function add");
assert(add(0, -5) == -5, "the same free function, written bare");
```

A definition the library did NOT mark `pub` is not reachable from outside at all — neither `internal\_add(3, 4)` nor `testlib::internal\_add(3, 4)` compiles here; the second answers "`internal\_add` is not `pub` in `testlib`". A library's surface is its `pub` items, by either spelling. The library's own `pub` functions still use them:

```
  assert(testlib::add(3, 4) == 7, "the pub surface answers");
```

Library types work as function parameter types when written with their full namespace prefix in the function signature. A struct with a vector field can be initialised via a struct literal, including elements that are themselves library structs.

```
  bag2 = testlib::Bag {label: "full", count: 3, items: [
  testlib::Point {x: 1.0, y: 0.0 },
  testlib::Point {x: 0.0, y: 1.0 }
  ] };
  assert(len(bag2.items) == 2, "initialized vector field: {len(bag2.items)}");
  assert(bag2.items[0].x == 1.0, "vector field element access");
```

Appending a struct variable with `+= \[var\]` and a whole vector with `+= vec` both work.

```
  extra = testlib::Point {x: 7.0, y: 8.0 };
  bag2.items +=[extra];
  assert(len(bag2.items) == 3, "append via var: len={len(bag2.items)}");
  assert(bag2.items[2].x == 7.0, "append via var: x={bag2.items[2].x}");
  more_pts =[testlib::Point {x: 9.0, y: 10.0 }, testlib::Point {x: 11.0, y:
12.0 }];
  bag2.items += more_pts;
  assert(len(bag2.items) == 5, "append vector: len={len(bag2.items)}");
  assert(bag2.items[3].x == 9.0, "append vector[3].x={bag2.items[3].x}");
```

Importing the same library twice is silently ignored. A library can itself import other libraries using `use`, so dependency chains work.

20.0.5. Package Layout and loft.toml

A library can be distributed as a directory package instead of a single flat file. The packaged directory layout is:

```
mylib/
  loft.toml           optional manifest
  src/
    mylib.loft        library source (default entry)
```

The directory form is found in any search directory you NAMED — `--lib \<dir\>`, a path in `LOFT\_LIB`, `~/.loft/lib/` — and in your own project's `lib/` once your program is itself a package, with a `loft.toml` of its own at the top. In all of those, `\<dir\>/mylib/` resolves to `\<dir\>/mylib/src/mylib.loft` on its own.

A lone script with no manifest above it reads its `lib/` flat, so there a library must be the single file `lib/mylib.loft` — a `lib/mylib/` directory beside such a script answers "Library 'mylib' not found". Giving the script a `loft.toml` is what turns the directory form on.

The `loft.toml` manifest carries the whole package declaration — name, version, dependencies, native crates, build targets. Two of its settings decide how the library above is LOADED:

```
[package]
loft = ">=2026.10"    the oldest loft release that can load it
```

```
[library]
entry = "src/mylib.loft"   override the default entry path
```

If the running loft is older than the stated minimum, loading the library produces a fatal compile error describing the version mismatch. If no manifest is present, the default entry `src/\<name\>.loft` is used.

20.0.6. Wildcard and Selective Imports

A bare `use lib;` brings in only the `lib::` prefix, and `use lib::\*` brings in every `pub` name. The selective forms give you a say in WHICH names arrive: `use lib::Name` takes one, and `use lib::(Name, Other)` takes a group. Multiple names must be parenthesised — the flat `use lib::Name, Other` is refused ("import multiple names with parentheses"). Only `pub` definitions can be named this way; the rest are not reachable at all.

Taking only what you name is the stronger position, because it makes you immune to the library GROWING a name. A wildcard import and your own definition of a name the library exports are a conflict, and it is refused naming both sites; under a selective import every name you did not ask for stays yours.

`use lib as short;` gives you the qualifier under another name — `short::name` — and, like a bare `use lib;`, brings nothing in bare. Single names alias too: `use lib::Name as N`, or `use lib::(Name as N, other)`.

20.0.7. Limitations

`use` must appear before all definitions. If you write a function first and then a `use`, the compiler reports a syntax error.

`pub` decides the surface. Only `pub` definitions come in bare, can be named by an import, or answer to `lib::name`; a library's non-`pub` items are its own.

Native extensions. A library can be pure `.loft`, or ship a Rust crate beside it: declare `\[library\] native = "..."` in `loft.toml` and loft builds and links it for you. See doc/claude/PACKAGES.md in the loft repository for the build + binding model.

```
}
```

21. Store Locks

Loft gives you two separate ways to protect a value from accidental changes:

1. **Compile-time const**: mark a variable or parameter with ‘const’ and the

```
compiler refuses to compile the writes that word forbids. The check happens
before the program even runs — zero runtime cost.
```

2. **Runtime lock**: the ‘#lock’ attribute on a reference lets you lock a

```
store at runtime so that any write attempt panics immediately, even across
function boundaries. This is useful for debugging: turn it on when you
suspect an unexpected mutation.
```

```
struct Counter {
  value: integer
}
```

21.0.1. The two places ‘const’ can go

A name has a slot, and the slot holds a value. Those are two different things to freeze, so ‘const’ has two positions and they mean opposite halves:

- **‘const’ before the NAME** — `const x = …` — freezes the **slot**. You can

```
never point `x` at something else, but the value it holds stays mutable:
you may still append to it and write its elements and fields. This is the
builder shape — set it up once, fill it in place.
```

- **‘const’ before the TYPE** — `x: const T` — freezes the **value**. You may

```
swap in a whole new value, but you can never reach into the one that is
there. This is the read-only shape — a shared table, a parameter a function
promises not to touch.
```

Write both — `const x: const T` — and the name is fully immutable.

A plain number or boolean has no interior separate from its slot, so for those the two positions mean the same thing: both freeze it completely.

‘const’ before the TYPE is what a function uses to promise “I will not modify this”. Every write through the parameter is a compile error: appending to it, writing an element, writing a field, or writing a field of a field.

```
fn read_value(self: const Counter) -> integer {
  self.value
}
```

A parameter without ‘const’ can write, and the write reaches the caller.

```
fn increment(self: Counter) {
  self.value += 1
}
```

```
fn main() {
```

21.0.2. const local variables

`const` before the name is the common case for a local: a configuration value or a lookup table that should never be pointed somewhere else deep inside a long function.

```
  const limit = 100;
  assert(limit == 100, "const integer is readable");
```

For a whole number the two positions agree, so this one is frozen outright: `limit = 200` and `limit += 1` are both compile errors.

A const local holding a collection is the builder shape — the slot is write-once, the contents keep growing.

```
  const totals = [1, 2, 3];
  totals += [4];
  totals[0] = 9;
  assert(len(totals) == 4, "a const local still appends: {len(totals)}");
  assert(totals[0] == 9, "a const local still takes an element write");
```

The other position freezes the contents instead, and leaves the slot free.

```
  frozen: const vector<integer> = [1, 2, 3];
  frozen = [7, 8];
  assert(len(frozen) == 2, "a value-const local still re-points: {len(frozen)}");
```

`frozen += \[9\]` would not compile: the value is read-only.

The same split applies to a struct.

```
  const cfg = Counter {value: 42 };
  assert(cfg.value == 42, "const reference is readable");
  assert(read_value(cfg) == 42, "const passed to const param");
```

21.0.3. Runtime store locks with #lock

`const` is checked once, when the program is compiled. A lock is checked on every write, while the program runs — so it catches a write the compiler cannot see coming, wherever in the program it happens.

`\#lock` is an attribute on any reference variable: a struct, a vector, any value that lives in a store. A freshly created one starts unlocked.

```
  d = Counter {value: 5 };
  assert(!d#lock, "new store starts unlocked");
  d#lock = true;
  assert(d#lock, "store is locked after assignment");
```

Reading a locked store is still fine — only writes are blocked.

```
  assert(read_value(d) == 5, "locked store is still readable");
```

A write is not. `d.value = 6` panics here, and so does `increment(d)` — the guard is on the store, so it fires wherever the write is written. That is the point of the lock: the program stops at the write instead of carrying a value someone changed behind your back.

Set it back to false to lift the guard.

```
  d#lock = false;
  increment(d);
  assert(d.value == 6, "unlocking lets the write through again");
```

21.0.4. When to use each approach

- Use ‘const’ on parameters and locals to express your design intent

```
and get compile-time safety at zero cost.
```

- Use ‘#lock = true’ when you want a runtime tripwire: you suspect some

```
code path is mutating a value it should not touch, and you want the
program to stop at that write rather than corrupt silently.
```

get_store_lock() is the function form of the #lock attribute. Both return the same boolean.

```
  e = Counter {value: 99 };
  assert(get_store_lock(e) == e#lock, "function form matches attribute before
lock");
  e#lock = true;
  assert(get_store_lock(e) == e#lock, "function form matches attribute after
lock");
}
```

22. Parallel execution

The `par(b=worker\_call, threads)` clause on a `for` loop runs a function on every element of a collection in parallel and gives you the results one by one in the loop body.

22.0.1. Why parallel loops?

When you have a large collection and each element can be processed independently — image filters, score calculations, data transforms — `par()` splits the work across CPU cores automatically. You move the per-element work into a function and name it in the clause; the loop body then reads that function's result instead of computing it.

22.0.2. Syntax

```
`for a in src par(b = func(a), N) { body }`
```

- `a` — loop variable, bound to the current element, exactly as in a loop

```
without `par`.  The body may still read it alongside the result.
```

- `b` — result variable, holds the return value of `func(a)`
- `func(a)` — worker function called on each element
- `N` — number of threads (1 = sequential, 4 = typical)

`src` is any source a `for` loop accepts: a vector, a range, a text, or a keyed collection. See "Order" below for what a `hash` source changes.

22.0.3. Two call forms

- **Form 1** — a free function: `par(b=my\_func(a), 4)`
- **Form 2** — a method on the element: `par(b=a.my\_method(), 4)`

A function whose first parameter is `self` answers both forms, as it does everywhere else: `par(b=get(a), 4)` and `par(b=a.get(), 4)` are one call. A free function answers only Form 1 — written as a method on the element, the compiler says the method is unknown.

22.0.4. What a worker may do

The compiler holds two of these for you, and you hold the third:

- **The loop element may be a reference** — a struct, say. Marking it

```
`const` documents that the worker only reads it.  For a plain number the
`const` has nothing to freeze and the compiler says so, so leave it off.
```

- **Every OTHER argument must be a scalar.** A worker runs on its own copy

```
of the heap, so it cannot reach a struct or vector from the calling
scope.  Read what you need into a number before the loop and forward that.
```

- **The worker must not touch anything shared** — no global state, no

```
`println`, no file access.  This one is yours to keep: a worker that
writes shared state is a program loft does not define, and it will not
stop you.  Three workers appending to one file leave one line, and which
line survives changes from run to run.
```

The compiler does refuse the case it can see: a worker that writes to state captured from around the loop.

22.0.5. Order

The body sees results in the order the source produced them, whatever order the threads finished in — so for a vector, a range, a text or a `sorted`, `par` gives exactly what the sequential loop gives, and `N` is a speed knob that never changes the answer.

A `hash` is the one exception, and it is worth knowing before you rely on it: `par` walks it in storage order rather than key order, so the results arrive in a different order from the plain `for` loop over the same hash — and in a different order on each run. Iterate a `sorted` instead when the order matters.

```
struct Score {
  value: integer
}
```

```
struct ScoreList {
  items: vector < Score >
}
```

```
struct DoubledScore {
  label: text,
  doubled: integer
}
```

```
fn double_score(thr_r: const Score) -> integer {
  thr_r.value * 2
}
```

```
fn scale_score(thr_r: const Score, thr_factor: integer) -> integer {
  thr_r.value * thr_factor
}
```

```
fn make_doubled(thr_r: const Score) -> DoubledScore {
  DoubledScore { label: "v{thr_r.value}", doubled: thr_r.value * 2 }
}
```

```
fn get_value(self: const Score) -> integer {
  self.value
}
```

```
fn make_scores() -> ScoreList {
  thr_q = ScoreList { };
  thr_q.items +=[Score {value: 10 }, Score {value: 20 }, Score {value: 30 }];
  thr_q
}
```

```
fn main() {
```

22.0.6. Free function (Form 1)

Each Score's value is doubled by `double\_score` across 4 threads. The loop body sees `b` with the doubled value, in original order.

```
  q = make_scores();
  sum = 0;
  for thr_a in q.items par(thr_b = double_score(thr_a), 4) {
    sum += thr_b;
  }
```

10*2 + 20*2 + 30*2 = 120

```
  assert(sum == 120, "parallel double: sum == 120");
```

22.0.7. Extra Arguments

The worker function can take extra arguments from the calling scope, as long as each is a number, a boolean or a character. Here `scale\_score(a, factor)` passes `factor=3` to every worker.

```
  q1b = make_scores();
  factor = 3;
  scaled_sum = 0;
  for thr_a in q1b.items par(thr_b = scale_score(thr_a, factor), 4) {
    scaled_sum += thr_b;
  }
```

10*3 + 20*3 + 30*3 = 180

```
  assert(scaled_sum == 180, "extra arg: scaled sum == 180");
```

22.0.8. Struct Return

A worker returns whatever its own type says: a number, a boolean, a character, a text, a struct, or a vector. Text and struct results are deep-copied, so they stay valid after the worker thread exits.

```
  q2 = make_scores();
  labels = "";
  for thr_sa in q2.items par(thr_ds = make_doubled(thr_sa), 1) {
    labels += "{thr_ds.label},";
```

```
  }
  assert(labels == "v10,v20,v30,", "struct return: {labels}");
```

22.0.9. Method call (Form 2)

`get\_value` takes `self`, so it is a method and `b=a.get\_value()` is the spelling that reaches it.

```
  q3 = make_scores();
  total = 0;
  for thr_a in q3.items par(thr_b = thr_a.get_value(), 4) {
    total += thr_b;
  }
  assert(total == 60, "method call: total == 60");
```

22.0.10. Any for-source

A range is a source like any other, and a worker over plain numbers wants no `const`.

```
  thr_range_sum = 0;
  for thr_i in 0..5 par(thr_d = thr_double(thr_i), 4) {
    thr_range_sum += thr_d;
  }
```

(0+1+2+3+4) * 2

```
  assert(thr_range_sum == 20, "range source: {thr_range_sum}");
```

22.0.11. Empty Vector

An empty vector is safe — the loop body never executes, no threads are spawned.

```
  empty = ScoreList { };
  thr_n = 0;
  for thr_a in empty.items par(thr_b = double_score(thr_a), 1) {
    thr_n += 1;
  }
  assert(thr_n == 0, "empty: 0 iterations");
```

22.0.12. Sequential fallback

Using `par(..., 1)` runs the worker on a single thread — useful for debugging. The answer is the same one every thread count gives; only the parallelism changes.

```
  q4 = make_scores();
  seq_sum = 0;
  for thr_a in q4.items par(thr_b = double_score(thr_a), 1) {
    seq_sum += thr_b;
  }
  assert(seq_sum == 120, "sequential par(1): same result");
}
```

```
fn thr_double(thr_n: integer) -> integer {
  thr_n * 2
}
```

23. Logging

Printing everything to the console is fine during development, but in a real application you usually want more control: write to a file, filter by severity, and keep the output even after the terminal session ends. Loft's logging functions give you that without changing your source code.

23.0.1. The four severity levels

Choose the level that matches how serious the event is:

- 'log_info' — routine progress; fine to see during development but often

```
silenced in production to keep log files small.
Example: "processing file X", "connected to database".
```

- 'log_warn' — something unexpected happened but the program recovered.

```
Example: "config key missing, using default", "retrying after timeout".
```

- 'log_error' — something went wrong and you need to investigate, but the

```
program can continue (perhaps degraded).
Example: "failed to save record", "unexpected null value".
```

- 'log_fatal' — a condition so serious that normal operation is impossible.

```
Example: "cannot open database", "required config file not found".
```

23.0.2. Configuring the log destination

By default, log calls do nothing — no file is written, no console output is produced. To switch logging on, place a 'log.conf' file in the same directory as your '.loft' file, or pass '–log-conf path/to/log.conf' on the command line. The directory is exact: a 'log.conf' one level up is not found, and a program with no config logs nothing and says nothing.

To see every setting with its default, print a documented template:

```
loft --generate-log-config             # writes it to the terminal
loft --generate-log-config log.conf    # writes it to that file
```

A minimal 'log.conf' looks like this:

```
[log]
file  = log.txt    # write messages here (relative to the .loft file)
level = info       # minimum level to record; choices: info warn error fatal
                   # (the default, when you leave this out, is 'warn')
```

```
[rotation]
max_size_mb = 500  # start a new log file after this many megabytes
```

```
daily       = true # also start a new file at midnight UTC
max_files   = 10   # keep at most this many log files (older ones are deleted)
```

```
[rate_limit]
per_site = 5       # suppress messages from the same source line after 5/minute
```

```
[levels]
# Override the global level for one file, by its name alone:
"debug_tool.loft" = info
"src/"            = error   # a path prefix, matched from the project root
```

23.0.3. Production mode

With ‘production = true’ in the ‘[log]’ section of log.conf, or ‘–production’ on the command line:

- ‘panic()’ becomes a fatal log entry instead of aborting the process.
- A failing ‘assert()’ becomes an error log entry instead of aborting.

The program keeps running and the problem is captured in the log — useful for long-running services where a single error should not bring everything down. It still exits non-zero at the end, so a supervisor watching the exit status learns that something went wrong.

23.0.4. Using format strings in log messages

Log messages are plain text, but you can embed any expression using the same ‘{…}’ format syntax as everywhere else in Loft.

The message is built BEFORE the call decides whether to write it — the argument is an ordinary expression, evaluated like any other. So a message that is discarded still costs what it costs, and that is true whether it was filtered out by the level, suppressed by the rate limit, or dropped because there is no log.conf at all. Keep an expensive call out of a message on a hot path, or guard the whole log line with an ‘if’.

```
fn main() {
```

Without a log.conf these calls do nothing — the tests below pass even though no output is produced.

```
  log_info("starting up");
  log_warn("this is a warning");
  log_error("something went wrong");
  log_fatal("critical failure");
```

23.0.5. Example log.txt output

When a ‘log.conf’ is present with ‘level = info’, the four calls above produce entries like this in ‘log.txt’:

```
2026-09-01T08:35:38.971Z INFO   app.loft:3  starting up
2026-09-01T08:35:38.971Z WARN   app.loft:4  this is a warning
2026-09-01T08:35:38.971Z ERROR  app.loft:5  something went wrong
2026-09-01T08:35:38.971Z FATAL  app.loft:6  critical failure
```

Each line contains: a UTC timestamp to the millisecond, the severity level, the source file and line number, then the message. This makes it easy to search the file for errors or trace back to the exact line that produced a message. The file path is written the way a `\[levels\]` key addresses it: relative to the project root inside a project, the bare file name for a script. A true assert never logs anything — it is only the false case that logs.

```
  assert(true, "this should never fail");
```

23.0.6. Typical usage pattern

In a real program you would write something like:

```
fn process(item: Item) {
    log_info("processing item {item.id}");
    result = do_work(item);
    if !result {
        log_error("work failed for item {item.id}");
        return;
    }
    log_info("finished item {item.id} successfully");
}
```

The log messages give you a record of exactly what the program did and where it went wrong, without cluttering the terminal during normal runs.

```
  println("logging test passed");
}
```

24. Time

Loft's standard library has two time functions, and they answer two different questions:

```
now()     — milliseconds since the Unix epoch (wall-clock time).
ticks()   — microseconds elapsed since program start (monotonic clock).
```

Both return an 'integer' (loft's 64-bit integer) — which they need, since a millisecond timestamp passed 32 bits in 1970 plus a few weeks.

Use 'now()' when you want to know WHEN something happened: timestamps, log entries, anything you will compare against a calendar. Use 'ticks()' when you want to know HOW LONG something took: benchmarks and frame timing. It is unaffected by system clock changes or NTP adjustments, which is exactly what 'now()' cannot promise.

24.0.1. Calendars are a library

These two are the whole clock surface: there is no year, month or weekday in the standard library, and no formatting. That is deliberate — calendar arithmetic is a large, opinionated subject — and it is a package away:

```
use time::*;
fn main() {
    ms = now();
    println(format_iso(ms));                    // 2026-09-01T09:03:58Z
    println("{year(ms)} {month_name(ms)} {day(ms)}, a {weekday_name(ms)}");
    println(format_date(add_days(ms, 30)));     // 2026-10-01
}
```

The 'time' package works on the same millisecond-since-epoch integer 'now()' returns, so nothing has to be converted. Install it with 'loft install time'.

```
fn main() {
```

24.0.2. Wall-clock time: now()

'now()' returns the current time as milliseconds since 1970-01-01T00:00:00 UTC, so it is well past the range a 32-bit number could hold.

```
  t = now();
  assert(t > 2147483647, "now() is a 64-bit millisecond stamp: {t}");
```

It reads the system clock, so it usually moves forward — but it is the system's clock, not yours: an NTP correction or a manual change can step it in either direction. Never subtract two 'now()' values to time something. That is what 'ticks()' is for.

```
  t2 = now();
  assert(t2 > 2147483647, "a second reading is a timestamp too: {t2}");
```

24.0.3. Elapsed time: ticks()

'ticks()' measures microseconds since the program started, so it begins near zero and only ever climbs. It uses a monotonic clock, which is the guarantee 'now()' does not have: it never jumps backward, whatever happens to the system clock while your program runs.

```
  start = ticks();
  assert(start >= 0, "ticks() must be non-negative");
  end = ticks();
  assert(end >= start, "ticks() must be monotonically non-decreasing");
```

24.0.4. Measuring elapsed time

Subtract two 'ticks()' values to get the duration in microseconds. Divide by 1000 to convert to milliseconds.

```
elapsed_us  = end_ticks - start_ticks
elapsed_ms  = elapsed_us / 1000
```

Example: measure how long a loop takes.

```
  t_before = ticks();
  sum = 0;
  for i in 0..1000 {
    sum += i
  }
  t_after = ticks();
  elapsed = t_after - t_before;
  assert(elapsed >= 0, "elapsed time must be non-negative: {elapsed}");
  assert(sum == 499500, "loop produced wrong sum: {sum}");
```

24.0.5. Common patterns

Timestamp a log entry (seconds since epoch):

```
seconds = now() / 1000
```

Seed the random number generator with the current time. The generator is in the 'random' package rather than the standard library, so the import is part of the pattern — without it the name does not resolve:

```
use random::(rand_seed);
rand_seed(now())
```

Simple stopwatch (integer division, so this truncates to whole ms):

```
start = ticks()
... do work ...
log_info("Done in {(ticks() - start) / 1000} ms")
```

```
}
```

25. Safety

Loft catches most mistakes at compile time. The ones that get through are surprises rather than crashes: a value that reads as null, a count in the wrong unit, a write that lands somewhere else than you meant. This page collects the ones that catch people out, and most sections prove themselves with a live example you can run. Helper used in the '??' double-evaluation example below. Increments the call counter and returns the given value unchanged.

```
fn counted_call(calls: &integer, value: integer) -> integer {
  calls += 1;
  value
}
```

Used to obtain a null text value (an uninitialized field is the NUL sentinel).

```
struct NullTextHolder {
  s: text,
}
```

```
struct OptTextHolder {
  s: text?,
}
```

A `boolean?` holds all three states; a plain `boolean` holds two.

```
struct OptBoolHolder {
  b: boolean?,
}
```

Used by the parameter section below: what a callee does to a collection it is GIVEN, versus what it does to a collection it BOUND.

```
fn appended_by_callee(v: vector<integer>)     { v += [4]; }
fn replaced_by_callee(v: vector<integer>)     { v = [7, 7]; }
fn replaced_through_ref(v: &vector<integer>)  { v = [7, 7]; }
```

One table, two indexes on it — see the hash section.

```
struct Item { key: text, count: integer }
struct Catalogue {
  all:    vector<Item>,
  by_key: hash<Item[key]>,
}
```

A field with a declared default, beside one without.

```
struct Tile      { name: text, palette_pick: integer = -1 }
struct PlainTile { name: text, palette_pick: integer }
```

```
fn main() {
```

25.0.1. Null values — hidden reserved values

Every type reserves one special value to mean "nothing here" (null). Be aware of what it is for each type:

- `integer` — the most negative 64-bit integer (-9 223 372 036 854 775 808) is null
- `float/single` — `NaN` (Not a Number) is null
- `character` — the NUL character (code point 0) is null
- `text` — a single NUL character (`'\\0'`) is null; the empty string `""` is NOT null
- `reference` — record 0 is null
- `boolean` — a third byte state, which is neither `false` nor `true`
- plain `enum` — both the 0 byte and the 255 byte are reserved, so an

```
                    enum holds at most 254 variants
```

For the first four, the reserved value is a value you can write, so there is one value per type that you cannot tell apart from null. For integers, that is −9 223 372 036 854 775 808. When defended with `?? null` (or a following null-check), division by zero produces this same value, so both paths look the same to your code:

```
  zero = 0;
  n = 1 / zero ?? null;
  assert(!n, "div-by-zero defended with `?? null` is null");
  assert(n != 0, "null is not zero; it is -9 223 372 036 854 775 808");
  assert(n < 0, "i64::MIN is the most negative 64-bit integer");
```

Arithmetic on null propagates: null plus anything is null.

```
  assert(! (n + 1), "null + 1 is still null");
```

Text null is the NUL character ('\0'), not the empty string. A `text?` local or field can hold it; a plain `text` cannot, and a parse respects that.

```
  holder = NullTextHolder.parse(`{{}}`);
  assert(holder.s == "", "a missing `text` field is empty, because it cannot be
null");
  opt = OptTextHolder.parse(`{{}}`);
  assert(!opt.s, "a missing `text?` field IS null — the declaration decides");
  empty = "";
  assert(empty, "empty string is NOT null — this surprises most newcomers");
```

`boolean` is the exception, and the useful one: its null is a third byte state that no boolean expression can produce, so `false` is a value like any other and a `boolean?` tells all three apart.

```
assert(!(false == null), "`false` is a value, not the boolean null");
maybe = OptBoolHolder.parse(`{{}}`);
assert(maybe.b == null, "a missing `boolean?` field IS null");
assert(maybe.b ?? true, "…so `??` reaches its default");
stored = OptBoolHolder { b: false };
assert(!(stored.b == null), "…while a stored `false` is not null");
assert(!(stored.b ?? true), "…and `??` keeps it");
```

25.0.2. Parsing text to a number needs a fallback

An unchecked `text as integer` is a COMPILE error: parsing can fail, and loft refuses to let that failure slip through as a silent null. Ask for a checked cast with `integer?` (yields the number or null), or supply a default with `?? \<value\>`. The same rule applies to `float`.

```
parsed = "42" as integer?;
assert(parsed == 42, "checked cast `as integer?` yields the number");
fallback = "oops" as integer? ?? -1;
assert(fallback == -1, "unparseable text falls back to the default");
```

25.0.3. Integer overflow yields null

A calculation that overflows the integer range is uncomputable, so loft yields null and keeps running — it never wraps to a silently-wrong number (the way C, or a Rust release build, would) and it never crashes. One bad value degrades locally; the rest of the program still computes.

```
huge = 9223372036854775807; huge + 1  →  null (not a wrapped negative)
```

Unlike a divide by zero, an overflow writes no log entry and prints no warning at any setting: the null IS the report, and there is nothing to switch on to get a second one. So check the result for null, or keep intermediate values within range.

```
huge = 9223372036854775807;
assert(! (huge + 1), "overflow yields null, not a wrapped value");
```

25.0.4. Bitwise operators with zero

All bitwise operators (AND, OR, XOR, shift) work correctly with zero. Zero is the identity element for OR, XOR, and shift; zero for AND.

```
assert(0b1010&0 == 0, "AND with 0: zero");
assert(0b1010|0 == 0b1010, "OR with 0: identity");
assert(0b1010^0 == 0b1010, "XOR with 0: identity");
assert(5<<0 == 5, "shift left by 0: identity");
assert(5>>0 == 5, "shift right by 0: identity");
```

25.0.5. Float null compares uniformly with every other scalar

Floats store null as `NaN` internally, but null COMPARISON is uniform with every other scalar type: `null == null` is TRUE, null is not equal to any real value, and null orders as the low extreme. Detect a null float with `f == null` (or `!f` / `f ?? default`) — NOT the `f != f` trick from other languages, which

never fires here because `null == null` is true. Both the defended and the undefended division yield null and the program continues; the undefended site also writes a warning to the log.

```
bad = 0.0 / 0.0 ?? null;
assert(!bad, "a null float is falsy");
assert(bad == null, "detect a null float with `== null`");
assert(bad == bad, "null == null is true (uniform null model)");
assert(! (bad != null), "a null float `!= null` is false");
assert(bad != 0.0, "null != 0.0 is true");
```

Use `f == null`, `!f`, or `f ?? default` to check for null floats.

25.0.6. Text length counts characters; size counts bytes

`len()` on text returns the number of characters (Unicode code points) — the human count. `size()` returns the number of UTF-8 bytes; multi-byte characters (accented letters, emoji, CJK) each occupy 2-4 bytes. The two values differ whenever the text contains any multi-byte character.

```
emoji = "Hi 😊!";
assert(len(emoji) == 5, "5 characters (H, i, space, 😊, !)");
assert(size(emoji) == 8, "8 UTF-8 bytes (the emoji is 4): {size(emoji)}");
count = 0;
for _ in emoji { count += 1; }
assert(count == 5, "for-loop iterates by character, not byte");
```

Indexing and slicing take BYTE offsets, and an offset landing inside a multi-byte character is not an error: it rounds OUTWARD to whole characters. So a slice can be longer than the byte range you asked for, and two different ranges can hand back the same text.

```
mixed = "aébc";                 // bytes: a, then 'é' across two, then b, c
assert(mixed[2] == 'é', "byte 2 is the second half of 'é', and reads as 'é'");
assert(mixed[0..2] == "aé", "the end rounds forward: three bytes come back, not
two");
assert(mixed[1..2] == mixed[1..3], "two byte ranges, one character, one
answer");
```

Mitigation: Use `for c in text` to iterate by character. Use `c\#index` and `c\#next` to get the byte boundaries of each character.

25.0.7. `\#index` means different things on text and vectors

On a text loop, `c\#index` is the byte offset of the current character. On a vector loop, `v\#index` is the 0-based element position. Both are called `\#index` but represent different units.

```
v = [10, 20, 30];
idx = 0;
for x in v { idx = x#index; }
assert(idx == 2, "vector #index: element position (0-based)");
```

Text #index is a byte offset, not a character count:

```
t = "aé";
byte_pos = 0;
for c in t { byte_pos = c#index; }
assert(byte_pos == 1, "text #index: byte offset of 'é' (byte 1, not char 1)");
```

25.0.8. ?? evaluates the left side exactly once

The `??` operator means "use this value, or if it is null, use the right side instead". The left-hand expression is evaluated exactly once regardless of whether the result is null. The example below uses `counted\_call` (defined above) to verify this.

```
calls = 0;
qq_result = counted_call(calls, 7) ?? 99;
assert(calls == 1, "?? evaluated left side exactly once: {calls}");
assert(qq_result == 7, "result is the value from that single evaluation:
{qq_result}");
```

This means `result = expensive\_call() ?? default` is safe: the function is called once. If it returns null, `default` is used. There is no double call.

25.0.9. Text indexing and slicing return different types

`txt\[i\]` returns a `character` (a single Unicode scalar value). `txt\[i..j\]` returns `text` (a UTF-8 string). These are different types.

```
txt = "hello";
ch = txt[0];
slice = txt[0..1];
assert(ch == 'h', "indexing returns a character");
assert(slice == "h", "slicing returns text");
```

A character appends to text directly:

```
result = "";
for c in "abc" { result += c; }
assert(result == "abc", "`+=` appends a character to text");
```

25.0.10. Format strings: braces are always interpreted

Every string literal in loft is a format string. Literal braces must be escaped as `{{` and `}}`:

```
n = 42;
assert("{n}" == "42", "single braces: format expression");
assert(len("{{}}") == 2, "double braces produce literal brace chars");
```

Forgetting to escape braces in expected output is a common mistake in assertions and comparisons.

25.0.11. A hash iterates, and collections over one record type index one table

`for e in my\_hash { }` works and visits the records in ascending key order — the same loop shape as `sorted` and `index`, which yield records too. A hash key runs one way only: the `-` prefix that reverses

a `sorted` key is refused on a hash, so reach for `sorted` when the order has to run the other way. The one loop attribute a hash does not carry is `e\#remove`, because the walk is over a sorted snapshot — remove through the key instead, with `my\_hash\[key\] = null`.

loft's store is an in-memory database, and collections over the same record type in one struct are INDEXES ON ONE TABLE — as ordinary here as several indexes on a table are in any database, and declared for the same reason. A vector beside a hash gives you insertion order and keyed lookup over the same rows; add a `sorted` and you have a third way in. The rows live in the table, so filling any one index fills them all, a write through one is visible through the others, and a removal through one removes the row itself.

What that asks of you is only at construction: hand the rows to ONE member and write `\[\]` for the others. That `\[\]` declares an index; it does not declare an empty collection to be filled separately. Hand rows to two members and the table gets both sets, so a vector you wrote two rows into holds four. loft advises on that literal (`advice\[linked-group-double-fill\]`).

```
  cat = Catalogue { all: [Item{key:"zebra", count:1}, Item{key:"apple",
count:5}], by_key: [] };
  insertion = "";
  for e in cat.all { insertion += "{e.key} "; }
  assert(insertion == "zebra apple ", "the vector keeps insertion order:
{insertion}");
  ordered = "";
  for e in cat.by_key { ordered += "{e.key} "; }
  assert(ordered == "apple zebra ", "the hash walks ascending key order:
{ordered}");
  assert(len(cat.by_key) == 2, "…over the records the vector was given:
{len(cat.by_key)}");
```

25.0.12. Mutation guard blocks appending during iteration

The compiler prevents `v += \[x\]` inside `for e in v`. This protects against infinite loops. The guard also catches field access: `for e in db.items { db.items += \[x\]; }` is blocked too. Removing is the one mutation a loop may make: `e\#remove` drops the current element, in a plain loop or in a filtered one.

```
  shrinking = [1, 2, 3];
  for e in shrinking if e > 1 { e#remove; }
  assert(len(shrinking) == 1, "`e#remove` drops the elements the filter
selected");
```

25.0.13. If-expression requires else when used as a value

Using `if` as a value expression without an `else` clause is a compile error. This prevents accidental null values from missing branches. If-statements (where the body has no value) do not need else. For example, `x = if cond { 1 }` is an error; write `x = if cond { 1 } else { 0 }`.

25.0.14. Match guards do not prove a variant is handled

A guarded arm like `Red if cond =\> ...` does not count as handling the `Red` variant because the guard can fail at runtime. Even if every variant has a guarded arm, you still need a wildcard `\_` or an unguarded arm so the compiler knows every case is covered.

25.0.15. A parameter shares; a bind copies

A struct or collection PARAMETER is a shared view of the caller's value — nothing is copied when you call. So everything the function does to the contents is visible to the caller once it returns: writing an element, appending, removing, clearing. None of that needs `&`.

`&` buys exactly one thing: replacing the WHOLE value. `v = \[7, 7\]` inside a plain parameter gives that function a different vector and leaves the caller's alone; through a `&vector\<T\>` the caller sees the new vector. So `&` means "I may replace this", not "I may add to it".

A plain BIND is the opposite, and the pair is what catches people: `d = self.data` COPIES, so `d += \[x\]` leaves `self.data` at its old length. Identical-looking lines, opposite outcomes, decided by whether the name came from a parameter or from an assignment.

```
  shared = [1, 2, 3];
  appended_by_callee(shared);
  assert(len(shared) == 4, "a plain parameter grew the CALLER's vector:
{len(shared)}");
  kept = [1, 2, 3];
  replaced_by_callee(kept);
  assert(len(kept) == 3, "…while replacing the whole value stayed local:
{len(kept)}");
  swapped = [1, 2, 3];
  replaced_through_ref(swapped);
  assert(swapped[0] == 7, "…and `&` is what makes a replacement reach back");
  bound = [1, 2, 3];
  alias = bound;
  alias += [4];
  assert(len(bound) == 3, "a BIND copies — this append reached nothing:
{len(bound)}");
```

25.0.16. Text file reading assumes UTF-8

`lines()` and `content()` decode the file as UTF-8. A file that is not valid UTF-8 — a binary file, or text in another encoding such as Latin-1 — neither crashes nor stops the program: the read warns, answers null, and `lines()` therefore hands back an EMPTY vector. A loop over it runs zero times and the program carries on, so a text read of the wrong file looks exactly like a read of an empty one. Read such a file as bytes instead: `read\_bytes(path)` for the whole file, or set a binary format on the handle — `f\#format = LittleEndian` (or `BigEndian`) — and read fields one at a time (see the File page).

25.0.17. A struct literal zeroes the fields it leaves out

Naming some fields and omitting others is legal, and each omitted field takes its type's zero — `0`, `""`, `false`. Nothing in the declaration chose that value, and where zero is a meaningful member of the field's domain (a palette index whose 0 is a real colour) it is a wrong value rather than an absent one.

loft advises on the partial literal (`advice\[omitted-field-zero\]`). The cure is a declared default on the field, after which the omission has an answer the declaration chose:

```
  plain_tile = PlainTile { name: "grass" };
  assert(plain_tile.palette_pick == 0, "an omitted field takes the type's zero");
  tile = Tile { name: "grass" };
  assert(tile.palette_pick == -1, "a declared default answers instead:
{tile.palette_pick}");
```

25.0.18. A field only some variants declare answers null on the others

Enum payloads are named fields you read straight — `shape.radius`. Where every variant declares the same name and type it is ONE slot and each variant reads its own value. Where only SOME declare it, a value of any other variant has no such field: the read answers null, although the field's type does not say it can, and a write to it is silently dropped. loft warns (`warning\[variant-field-unchecked\]`); bind the field in a `match` arm or an `is` test, which is per-variant and cannot reach the wrong one.

25.0.19. A yield inside an expression runs eagerly on –native

A generator is suspended at each `yield` on both backends when the `yield` is a statement — in any loop, `if`, `match`, nested loop or beside a `continue`, for any yielded type. A `yield` inside an expression is different when an operand before it could change while the generator is suspended, as in `s = s + { yield x; "b" }` with `s` a text. On –native that loop runs to its end before the first value is handed out. The values agree, the side effects do not, and an endless loop of that shape never hands out a value on –native: it runs until memory runs out. Make the `yield` a statement of its own: `yield x; s = s + "b";`.

25.0.20. XOR is ^, not exponentiation

Unlike some languages where ^ means "power", in loft ^ is bitwise XOR. For exponentiation use the `\*\*` operator (`2 \*\* 10 == 1024`, `2.0 \*\* 3.0 == 8.0`) or the `pow()` function. Watch one precedence footgun: a leading `-` binds TIGHTER than `\*\*`, so `-x \*\* y` raises the NEGATED BASE. loft warns on the bare spelling; write the parentheses for whichever of the two you mean.

```
  assert((0b1010^0b1100) == 0b0110, "^ is XOR");
  assert(2**10 == 1024, "** is the power operator");
  assert(pow(2.0, 3.0) == 8.0, "pow() also computes powers");
  base = 2;
  assert((-base) ** 2 == 4, "`-x ** y` is this one: the sign binds to the base");
  assert(-(base ** 2) == -4, "…and this one needs the parentheses");
}
```

26. JSON

Loft has built-in JSON support, and there are two ways in. Reach for the one that matches what you know about the document:

```
* you know its SHAPE — it maps onto a struct you have declared. Serialise
  with the ':j' format flag and parse with 'Type.parse(text)'. No
  annotations and no code generation: it works from the declaration.
* you do NOT know its shape, or you have to report precisely what was wrong
  with it. Parse to a 'JsonValue' with 'json_parse(text)' and walk it.
```

Both are built in and neither is going away. The struct form is shorter and is what most programs want; the ‘JsonValue’ form is the one that can answer “what is in here?” and “what exactly was wrong?”.

26.0.1. Serialisation — struct to JSON

Use the ‘:j’ format flag inside a format string to produce JSON output. Field names become quoted keys; strings are escaped (quotes, backslashes, control characters); numbers and booleans are written as JSON literals.

```
"{my_struct:j}"
```

```
struct User {
  id: integer,
  name: text,
  email: text
}
```

```
struct MaybeUser {
  id: integer,
  name: text?,
}
```

26.0.2. Parsing — JSON to struct

Call ‘Type.parse(text)’ to create a struct from JSON text. A text argument is wrapped through ‘json_parse’ internally, so the one-step form is the whole story for a document you expect to be well-formed.

A field the JSON does not mention — and a field written ‘null’ — gets the DECLARED type’s absent value. For a plain field that is its zero, because a plain field cannot hold null; write the field ‘integer?’ / ‘float?’ if you need to tell “absent” from “zero” apart:

```
struct Reading { id: integer, drift: float? }
r = Reading.parse(`{{}}`)     // r.id == 0, r.drift == null
```

Note the backticks and the doubled braces: every loft string is a format string, so a JSON literal written in source needs ‘{{‘ and ‘}}’ for its own braces (see the Safety page).

26.0.3. Vectors

Parse a JSON array into a vector of structs with ‘vector<T>.parse(text)’.

```
scores = vector<Score>.parse(`[{{"value":1}},{{"value":2}}]`)
```

```
struct Score {
  value: integer
}
```

```
struct Item { key: text, count: integer }
struct Catalogue { all: vector<Item>, by_key: hash<Item[key]> }
```

26.0.4. What went wrong — ‘json_errors()’ and ‘#errors’

Both parse forms report. ‘json_errors()’ holds the LAST parse’s diagnostics and is replaced by the next parse, so a successful parse leaves it empty. It is a TEXT, not a collection — several diagnostics are joined with ‘|’ — so ask ‘json_errors() != “”’, not ‘len(…) > 0’, and never loop over it.

```
if json_errors() != "" { log_warn(json_errors()); }
```

‘record#errors’ answers for the ‘Type.parse(…)’ that built that record, as a vector<text> with one entry per value that did not fit — every mismatch, whether the one-step ‘Type.parse(text)’ or the staged ‘Type.parse(json_parse(text))’ built it; a field whose value did not fit holds the type’s default, the rest is read. Each parsed record keeps its own:

```
for e in user#errors { log_warn(e); }
```

The two parse spellings differ in WHICH HALF of the answer they give, and that is the reason to choose between them:

```
User.parse(text)               ->  line 1:33 path:addr.zip
User.parse(json_parse(text))   ->  Addr.zip: expected JNumber, got JString
```

The one-step form says WHERE in the document, by line, column and field path. The staged form says WHAT was wrong, by declared type against found type — and on malformed input it renders the offending line with a caret. Take the first when a human will look at the document, the second when your program has to explain the mismatch.

26.0.5. Nested Structs

Structs with struct-typed fields parse nested JSON objects automatically.

```
struct Address {
  city: text,
  zip: text
}
```

```
struct Contact {
  name: text,
  address: Address
}
```

26.0.6. Reading a document whose shape you do not know

‘json_parse(text)’ answers a ‘JsonValue’ — a tree you can ask about instead of a struct you had to declare first. ‘kind()’ names the variant, ‘field(name)’ and ‘item(i)’ step into objects and arrays, ‘keys()’ lists an object’s field names, and the ‘as_*’ extractors answer null on a kind mismatch rather than faulting. Every miss answers ‘JNull’, so a walk over an unexpected document degrades instead of stopping.

```
struct Tagged { tag: text, n: integer }
```

Used by the wire-shape section at the end.

```
enum Shade { Red, Green }
struct Flag   { on: Shade }
struct Pair   { p: (integer, text) }
struct Holder { n: integer, rows: vector<Item>? }
struct Leaf   { name: text }
struct Pick   { leaves: vector<Leaf>, best: reference<Leaf>? }
struct Link   { label: text, to: reference<Leaf> }
```

```
fn main() {
  u = User { id: 42, name: "Alice", email: "alice@example.com" };
  json = "{u:j}";
  expected = `{{"id":42,"name":"Alice","email":"alice@example.com"}}`;
  assert(json == expected, "to json");
```

```
  bob = User.parse(`{{"id":7,"name":"Bob","email":"bob@test.org"}}`);
  assert(bob.id == 7, "parsed id: {bob.id}");
  assert(bob.name == "Bob", "parsed name");
  assert(bob.email == "bob@test.org", "parsed email");
```

```
  u2 = User.parse("{u:j}");
  assert(u2.id == u.id, "round-trip id");
  assert(u2.name == u.name, "round-trip name");
```

Escaping is not optional and not yours to do: a quote, a backslash and a control character all come back as JSON requires.

```
  quoted = User { id: 1, name: "a\"b\\c", email: "x\ny" };
  assert("{quoted:j}" == `{{"id":1,"name":"a\\"b\\\\c","email":"x\\ny"}}`,
         "text is escaped: {quoted:j}");
```

A type-mismatched field leaves the record at its DECLARED default — `id` is plain, so that is 0, never a null in a slot the declaration says cannot hold one — and the mismatch is reported through `json\_errors()`.

```
  bad = User.parse(`{{"id":"not_a_number","name":42}}`);
  assert(bad.id == 0, "type-mismatched id keeps its default: {bad.id}");
  assert(json_errors() != "", "the mismatch was reported: [{json_errors()}]");
```

...and EVERY mismatch is named, on the record: `id` and `name` both, in either form.

```
  assert(len(bad#errors) == 2, "both mismatches on the record: {bad#errors}");
  staged = User.parse(json_parse(`{{"id":"not_a_number","name":42}}`));
  assert(len(staged#errors) == 2, "the staged form names both too:
{staged#errors}");
```

...and the next parse replaces it, so a good parse reads clean.

```
  fine = User.parse(`{{"id":2,"name":"n","email":"e"}}`);
  assert(fine.id == 2, "the good parse landed");
  assert(json_errors() == "", "a successful parse clears the errors:
[{json_errors()}]");
```

```
  scores = vector<Score>.parse(`[{{"value":10}},{{"value":20}},{{"value":30}}]`);
  total = 0;
  for s in scores {
    total += s.value;
  }
  assert(total == 60, "vector sum: {total}");
```

An element the declaration cannot take keeps its slot at the defaults, and the elements around it are read — as a struct keeps its other fields; `\#errors` names it.

```
  spoiled = vector<Score>.parse(`[{{"value":"x"}},{{"value":2}}]`);
  assert(len(spoiled) == 2, "one bad element keeps its slot: {len(spoiled)}");
  assert(spoiled[0].value == 0 && spoiled[1].value == 2, "the fitting element is
read: {spoiled}");
  assert(len(spoiled#errors) == 1, "the mismatch is named: {spoiled#errors}");
```

```
  c = Contact.parse(`{{"name":"Carol","address":
{{"city":"Amsterdam","zip":"1012"}}}}`);
  assert(c.name == "Carol", "nested name");
  assert(c.address.city == "Amsterdam", "nested city: {c.address.city}");
  assert(c.address.zip == "1012", "nested zip");
```

26.0.7. A missing field gets its own declared absence

When a field is absent from the JSON, it gets the absent value its DECLARATION allows — not the type's null sentinel. A plain field cannot hold null, so it takes its empty value: 0 for a number, "" for

text, false for a boolean. Declare the field ‘T?’ when “the document did not say” has to be tellable from “the document said zero”, and check it with ‘!’ or ‘??’ before use.

```
  partial = User.parse(`{{"id":1}}`);
  assert(partial.id == 1, "partial id");
  assert(partial.name == "", "a missing `text` field is empty:
[{partial.name}]");
  maybe = MaybeUser.parse(`{{"id":1}}`);
  assert(!maybe.name, "a missing `text?` field is null");
```

Walking a document by shape rather than by declaration.

```
  doc = json_parse(`{{"tag":"reading","n":3,"items":["a","b"]}}`);
  assert(doc.kind() == "JObject", "the document is an object: {doc.kind()}");
  assert(doc.has_field("tag"), "it carries `tag`");
  assert((doc.field("n").as_long() ?? -1) == 3, "n is 3");
  assert((doc.field("tag").as_text() ?? "") == "reading", "tag is `reading`");
  assert(len(doc.keys()) == 3, "three keys: {len(doc.keys())}");
  items = doc.field("items");
  assert(items.kind() == "JArray", "items is an array");
  assert((items.item(1).as_text() ?? "") == "b", "second item is `b`");
```

A field that is not there is `JNull`, not a fault — so a walk keeps going.

```
  assert(doc.field("absent").kind() == "JNull", "a missing field answers JNull");
  assert(!doc.field("n").as_text(), "an `as_` on the wrong kind answers null");
```

Once you know the shape, hand the same tree to the struct form.

```
  t = Tagged.parse(doc);
  assert(t.tag == "reading", "the tree parses into a struct too: {t.tag}");
```

Building a document without a struct to describe it.

```
  reply = json_object([
    JsonField { name: "ok", value: json_bool(true) },
    JsonField { name: "n",  value: json_number(42.0) },
  ]);
  assert(reply.to_json() == `{{"ok":true,"n":42}}`, "built: {reply.to_json()}");
```

26.0.8. The shapes on the wire

Five encodings a reader interoperating with another system needs, none of which the field declaration makes obvious:

```
* a plain enum variant is an OBJECT keyed by the variant name — `Green`
  is `{"Green":{}}`, not the string `"Green"`;
* a tuple is an object with `_0`, `_1` … keys, not a JSON array;
* an absent field is written `null`, an absent `vector<T>?` too — it is
```

```
  not `[]`, which is the present-but-empty collection and a different value;
* a keyed member of a linked collection group is left out as well, because
  it is an INDEX on the same rows rather than data of its own — and parsing
  the document back rebuilds it from them;
* a `reference<T>` field is a LINK, so it is written as one: a JSON Pointer
  to the record it names, `{"$ref":"#/leaves/1"}`. A record the written
  value does not contain is written once under a last member `$refs`, and
  the reference points there. Parsing reads both back, and two references
  to one record name one record again.
```

```
  colour = Flag { on: Shade.Green };
  assert("{colour:j}" == `{{"on":{{"Green":{{}}}}}}`, "an enum variant:
{colour:j}");
  pair = Pair { p: (7, "seven") };
  assert("{pair:j}" == `{{"p":{{"_0":7,"_1":"seven"}}}}`, "a tuple: {pair:j}");
  nothing = Holder { n: 1, rows: null };
  assert("{nothing:j}" == `{{"n":1,"rows":null}}`, "an absent collection is
written null: {nothing:j}");
  something = Holder { n: 1, rows: [] };
  assert("{something:j}" == `{{"n":1,"rows":[]}}`, "an empty one is not:
{something:j}");
  assert(Holder.parse("{nothing:j}").rows == null, "absent comes back absent");
  assert(!(Holder.parse("{something:j}").rows == null), "empty comes back
empty");
```

The index is derived, so it costs nothing on the wire and is rebuilt on the way back in.

```
  cat2 = Catalogue { all: [Item{key:"a", count:1}], by_key: [] };
  assert("{cat2:j}" == `{{"all":[{{"key":"a","count":1}}]}}`,
         "the index is not written: {cat2:j}");
  back = Catalogue.parse("{cat2:j}");
  assert(len(back.by_key) == 1, "…and is rebuilt on the way in:
{len(back.by_key)}");
  assert(back.by_key["a"].count == 1, "the rebuilt index answers a lookup");
```

A reference into the same value points at its place in the document.

```
  pick = Pick { leaves: [Leaf { name: "a" }, Leaf { name: "b" }] };
  pick.best = &pick.leaves[1];
  assert("{pick:j}" == `{{"leaves":[{{"name":"a"}},{{"name":"b"}}],"best":
{{"$ref":"#/leaves/1"}}}}`,
          "a reference is a pointer: {pick:j}");
  picked = Pick.parse("{pick:j}");
  picked.best.name = "B";
  assert(picked.leaves[1].name == "B", "…which reads back as a link to that
element");
```

A reference to a record outside the value brings the record along.

```
  far = Leaf { name: "far" };
  link = Link { label: "x", to: &far };
  assert("{link:j}" == `{{"label":"x","to":{{"$ref":"#/$refs/0"}},"$refs":
[{{"name":"far"}}]}}`,
         "an outside target is written under $refs: {link:j}");
  linked = Link.parse("{link:j}");
  assert(linked.to.name == "far", "…and is read back: {linked.to.name}");
}
```

27. Generics

A generic function uses a type variable to work with any type. Write the function once; the compiler creates a specialised copy for each concrete type you call it with.

27.0.1. Declaring a generic function

Place a type variable in angle brackets after the function name. The type variable must appear in a parameter (directly or as a container element like vector<T>) — any parameter; a T that appears only in the return type is refused, since a call infers T from its arguments. A header may declare several variables — `\<K: Printable, V: Printable\>` — each with its own bounds, each inferred from the parameters that name it.

A struct may declare type variables too — see Generic structs below.

```
fn identity<T>(x: T) -> T { x }
```

27.0.2. Calling a generic function

No special syntax at the call site — the compiler infers T from the arguments' types and creates a specialised copy automatically.

```
fn test_identity() {
  assert(identity(42) == 42, "identity integer");
  assert(identity(3.14) == 3.14, "identity float");
  assert(identity("hello") == "hello", "identity text");
  assert(identity(true) == true, "identity bool");
}
```

27.0.3. Multiple parameters of the same type

Additional parameters can also use T. They all share the same concrete type.

```
fn pick_second<T>(gen_a: T, gen_b: T) -> T {
  _x = gen_a;
  gen_b
}
fn test_pick_second() {
  assert(pick_second(1, 99) == 99, "pick second int");
  assert(pick_second("a", "b") == "b", "pick second text");
}
```

27.0.4. Generic functions on vectors

The most common use of generics: write a function that works on any vector. T is inferred from the vector's element type.

```
fn first_element<T>(gen_v: vector<T>) -> T {
  gen_v[0]
}
```

A computed index `gen\_v.len() - 1` is not provably in-bounds, so the read is nullable; a generic has no value of T of its own to write after `??`, so the honest result type is `T?` (or take the fallback as a parameter: `v\[i\] ?? d`).

```
fn last_element<T>(gen_v: vector<T>) -> T? {
  gen_v[gen_v.len() - 1]
}
fn test_vector_generics() {
  ints = [10, 20, 30];
  assert(first_element(ints) == 10, "first int");
  assert(last_element(ints) == 30, "last int");
  words = ["alpha", "beta", "gamma"];
  assert(first_element(words) == "alpha", "first text");
  assert(last_element(words) == "gamma", "last text");
```

A CONSTANT index is trusted, and an empty vector is where that trust runs out: `gen\_v\[0\]` is typed `T` and still answers null on an empty vector, so the null travels through a return type that says it cannot be there. Check the length before calling, or declare the result `T?` as `last\_element` does and let the caller discharge it.

```
  empty: vector<integer> = [];
  assert(!first_element(empty), "`v[0]` on an empty vector is null despite the
`T` return");
}
```

27.0.5. Bounded generics with interfaces

By default, you can only assign and return T — no arithmetic, and no comparison either: `a == b` on an unbounded T is refused. To use an operator on T, add an interface bound.

Each bound guarantees EXACTLY the operations in this table, and no more — a `+` under `Numeric` is refused as surely as one under no bound at all. Each names the FEWEST methods it needs, because the operators are derived: `\<`, `\<=`, `\>` and `\>=` all read `compare`, and `!=` comes from `==`.

```
Ordered    — declares `compare`; gives you `<`, `<=`, `>`, `>=`
Equatable  — declares nothing; every type has `==` and `!=`
Addable      — `+` only.  NOT `-`
Numeric      — `*` and the UNARY `-a`.  NOT `+`, NOT `/`
Subtractable — the BINARY `a - b`
Scalable   — the method `scale(self, factor: integer) -> integer`.  Not an
             operator, and no built-in type satisfies it — it is for your own
types
Printable  — the method `to_text() -> text`, and `"{x}"` interpolation
Walkable   — the method `children() -> vector<Self>`.  The standard library's
             `tree_walk` is bounded by it, so defining `children` on your own
             type gets you a breadth-first walk without writing one
```

Reach for `Ordered + Addable` when you want + and a comparison.

- desugars to the same `OpMin` name at BOTH arities, so unary negation and binary subtraction are two SIGNATURES of one name — and therefore two bounds: `\<T: Numeric\>` gives you `-a`, `\<T: Subtractable\>` gives you `a - b`, and `\<T: Numeric + Subtractable\>` gives you both. A user type may declare `OpMin` at both arities, in either order: `a - b` reaches the binary one and `-a` the unary one, so it satisfies both bounds.

```
fn gen_diff<T: Subtractable>(gen_a: T, gen_b: T) -> T { gen_a - gen_b }
fn gen_negdiff<T: Numeric + Subtractable>(gen_a: T, gen_b: T) -> T { -(gen_a -
gen_b) }
```

```
fn gen_max<T: Ordered>(gen_x: T, gen_y: T) -> T {
  if gen_x > gen_y { gen_x } else { gen_y }
}
fn gen_min<T: Ordered>(gen_x: T, gen_y: T) -> T {
  if gen_x < gen_y { gen_x } else { gen_y }
}
fn same<T: Equatable>(gen_x: T, gen_y: T) -> boolean {
  gen_x == gen_y
}
fn described<T: Printable>(gen_x: T) -> text {
  gen_x.to_text()
}
fn test_bounded() {
  assert(gen_max(3, 7) == 7, "max int");
  assert(gen_max(2.5, 1.5) == 2.5, "max float");
  assert(gen_min(3, 7) == 3, "min int");
  assert(gen_min("apple", "banana") == "apple", "min text");
  assert(gen_diff(10, 3) == 7, "subtraction under a bound");
  assert(gen_diff(2.5, 0.5) == 2.0, "subtraction under a bound, float");
  assert(gen_negdiff(10, 3) == -7, "both arities of `-` on one variable");
  assert(same(4, 4), "equatable on integers");
  assert(!same("a", "b"), "…and it says no as well");
  assert(described(7) == "7", "printable: {described(7)}");
}
```

27.0.6. Combining bounds

Use + to require multiple interfaces: `\<T: Ordered + Addable\>`.

```
fn clamped_add<T: Ordered + Addable>(gen_a: T, gen_b: T, gen_hi: T) -> T {
  result = gen_a + gen_b;
  if result > gen_hi { gen_hi } else { result }
}
fn test_combined_bounds() {
  assert(clamped_add(3, 4, 10) == 7, "under limit");
  assert(clamped_add(8, 5, 10) == 10, "clamped to limit");
  assert(clamped_add(1.0, 2.0, 2.5) == 2.5, "float clamped");
}
```

27.0.7. Your own types under a bound

The built-in bounds are not only for built-in types, and this is what they are for: a struct satisfies one by DEFINING the operation, with no declaration that it does. An operator is an `operator` method — written with the keyword `operator` where a function has `fn` — named for what it does:

```
Ordered       operator compare(self: T, other: T) -> Ordering   (`<` `<=` `>` `>=`)
Addable       operator plus(self: T, other: T) -> T             (`+`)
Subtractable  operator minus(self: T, other: T) -> T            (binary `-`)
Numeric       operator times(self: T, other: T) -> T            (`*`)
              operator negate(self: T) -> T                     (unary `-`)
Scalable      fn scale(self: T, factor: integer) -> integer
Printable     operator to_text(self: T) -> text
Walkable      fn children(self: T) -> vector<T>
```

`compare` answers `Less`, `Equal` or `Greater`; every built-in ordered type has one, so `self.cents.compare(other.cents)` is the whole body for one field. `Equatable` is met by every type: `==` always compares what two values hold — a struct field by field — and no type can redefine it; a comparison of your own is a named method.

The same definition serves the bare operator: once `Money` has `compare`, both `a \< b` and `\<T: Ordered\>` work on it. Miss one and the error names it — "'Money' does not satisfy interface 'Ordered': missing `operator compare(self: Money, other: Money) -\> Ordering`" — at the CALL site, because that is where the concrete type is known.

```
struct Money { cents: integer }
operator compare(self: Money, other: Money) -> Ordering
{ self.cents.compare(other.cents) }
operator plus(self: Money, other: Money) -> Money { Money { cents: self.cents +
other.cents } }
operator minus(self: Money, other: Money) -> Money { Money { cents: self.cents -
other.cents } }
operator negate(self: Money) -> Money { Money { cents: 0 - self.cents } }
operator times(self: Money, other: Money) -> Money { Money { cents: self.cents *
other.cents } }
operator to_text(self: Money) -> text { "{self.cents}c" }
fn test_user_type_bounds() {
  cheap = Money { cents: 300 };
  dear = Money { cents: 700 };
  best = gen_max(cheap, dear);
  assert(best.cents == 700, "Ordered on a user type: {best.cents}");
  assert(cheap < dear, "…and the bare operator, from the same definition");
  assert(same(cheap, Money { cents: 300 }), "Equatable on a user type");
  assert(same(Tile { height: 1, name: "t" }, Tile { height: 1, name: "t" }), "…
and on one that defines no operator");
  assert(!same(Tile { height: 1, name: "t" }, Tile { height: 2, name: "t" }), "…
comparing every field");
  total = clamped_add(cheap, cheap, Money { cents: 1000 });
  assert(total.cents == 600, "Addable on a user type: {total.cents}");
```

```
  assert(described(dear) == "700c", "Printable on a user type:
{described(dear)}");
  assert(gen_diff(dear, cheap).cents == 400, "Subtractable on a user type");
  assert(gen_negdiff(dear, cheap).cents == -400, "Numeric + Subtractable on a
user type");
}
struct Crate { label: text, kids: vector<Crate> }
fn children(self: Crate) -> vector<Crate> { self.kids }
fn test_walkable() {
```

`children` is the whole of what Walkable asks for; `tree\_walk` is the stdlib's, bounded by it, and it visits the root before either child.

```
  tree = Crate { label: "root", kids: [
    Crate { label: "a", kids: [] },
    Crate { label: "b", kids: [] },
  ] };
  seen = "";
  for node in tree_walk(tree, 10) { seen += "{node.label} "; }
  assert(seen == "root a b ", "Walkable on a user type: {seen}");
}
```

27.0.8. Generic structs

A struct may declare type variables: `struct Box\<T\> { v: T }`. `Box\<integer\>` names an INSTANCE — an ordinary struct whose `v` is an `integer`, laid out as a hand-written `struct BoxInteger { v: integer }` would be. A literal builds the instance its annotation names, or else the one its field values bind: `Box { v: 1 }` is a `Box\<integer\>`, as `identity(1)` makes T an integer. A literal with nothing to bind a variable from (`Box {}`, `Box { v: \[\] }` for a `vector\<T\>` field) is refused, naming the variable; give the binding its type.

```
struct Box<T> { v: T }
struct Stack<T> { items: vector<T>, name: text }
fn twice(b: Box<integer>) -> integer { b.v * 2 }
fn test_generic_struct() {
  b = Box { v: 21 };
  t = Box { v: "hi" };
  s: Stack<float> = Stack { items: [], name: "floats" };
  s.items += [1.5, 2.5];
  assert(twice(b) == 42, "an inferred Box<integer> is the one the parameter
names");
  assert(t.v == "hi" && len(s.items) == 2, "Box<text> and Stack<float>");
}
```

27.0.9. Two type variables and a callback

A generic may declare several variables, and one no argument binds may be bound by a callback's RESULT: `map\_grid` below turns a `Grid\<T\>` into a `Grid\<U\>`, and the short lambda's body decides what `U` is — `|t| { t.height }` makes it an `integer`. A generic beside a concrete function of the same

name is one overload set: the concrete `show` takes a `Grid\<Tile\>`, the bounded generic every other `Grid` it can print.

```
struct Tile { height: integer, name: text }
operator to_text(self: Tile) -> text { self.name }
struct Grid<T> { w: integer, cells: vector<T> }
fn map_grid<T, U>(g: Grid<T>, f: fn(T) -> U) -> Grid<U> {
  out: vector<U> = [];
  for c in g.cells { out += [f(c)]; }
  Grid { w: g.w, cells: out }
}
fn show<T: Printable>(g: Grid<T>) -> text { r = ""; for c in g.cells { r +=
"[{c.to_text()}]"; } r }
fn show(g: Grid<Tile>) -> text { "tiles:{len(g.cells)}" }
fn test_two_variables() {
  tiles = Grid { w: 2, cells: [Tile { height: 3, name: "a" }, Tile { height: 5,
name: "b" }] };
  heights = map_grid(tiles, |t| { t.height });
  assert(show(tiles) == "tiles:2" && show(heights) == "[3][5]",
"{show(heights)}");
}
```

27.0.10. Disallowed operations

The compiler rejects operations the bound does not cover, and it says which:

```
`x + y` on unbounded T   — "generic type T: operator '+' requires a concrete
type"
`x.field` on T           — "generic type T: field access requires a concrete
type"
```

The same wording covers an operator OUTSIDE a bound: `x + y` under `Numeric` and `x - y` under `Addable` are refused in exactly those words.

```
fn main() {
  test_identity();
  test_pick_second();
  test_vector_generics();
  test_bounded();
  test_combined_bounds();
  test_user_type_bounds();
  test_walkable();
  test_generic_struct();
  test_two_variables();
}
```

28. Closures

A closure is a lambda that reads variables from the function scope in which it is written. No explicit capture list is needed — the compiler detects which outer variables the lambda body uses and packages them automatically. How a variable travels into the closure depends on what it is: a scalar or a text is COPIED, and a collection or struct is SHARED. Both rules are below, and the difference between them is the thing to remember on this page.

28.0.1. Integer capture

A lambda may read any integer variable in scope. Store the lambda, then call it by name.

28.0.2. Multiple integer captures

A lambda can read more than one outer variable at once. All are captured when the lambda is written.

28.0.3. Text capture

Text values are captured by deep-copy, so they are independent of the original variable after capture.

28.0.4. Capture timing

Scalars and text are copied when the lambda is written (definition time), not when it is called. Changing such a variable afterwards does not change what the lambda sees.

28.0.5. Collections and structs are shared

Every collection — vector, hash, sorted, index, spatial — and every struct is captured by reference: the closure and the enclosing scope work on the same value, so changes travel BOTH ways.

28.0.6. Writing to a captured variable

A closure may assign to a captured scalar, and those writes are visible in the enclosing scope.

28.0.7. Cross-scope closures

A function can return a capturing lambda to the caller. The captured values travel with the lambda — no dangling references.

28.0.8. Closures with higher-order functions

A capturing closure can be called directly, and it can also be handed to map or filter — the captures travel with it.

28.0.9. Non-capturing lambdas with higher-order functions

Lambdas that use only their own parameters (no capture) also work fine.

28.0.10. Capturing a '&' parameter

A closure inside a function can use the function's '&' parameters. What it captures is what the parameter points at, by the same two rules as above.

28.0.11. What a closure cannot do

Two shapes the compiler refuses, and what replaces each.

```
struct Counter {
  hits: integer,
}
```

```
struct Stepper {
  advance: fn(integer) -> integer,
}
```

Zero every even element, from inside a closure, then append a marker.

```
fn clear_evens(v: &vector<integer>) {
  wipe = fn(i: integer) { if v[i] % 2 == 0 { v[i] = 0; } };
  for i in 0..len(v) { wipe(i); }
  v += [9];
}
```

```
fn make_adder(base_val: integer) -> fn(integer) -> integer {
  fn(n: integer) -> integer { base_val + n }
}
```

```
fn main() {
```

28.0.12. Integer capture

```
  offset = 100;
  shift = fn(x: integer) -> integer { x + offset };
  assert(shift(5) == 105, "shift(5): {shift(5)}");
  assert(shift(-3) == 97, "shift(-3): {shift(-3)}");
```

28.0.13. Multiple integer captures

```
  lo = 10;
  hi = 20;
  clamp_val = fn(x: integer) -> integer {
    if x < lo { lo } else if x > hi { hi } else { x }
  };
  assert(clamp_val(5) == 10, "clamp below: {clamp_val(5)}");
  assert(clamp_val(15) == 15, "clamp mid: {clamp_val(15)}");
  assert(clamp_val(25) == 20, "clamp above: {clamp_val(25)}");
```

28.0.14. Text capture

The closure holds its own copy of the text, so reassigning ‘greeting’ afterwards leaves the message it builds unchanged.

```
  greeting = "Hello";
  make_msg = fn(name: text) -> text { "{greeting}, {name}!" };
  greeting = "Goodbye";
  assert(make_msg("World") == "Hello, World!", "greeting capture");
  assert(make_msg("Loft") == "Hello, Loft!", "greeting capture 2");
```

28.0.15. Capture timing

'base' is 10 when the lambda is written; reassigning it to 20 afterwards does not affect the captured value.

```
base = 10;
add_base = fn(n: integer) -> integer { base + n };
base = 20;
assert(add_base(5) == 15, "sees base=10 at definition time: {add_base(5)}");
```

28.0.16. Collections and structs are shared

A collection is NOT copied the way the scalar above is: the closure works on the caller's vector. Appending to 'items' after the lambda is written changes what the lambda counts, and a write made inside the lambda is visible outside it. That sharing is what lets a closure gather results into a vector the enclosing function keeps.

```
items = [1, 2, 3];
total = fn() -> integer {
  sum = 0;
  for e in items { sum += e; }
  sum
};
assert(total() == 6, "before the append: {total()}");
items += [10];
assert(total() == 16, "the append reaches the closure: {total()}");
```

A struct behaves the same way, in the other direction: the closure writes through to the record the enclosing scope holds.

```
tally = Counter { hits: 0 };
record_hit = fn() { tally.hits += 5; };
record_hit();
assert(tally.hits == 5, "the closure's write is visible outside:
{tally.hits}");
```

28.0.17. Writing to a captured variable

Assigning to a captured scalar shares it instead of copying it, so the closure's writes are visible for the rest of the function. This is how an accumulator is written. One restriction: a scalar a closure WRITES to may be captured by only one closure — two of them is refused, and the cure is to hold the shared state in a struct and capture that, as 'tally' does above.

```
count = 0;
bump = fn() { count += 1; };
bump();
bump();
assert(count == 2, "the closure's writes reach the enclosing scope: {count}");
```

28.0.18. Cross-scope closures

make_adder returns a lambda that captured base_val from its parameter.

```
add10 = make_adder(10);
add100 = make_adder(100);
assert(add10(5) == 15, "add10(5): {add10(5)}");
assert(add100(5) == 105, "add100(5): {add100(5)}");
```

28.0.19. Closures with higher-order functions

```
factor = 3;
scale = fn(x: integer) -> integer { x * factor };
assert(scale(5) == 15, "scale(5): {scale(5)}");
```

The same closure passed to map — 'factor' travels with it.

```
scaled = map([1, 2, 3], scale);
assert(scaled[0] == 3, "scaled[0]: {scaled[0]}");
assert(scaled[2] == 9, "scaled[2]: {scaled[2]}");
```

A capturing lambda written inline works too, in filter as well as map.

```
limit = 2;
big = filter([1, 2, 3, 4], |x| { x > limit });
assert(len(big) == 2, "len(big): {len(big)}");
assert(big[0] == 3, "big[0]: {big[0]}");
```

28.0.20. Non-capturing lambdas with higher-order functions

No capture needed here — the lambda uses only its own parameter.

```
nums = [1, 2, 3, 4, 5];
doubled = map(nums, |x| { x * 2 });
assert(doubled[0] == 2, "doubled[0]: {doubled[0]}");
assert(doubled[4] == 10, "doubled[4]: {doubled[4]}");
```

```
evens = filter(nums, |x| { x % 2 == 0 });
assert(evens[0] == 2, "evens[0]: {evens[0]}");
assert(evens[1] == 4, "evens[1]: {evens[1]}");
```

28.0.21. Capturing a '&' parameter

A '&vector' or a '&' struct is SHARED, so writes from inside the closure reach the caller's value. A '&integer' or a '&text' is COPIED when the closure is created, like any scalar: the closure can read it, and a write to it is refused (see below).

```
evens = [1, 2, 3, 4];
clear_evens(evens);
```

```
  assert("{evens}" == "[1,0,3,0,9]", "writes through a captured '&vector' reach
the caller: {evens}");
```

28.0.22. What a closure cannot do

A closure cannot WRITE a captured ‘&integer’ or ‘&text’ parameter: it holds a copy, so the write would not reach the caller, and the compiler says so. Copy the parameter into a local, let the closure update the local, and write it back before returning.

A capturing closure cannot be stored in a COLLECTION: ‘vector<fn(...)>’ and the keyed collections take non-capturing lambdas only. A struct FIELD holds one without trouble, which is the shape to reach for.

```
  step = 7;
  stepper = Stepper { advance: fn(x: integer) -> integer { x + step } };
  assert(stepper.advance(10) == 17, "a struct field holds a capturing closure:
{stepper.advance(10)}");
}
```

29. Coroutines

A generator function produces values one at a time. Declare the return type as ‘iterator<T>’ and use ‘yield’ to emit each value. The function body is suspended between yields and resumed when the next value is requested. When the body finishes the generator is exhausted. Generators are useful when you want to produce values on demand, stop early with ‘break’, or chain sequences without building intermediate collections.

29.0.1. Iterating with a for loop

A ‘for’ loop drives the generator forward automatically. The body runs once per yielded value. Use ‘break’ to stop early without consuming the rest.

29.0.2. The body really is suspended

Calling a generator runs none of its body; each advance runs exactly one more slice, up to the next ‘yield’. This holds on both backends for every ‘yield’ that is a statement of its own.

29.0.3. Manual advance with next() and exhausted()

Call ‘next(gen)’ yourself when you need fine-grained control.

29.0.4. Generators with parameters

A generator function may take any parameters, including text. Parameter values are preserved across yields so the generator can use them throughout its lifetime.

29.0.5. Delegation with yield from

‘yield from sub_gen()’ forwards all values from another generator inline, as if each of its yields appeared at this point directly.

29.0.6. Early termination with break

Stopping a ‘for’ loop before the generator is exhausted is safe. The abandoned generator frame is freed automatically.

29.0.7. A generator runs once, and has no return

Two rules a reader coming from another language will look for.

```
struct Trace {
  steps: integer,
}
```

```
fn count_up(start: integer, stop: integer) -> iterator<integer> {
  for i in start..stop {
    yield i;
  }
}
```

```
fn squares(n: integer) -> iterator<integer> {
  for i in 1..=n {
    yield i * i;
```

```
  }
}
```

Records one step per value produced, so the consumer can see how much of the body actually ran.

```
fn counted(limit: integer, t: Trace) -> iterator<integer> {
  for i in 1..=limit {
    t.steps += 1;
    yield i;
  }
}
```

Yields the length of 's' twice — shows that a text parameter survives yields.

```
fn len_twice(s: text) -> iterator<integer> {
  yield len(s);
  yield len(s);
}
```

```
fn inner_vals() -> iterator<integer> {
  yield 10;
  yield 20;
}
```

```
fn outer_combined() -> iterator<integer> {
  yield 1;
  yield from inner_vals();
  yield 2;
}
```

Delegating to a generator that takes an ARGUMENT: a plain forwarding loop means the same thing as 'yield from scaled_by(4)', and either spelling works.

```
fn scaled_by(factor: integer) -> iterator<integer> {
  yield factor;
  yield factor * 10;
}
```

```
fn forwards_with_argument() -> iterator<integer> {
  for v in scaled_by(4) {
    yield v;
  }
}
```

```
fn large_range(limit: integer) -> iterator<integer> {
  for i in 1..=limit {
```

```
    yield i;
  }
}
```

Ends early with ‘break’. A generator has no ‘return’, so ‘break’ is how a body stops before its loop would.

```
fn first_positive(v: vector<integer>) -> iterator<integer> {
  for x in v {
    if x > 0 { yield x; break; }
  }
}
```

```
fn main() {
```

29.0.8. Iterating with a for loop

```
  total = 0;
  for n in count_up(1, 6) {
    total += n;
  }
  assert(total == 15, "sum 1..5: {total}");
```

Squares via for loop with index tracking.

```
  idx = 0;
  for s in squares(4) {
    idx += 1;
    if idx == 1 { assert(s == 1, "sq1: {s}"); }
    if idx == 2 { assert(s == 4, "sq2: {s}"); }
    if idx == 3 { assert(s == 9, "sq3: {s}"); }
    if idx == 4 { assert(s == 16, "sq4: {s}"); }
  }
  assert(idx == 4, "squares count: {idx}");
```

29.0.9. The body really is suspended

‘counted’ is asked for a thousand values and the loop breaks after six. Its step counter shows the body ran six times, not a thousand — the work for the values nobody asked for was never done.

⚠️ That holds on BOTH backends for every ‘yield’ that is a statement: in a ‘for’ or ‘while’ loop, under an ‘if’ or ‘match’, in a nested loop, beside a ‘continue’, whatever type it yields. A ‘yield’ inside an EXPRESSION, after an operand that could change while the generator waits (‘s = s + { yield x; “b” }’ with ‘s’ a text), runs the whole loop eagerly on –native, so its side effects happen for values the consumer never asks for. The VALUES are the same either way. An ENDLESS eager loop never hands out a value at all: on –native it runs until memory runs out. Keep each ‘yield’ a statement of its own.

```
  trace = Trace { steps: 0 };
  seen = 0;
  for n in counted(1000, trace) {
    seen += 1;
    if seen > 5 { break; }
  }
  assert(trace.steps == 6, "the body ran once per value asked for:
{trace.steps}");
```

29.0.10. Manual advance with next() and exhausted()

'exhausted' is true once an advance has run off the END of the body — not once the last value has been handed out. After the third value here there are no more values, and 'exhausted' is still false; it takes one more 'next' to discover the end, and that call answers null.

So 'while !exhausted(g)' runs one iteration too many. Drive the generator with 'for', or stop on the null that 'next' answers.

```
  gen = count_up(10, 13);
  a = next(gen);
  b = next(gen);
  c = next(gen);
  assert(a == 10 && b == 11 && c == 12, "manual next: {a},{b},{c}");
  assert(!exhausted(gen), "not yet exhausted, though no values remain");
  past_end = next(gen);
  assert(past_end == null, "next past the last value answers null");
  assert(exhausted(gen), "and now it is exhausted");
```

29.0.11. Generators with parameters

'len_twice' takes a text parameter and yields its length twice. The text value is preserved across the yield.

```
  lt_total = 0;
  for n in len_twice("hello") {
    lt_total += n;
  }
  assert(lt_total == 10, "len_twice: {lt_total}");
```

29.0.12. Delegation with yield from

```
  yf_total = 0;
  for n in outer_combined() {
    yf_total += n;
  }
  assert(yf_total == 33, "yield from: 1+10+20+2={yf_total}");
```

A forwarding loop — 'for v in sub(arg) { yield v; }' — means the same thing as 'yield from sub(arg)'; both work on both backends.

```
  fwd_total = 0;
  for n in forwards_with_argument() {
    fwd_total += n;
  }
  assert(fwd_total == 44, "forwarded with an argument: 4+40={fwd_total}");
```

29.0.13. Early termination with break

Stop after the first 5 values from a 1000-element generator.

```
  limit_total = 0;
  for n in large_range(1000) {
    if n > 5 { break; }
    limit_total += n;
  }
  assert(limit_total == 15, "early break sum 1..5: {limit_total}");
```

29.0.14. A generator runs once, and has no return

A generator is single-pass. Once it is exhausted it stays exhausted, so a second ‘for’ over the same generator value runs its body zero times — quietly, because an exhausted iterator is empty rather than an error. Call the generator function again when you need the sequence again.

```
  once = count_up(1, 4);
  first_pass = 0;
  for n in once { first_pass += n; }
  second_pass = 0;
  for n in once { second_pass += n; }
  assert(first_pass == 6, "first pass: {first_pass}");
  assert(second_pass == 0, "the same generator a second time is empty:
{second_pass}");
```

And a generator has no ‘return’: values leave it only through ‘yield’, so ‘return’, ‘return value’ and a body ending in a value are all refused at compile time. ‘break’ is how a body ends early — ‘first_positive’ yields the first positive element and stops there.

```
  fp_seq = 0;
  for n in first_positive([-1, 5, 9]) { fp_seq = fp_seq * 10 + n; }
  assert(fp_seq == 5, "first_positive stops after its one value: {fp_seq}");
}
```

30. Tuples

A tuple groups a fixed number of values of possibly different types. You do not need to give a tuple a name — use it directly where you need to pass or return multiple values together. Access individual elements with '.0', '.1', etc.

30.0.1. Creating tuples

Write the elements in parentheses, separated by commas. The compiler infers the element types from what you put in. You can add an explicit type annotation when needed.

30.0.2. Elements of different types

The elements do not have to share a type — this is the usual reason to reach for a tuple instead of a vector.

30.0.3. Accessing elements

Use '.0', '.1', '.2', ... to read individual elements. The index is part of the program text, so it is checked when you compile: naming an element the tuple does not have is an error, never a null.

30.0.4. Modifying elements

Tuple elements can be reassigned like ordinary variables.

30.0.5. Tuples as function parameters

Pass a tuple to a function by value (a copy) or by reference. A value parameter gets its own copy — changes inside the function do not affect the caller. A reference parameter ('&') gives the function a direct link to the caller's tuple so it can modify individual elements in place, text elements included.

30.0.6. Returning tuples

A function can return a tuple to give back more than one value at once.

30.0.7. Destructuring

Assign a tuple to multiple names in one step using the '(a, b) = expr' form. This is concise when a function returns a tuple and you need both values.

30.0.8. Comparing tuples

Two tuples of the same shape compare element by element, left to right.

30.0.9. Three or more elements

Tuples can have any number of elements.

```
fn min_max(lo: integer, hi: integer) -> (integer, integer) {
  if lo <= hi {
    (lo, hi)
  } else {
    (hi, lo)
  }
}
```

```
fn swap_ints(pair: &(integer, integer)) {
  tmp = pair.0;
```

```
  pair.0 = pair.1;
  pair.1 = tmp;
}
```

Writes to its own copy. The caller's tuple is a different tuple, so the write below is invisible outside this function.

```
fn doubled_first(p: (integer, integer)) -> integer {
  p.0 = p.0 * 2;
  p.0 + p.1
}
```

A tuple is the natural return type when the two values have different types.

```
fn label_and_score(n: integer) -> (text, integer) {
  if n >= 50 { ("pass", n) } else { ("fail", n) }
}
```

```
fn main() {
```

30.0.10. Creating tuples

```
  t = (10, 20);
  assert(t.0 == 10, "t.0: {t.0}");
  assert(t.1 == 20, "t.1: {t.1}");
```

Type annotation

```
  w: (integer, integer) = (3, 4);
  assert(w.0 + w.1 == 7, "annotated sum: {w.0+w.1}");
```

30.0.11. Elements of different types

Each element keeps its own type, and '.0' / '.1' read them as that type.

```
  entry = (1, "one");
  assert(entry.0 == 1, "entry.0: {entry.0}");
  assert(entry.1 == "one", "entry.1: {entry.1}");
```

```
  mixed: (text, float, boolean) = ("pi", 3.5, true);
  assert(mixed.0 == "pi", "mixed.0: {mixed.0}");
  assert(mixed.1 == 3.5, "mixed.1: {mixed.1}");
  assert(mixed.2, "mixed.2: {mixed.2}");
```

A tuple element may itself be a tuple.

```
  nested = (1, (2, 3));
  assert(nested.1.0 == 2 && nested.1.1 == 3, "nested: {nested.1.0},
{nested.1.1}");
```

30.0.12. Modifying elements

```
  v = (1, 2);
  v.0 = 10;
  assert(v.0 + v.1 == 12, "after assign: {v.0}+{v.1}");
```

30.0.13. Tuples as function parameters

'doubled_first' writes to its parameter and returns what it computed. The caller's tuple still reads 10 — the parameter was a copy of it.

```
  p = (10, 20);
  inner = doubled_first(p);
  assert(inner == 40, "the function saw its own doubled copy: {inner}");
  assert(p.0 == 10, "and the caller's tuple is untouched: {p.0}");
```

30.0.14. Tuples as reference parameters (swap in place)

'swap_ints' exchanges the caller's two elements through its '&' parameter.

```
  pair = (3, 7);
  swap_ints(pair);
  assert(pair.0 == 7 && pair.1 == 3, "after swap: ({pair.0},{pair.1})");
```

30.0.15. Returning a tuple from a function

```
  bounds = min_max(5, 2);
  assert(bounds.0 == 2, "min: {bounds.0}");
  assert(bounds.1 == 5, "max: {bounds.1}");
```

Already in order

```
  bounds2 = min_max(1, 9);
  assert(bounds2.0 == 1, "min2: {bounds2.0}");
  assert(bounds2.1 == 9, "max2: {bounds2.1}");
```

A returned tuple with elements of different types works the same way.

```
  graded = label_and_score(72);
  assert(graded.0 == "pass" && graded.1 == 72, "graded: {graded.0},{graded.1}");
```

30.0.16. Destructuring

```
  (lo, hi) = min_max(8, 3);
  assert(lo == 3, "destructured lo: {lo}");
  assert(hi == 8, "destructured hi: {hi}");
```

Destructuring works for mixed types too, and for more than two names.

```
  (name, score) = label_and_score(20);
  assert(name == "fail" && score == 20, "destructured mixed: {name},{score}");
```

30.0.17. Comparing tuples

The first element decides; later elements are consulted only while the earlier ones are equal. Text elements compare by value.

```
  assert((1, 9) < (2, 0), "the first element decides");
  assert((1, 9) < (1, 10), "it ties, so the second decides");
  assert(!((1, 9) < (1, 9)), "identical is not strictly less");
  assert((1, 9) <= (1, 9), "but it is less-or-equal");
  assert((1, "abc") == (1, "abc"), "text compares by value, not identity");
  assert((1, "abc") != (1, "abd"), "and differing text is not equal");
```

30.0.18. Three or more elements

```
  u = (1, 2, 3);
  assert(u.0 + u.1 + u.2 == 6, "triple sum: {u.0}+{u.1}+{u.2}");
```

```
  triple = (10, 20, 30);
  triple.1 = 99;
  assert(triple.0 == 10 && triple.1 == 99 && triple.2 == 30,
    "triple modified: ({triple.0},{triple.1},{triple.2})");
```

```
  wide = (1, 2, 3, 4, 5, 6, 7, 8);
  assert(wide.7 == 8, "eight elements, last one reachable: {wide.7}");
}
```

31. Match

The `match` expression lets you compare a value against a series of patterns and run different code for each case. It is similar to `switch` in other languages but more powerful: patterns can destructure structs, bind fields to local variables, and include `if` guards. Match is an expression — it returns a value, so you can assign the result or use it directly inside a larger expression.

31.0.1. Simple enum matching

The most common use: branch on an enum variant. Every variant must be covered or a `\_` wildcard must appear — for an ENUM the compiler checks exhaustiveness, because it knows the whole set of variants.

```
enum Direction { North, South, East, West }
```

A struct-enum: its variants carry named fields, which a match arm can destructure.

```
enum Shape {
  Circle { radius: integer },
  Rect { w: integer, h: integer },
}
```

A variant whose field is itself an enum.

```
enum Paint { Solid { rgb: integer }, Clear }
enum Tile { Painted { paint: Paint, n: integer }, Bare }
```

```
fn direction_name(d: Direction) -> text {
  match d {
    North => "north",
    South => "south",
    East  => "east",
    West  => "west",
  }
}
```

A guard reading the field its own pattern bound.

```
fn size_of(s: Shape) -> text {
  match s {
    Circle { radius } if radius > 10 => "big circle",
    Circle { radius } => "small circle",
    Rect { w, h } if w == h => "square",
    Rect { w, h } => "rect",
  }
}
```

A slice match whose arms together cover every length needs no `\_`: `\[\]` covers the empty vector and `\[x, ..rest\]` every other length — the list recursion.

```
fn total(v: vector<integer>) -> integer {
  match v {
    [] => 0,
    [x, ..rest] => x + total(rest),
  }
}
```

```
fn main() {
  assert(direction_name(North) == "north", "simple enum match");
  assert(direction_name(West) == "west", "west");
```

31.0.2. Match as expression

Because match returns a value you can use it in assignments and arguments.

```
  score = match Direction.East {
    North | South => 10,
    East | West   => 20,
  };
  assert(score == 20, "match expression: {score}");
```

31.0.3. Struct-enum destructuring

When an enum has variants with fields (a struct-enum) you can bind the fields to variables in the match arm.

```
Circle { radius } — binds the 'radius' field to a local variable
Rect { w, h }     — binds both 'w' and 'h'
```

In this short form the variable names must match the field names exactly; write `Circle { radius: r }` to bind the field under another name.

```
  shape = Rect { w: 3, h: 4 };
  shape_area = match shape {
    Circle { radius } => radius * radius * 3,
    Rect { w, h }     => w * h,
  };
  assert(shape_area == 12, "struct-enum destructure: {shape_area}");
```

31.0.4. Patterns inside a field

A field can take a pattern instead of a name. A literal matches only that value, and when the field is itself an enum, a variant matches only that variant and binds its own fields in turn. A field the arm does not name is simply not tested. An arm with a pattern inside a field covers only part of its variant, so the match still needs `\_` (or an arm that binds the fields plainly) to cover the rest.

```
  tile = Painted { paint: Solid { rgb: 255 }, n: 2 };
  what = match tile {
    Painted { paint: Clear, n } => "clear x{n}",
```

```
    Painted { paint: Solid { rgb: 0 }, n } => "black x{n}",
    Painted { paint: Solid { rgb }, n } => "colour {rgb} x{n}",
    _ => "bare",
  };
  assert(what == "colour 255 x2", "a pattern inside a field: {what}");
```

31.0.5. Guards

Add `if condition` after a pattern to restrict when the arm matches. A guard can reject, so a guarded arm never counts towards covering an enum: a match whose arms name every variant but guard them all is still refused.

```
  v = 42;
  label = match v {
    0      => "zero",
    _ if v < 0 => "negative",
    _ if v > 100 => "large",
    _      => "normal",
  };
  assert(label == "normal", "guard: {label}");
```

A guard can also read what the PATTERN bound, which is where guards earn their keep — the field is in scope inside the condition.

```
  assert(size_of(Circle { radius: 20 }) == "big circle", "guard on a binding");
  assert(size_of(Circle { radius: 2 }) == "small circle", "guard on a binding,
false");
  assert(size_of(Rect { w: 3, h: 3 }) == "square", "guard comparing two
bindings");
  assert(size_of(Rect { w: 3, h: 4 }) == "rect", "guard comparing two bindings,
false");
```

31.0.6. Wildcard _

The underscore _ matches anything, and it must be the LAST arm: it matches everything, so an arm written after it could never be selected, and the compiler says so.

Whether you NEED a _ depends on the subject. For an enum the compiler knows every variant, so it requires them all to be covered or a _ to be present, and refuses the match otherwise — naming the variants you left out. For an integer, character or text subject there is no finite set to check, so no _ is required and none is missed: a value that matches no arm makes the match answer null. That is the case to watch, because nothing warns you — see “When nothing matches” below.

```
  x = 7;
  kind = match x {
    1 => "one",
    2 => "two",
    _ => "other",
```

```
};
assert(kind == "other", "wildcard: {kind}");
```

31.0.7. Integer matching

Match works on integers — each arm is compared with ==.

```
name = match 3 {
  1 => "one",
  2 => "two",
  3 => "three",
  _ => "many",
};
assert(name == "three", "integer match: {name}");
```

31.0.8. Range patterns

A numeric arm can match a whole inclusive range with `low..=high` — handy for grading, bucketing, or any "is it in this band" test.

```
grade = match 95 {
  90..=100 => "A",
  80..=89  => "B",
  _        => "lower",
};
assert(grade == "A", "range pattern: {grade}");
```

31.0.9. Null patterns

When the subject is nullable, a `null` arm handles the absent case explicitly. With null-flow this is the idiomatic way to branch on a `T?` inside a match.

```
maybe: integer? = null;
presence = match maybe {
  null => "absent",
  0    => "zero",
  _    => "present",
};
assert(presence == "absent", "null pattern: {presence}");
```

31.0.10. Text matching

Match also works on text values.

```
greeting = "hi";
reply = match greeting {
  "hello" => "formal",
  "hi"    => "casual",
  _       => "unknown",
};
assert(reply == "casual", "text match: {reply}");
```

31.0.11. Multiple patterns with |

Combine patterns with | to share the same arm body.

```
d = Direction.South;
axis = match d {
  North | South => "vertical",
  East | West   => "horizontal",
};
assert(axis == "vertical", "multi-pattern: {axis}");
```

| joins variant NAMES. To bind fields in several variants with one arm, list the whole patterns separated by , instead: each pattern binds from its own variant, and the arm body runs for whichever one matched. A name every pattern binds must have the same type in each; a name only some patterns bind is nullable in the body, `null` when a pattern without it matched.

```
long_side = match Rect { w: 3, h: 8 } {
  Circle { radius }, Rect { h: radius } => radius,
  _ => 0,
};
assert(long_side == 8, "a field bound from either variant: {long_side}");
```

31.0.12. Tuple patterns

A tuple subject matches element by element. Write _ for an element you do not care about.

```
point = (2, "b");
where = match point {
  (0, _)   => "origin row",
  (2, "a") => "two-a",
  (2, "b") => "two-b",
  _        => "elsewhere",
};
assert(where == "two-b", "tuple pattern: {where}");
```

An element can be an enum variant with its fields bound, so one arm reads both sides at once, and a guard can compare them.

```
a: Shape = Rect { w: 2, h: 3 };
b: Shape = Circle { radius: 5 };
bigger = match (a, b) {
  (Rect { w, h }, Circle { radius }) if w * h > radius => "rect",
  (Rect { w, h }, Circle { radius }) => "circle",
  _ => "other",
};
assert(bigger == "rect", "variants inside a tuple pattern: {bigger}");
```

31.0.13. Slice arms that cover every length

One slice arm is length-constrained, so it never covers every length on its own; the ARMS do when an exact arm covers each short length and a rest arm every length from there on, each with plain element

bindings and no guard. Then no `\_` is needed and the match is not nullable. A set that leaves a length out — `\[x, ..rest\]` alone — is refused ("a slice pattern can fail").

```
  assert(total([]) == 0, "the empty arm: {total([])}");
  assert(total([1, 2, 3]) == 6, "the recursion over the rest: {total([1, 2,
3])}");
```

31.0.14. When nothing matches

This match names no `\_`, and 7 is none of its arms. On an enum that would not compile; on a scalar there is no finite set to check, so the match answers null and the null travels on. Give the result a fallback with `??`, or add a `\_` arm. The compiler advises at the match (`match-scalar-hole`); `LOFT\_NO\_SCALAR\_HOLE` keeps a deliberate hole quiet.

```
  unmatched = match 7 {
    1 => "one",
    2 => "two",
  };
  assert(unmatched == null, "a scalar match that selects no arm answers null");
  assert((unmatched ?? "none") == "none", "so give it a fallback");
```

31.0.15. Match inside an arm

Match is an expression, so it can appear inside other expressions, including another match's arm.

```
  n = 15;
  result = match n % 2 == 0 {
    true  => "even",
    false => match n % 3 == 0 {
      true  => "odd multiple of 3",
      false => "odd",
    },
  };
  assert(result == "odd multiple of 3", "nested: {result}");
}
```

32. Formatting

Loft strings can embed expressions inside `{...}` braces. A colon after the expression introduces a format specifier that controls width, alignment, precision, number base, and output style.

32.0.1. Basic interpolation

Any expression inside `{...}` is evaluated and converted to text.

```
struct Point { x: integer, y: integer }
```

A type that receives the PARTS of a format string. See the last section.

```
struct Query {
  const parts: vector<text>,
  const values: vector<text>,
}
```

`lit` gets the bytes you wrote in the source file.

```
fn lit(self: Query, s: text) { self.parts += [s]; }
```

`hole\_text` gets an interpolated value. It is never added to `parts`.

```
fn hole_text(self: Query, v: text?) { self.values += [v ?? ""]; }
```

```
fn hole_int(self: Query, v: integer) { self.values += ["{v}"]; }
```

Show the two lists separately, so you can see what went where.

```
fn shape(self: Query) -> text {
  out = "";
  for q_part in self.parts { out += "<{q_part}>" }
  for q_val in self.values { out += "[{q_val}]" }
  return out;
}
```

```
fn main() {
  name = "world";
  assert("hello {name}" == "hello world", "basic interpolation");
  assert("1 + 2 = {1 + 2}" == "1 + 2 = 3", "expression in braces");
```

The braces hold any expression, not just a name — a field, an index, a call, arithmetic. Each is evaluated left to right as the text is built.

```
  assert("{len(name)}" == "5", "a call inside a hole");
```

32.0.2. Writing a real brace

A single `{` opens a hole, so double it to mean the character itself. `{{` is `{` and `}}` is `}`. This is why the struct examples further down compare against `"{{x:1,y:2}}"` — that is the six-character text `{x:1,y:2}`.

```
assert("{{literal}}" == "{{literal}}", "doubled braces are one brace each");
assert(len("{{}}") == 2, "`{{}}` is two characters, not four");
```

32.0.3. Width and alignment

A number after `:` sets the minimum width. The value is padded with spaces to fill the width.

```
{val:6}   — right-aligned (default for numbers)
{val:>6}  — right-aligned (explicit)
{val:<6}  — left-aligned
{val:^6}  — centered
```

Numbers are right-aligned by default; everything else — text, booleans, characters, vectors, structs — is left-aligned.

```
assert("{42:6}" == "    42", "default number align is right");
assert("{42:>6}" == "    42", "explicit right-align");
assert("{42:<6}" == "42    ", "left-align number");
assert("{42:^6}" == "  42  ", "center-align number");
s = "hi";
assert("{s:6}" == "hi    ", "default text align is left");
assert("{s:>6}" == "    hi", "right-align text");
assert("{s:^6}" == "  hi  ", "center-align text");
```

A width shapes the field whatever the value is, so it pads a boolean or a vector exactly as it pads a number.

```
assert("{true:6}" == "true  ", "a boolean is padded too");
assert("{[1,2]:8}" == "[1,2]   ", "a vector is padded too");
```

32.0.4. A fill character

Put any single non-digit character before the alignment to pad with that character instead of a space. It comes first, before the other flags.

```
assert("{s:*>6}" == "****hi", "fill with `*`");
assert("{42:*^11}" == "****42*****", "fill and centre");
```

A digit cannot be a fill character, because a digit is how a width starts: `{n:0\>5}` is the comparison `0 \> 5`, not “pad with 0, right-align, width 5”. Loft refuses that spec rather than guessing; to zero-pad, write `05`.

32.0.5. The flags may be written in any order

```
assert("{0.5:+<8.3}" == "+0.500  ", "sign before alignment");
assert("{0.5:<+8.3}" == "+0.500  ", "alignment before sign");
```

32.0.6. Zero padding

Prefix the width with `0` to pad with zeros instead of spaces.

```
assert("{7:03}" == "007", "zero-padded 3 digits");
assert("{42:05}" == "00042", "zero-padded 5 digits");
assert("{-1:04}" == "-001", "zero-padded negative: sign before zeros");
```

A zero pad fills the number, so the sign — and a `0x` marker — stays in front of the zeros it adds. This holds for floats and singles too.

```
assert("{3.125:08.2}" == "00003.12", "zero-padded float");
assert("{-3.5:08.2}" == "-0003.50", "zero-padded negative float");
assert("{1.5f:08.2}" == "00001.50", "zero-padded single");
```

32.0.7. Signed format

`+` forces a sign on positive numbers.

```
assert("{42:+}" == "+42", "explicit positive sign");
assert("{-42:+}" == "-42", "negative sign always shown");
assert("{0:+}" == "+0", "sign on zero");
```

`+` is about numbers, not about integers — a float and a single take it too.

```
assert("{3.5:+}" == "+3.5", "sign on a float");
assert("{0.5:+.3}" == "+0.500", "sign beside a precision");
```

32.0.8. Hexadecimal, binary, octal

`:x` for lowercase hex, `:X` for uppercase, `:\#` adds the `0x` / `0b` / `0o` marker. `:b` for binary, `:o` for octal, `:d` for the decimal default.

```
assert("{255:x}" == "ff", "hex lowercase");
assert("{255:X}" == "FF", "hex uppercase");
assert("{255:#x}" == "0xff", "hex with prefix");
assert("{10:b}" == "1010", "binary");
assert("{8:o}" == "10", "octal");
```

A width goes before the base letter, and the marker keeps its place in front of a zero pad — the result is something you can paste back into a program.

```
assert("{255:6x}" == "    ff", "hex in a 6-wide field");
assert("{255:#06x}" == "0x00ff", "the `0x` marker precedes the zeros");
assert("{10:#06b}" == "0b1010", "the `0b` marker precedes the zeros");
```

A base is a property of an integer, so asking for one anywhere else is a compile error rather than a spec that quietly does nothing. `{2.5:x}` says "`x` has no effect on float".

32.0.9. Float precision

`.N` after the colon fixes the number of decimal places. Only `float` and `single` have decimal places, so `.N` on any other type is a compile error: `{7:.2}` says "a precision has no effect on integer". That matters because the mistake is easy to make — a `price` you thought was a float is an integer, and `{price:.2}` would otherwise render no decimals at all.

```
assert("{3.125:.1}" == "3.1", "1 decimal place");
assert("{3.125:.2}" == "3.12", "2 decimal places");
assert("{3.125:.0}" == "3", "0 decimal places");
assert("{0.0:.3}" == "0.000", "trailing zeros");
```

32.0.10. Width + precision

Combine width and precision: `{val:W.P}` where W is total width and P is decimal places.

```
assert("{3.125:8.2}" == "    3.12", "width 8 precision 2");
assert("{-3.125:8.2}" == "   -3.12", "negative width+precision");
```

32.0.11. JSON format

`:j` (also spelled `:json` or `:J`) renders a struct, a vector or an enum as JSON instead of loft's own compact form: quoted keys, and nothing else changed. See the JSON documentation page for full details.

```
jp = Point { x: 1, y: 2 };
assert("{jp}" == "{{x:1,y:2}}", "the default form is loft's own");
assert("{jp:j}" == "{{\"x\":1,\"y\":2}}", "`:j` quotes the keys");
assert("{jp:json}" == "{{\"x\":1,\"y\":2}}", "`:json` is the same spec");
assert("{[jp]:j}" == "[{{\"x\":1,\"y\":2}}]", "a vector of structs");
```

`:j` is a choice between two renderings of a composite value, so it is not available on a scalar: `{7:j}` and `{"a":j}` are compile errors. A scalar has one rendering, and inside a struct it is already the JSON one.

32.0.12. Vector format

Vectors are formatted as `\[a,b,c\]` by default. A format specifier inside a `for` loop applies to each element.

```
v = [1, 2, 3];
assert("{v}" == "[1,2,3]", "default vector format");
assert("{v:>10}" == "   [1,2,3]", "a vector takes a field like anything else");
assert("{v:#}" == "[ 1, 2, 3 ]", "`:#` spaces a vector out too");
assert("{for fmt_n in 1..4 {fmt_n * 10}:04}" == "[0010,0020,0030]", "formatted
vector elements");
```

32.0.13. Large integers and single-precision floats

`integer` is a 64-bit type, so a value near its limit formats like any other number rather than losing digits.

```
  n = 1000000;
  assert("{n}" == "1000000", "integer default");
  assert("{n:>10}" == "   1000000", "integer right-aligned");
  big = 9223372036854775807;
  assert("{big}" == "9223372036854775807", "the largest integer keeps every
digit");
  assert("{big:x}" == "7fffffffffffffff", "and renders in any base");
```

A `single` is a float of half the width; every specifier above applies to it unchanged.

```
  f = 1.5f;
  assert("{f}" == "1.5", "single default");
  assert("{f:8.2}" == "    1.50", "single width and precision");
  assert("{f:+}" == "+1.5", "single with a forced sign");
```

32.0.14. Pretty-printing a struct

The `:\#` specifier expands a struct across a spaced, readable layout instead of the compact default form.

```
  p = Point { x: 1, y: 2 };
  assert("{p}" == "{{x:1,y:2}}", "default struct format is compact");
  assert("{p:#}" == "{{ x: 1, y: 2 }}", "`:#` pretty-prints with spaces");
```

32.0.15. Null, and why formatting never fails

Every type has a null, and a null renders as the word `null`. It is a value, not an error, so it pads like any other rendering.

```
  absent: integer? = null;
  assert("{absent}" == "null", "a null integer renders as `null`");
  assert("{absent:>8}" == "    null", "and takes the field it was given");
```

A zero pad is for numbers, and `null` is not one, so it pads with spaces.

```
  assert("{absent:08}" == "    null", "`null` is never zero-padded");
```

An operation inside a hole that cannot produce a value yields null rather than stopping the program, and the null prints as `null`. Building a message — a log line, an error report — can therefore never itself fail.

```
  top = 5;
  bottom = 0;
  assert("{top / bottom}" == "null", "a division by zero prints null");
```

32.0.16. Character format

```
  c = 'A';
  assert("{c}" == "A", "character default");
  assert("{c:>5}" == "    A", "a character takes a field like anything else");
```

32.0.17. Boolean format

```
   assert("{true}" == "true", "boolean true");
   assert("{false}" == "false", "boolean false");
```

32.0.18. Building a value instead of text

Everything above joins the pieces into one `text`. When the type you assign to says so, the format string **builds that type instead**, and the type is told which bytes you wrote and which came from a value.

A type opts in by defining `lit` (for your bytes) and one `hole\_…` method per value kind it accepts: `hole\_text`, `hole\_int`, `hole\_float`, `hole\_single`, `hole\_boolean`, `hole\_character`. There is no new syntax — the type you assign to decides — and plain `text` is unchanged.

```
   who = "ada";
   q: Query = "SELECT * FROM t WHERE name = {who}";
```

The text you wrote is in `parts`; the value is in `values`, never in the text. That separation is the point: a value cannot turn into syntax, because the only way into the text is `lit`, and `lit` only ever receives bytes from your source file.

```
   assert(q.shape() == "<SELECT * FROM t WHERE name = >[ada]",
          "the literal and the value stay apart");
```

The value can be anything at all, including text that would otherwise change the meaning of the statement. It is still just a value.

```
   danger = "'; DROP TABLE t; --";
   q2: Query = "WHERE name = {danger}";
   assert(q2.shape() == "<WHERE name = >['; DROP TABLE t; --]",
          "a value that looks like syntax is still a value");
```

Each kind goes to its own method, so the type sees the real type.

```
   q3: Query = "id={7} name={who}";
   assert(q3.shape() == "<id=>< name=>[7][ada]", "an integer hole calls
hole_int");
```

The type is taken from an annotated binding, a struct-literal field or the function's declared return type. A call argument does NOT target — `db.db\_rows("… {name}")` is an ordinary text and reports `expected Query, got text` — so route through a local:

```
   q: SqlText = "SELECT id FROM users WHERE name = {name}";
   db.db_rows(q)
```

A format spec on a target's hole is a compile error, because the value is handed to the type rather than rendered: there is nothing for a width to pad.

```
}
```

33. Reference parameter forwarding

When you pass a value to a function, what can that function change about it? The answer depends on the value's type and on one annotation, &. This chapter is about the whole of that rule, because the half of it people usually assume is the wrong half.

```
struct Item { val: integer }
struct Box { items: vector<Item> }
```

33.0.1. A scalar argument is copied

The callee gets its own number. Nothing it does can reach the caller.

```
fn bump(n: integer) { n = n + 1; }
```

33.0.2. A struct or collection argument is SHARED

No annotation needed. Writing a field, writing an element, appending, removing and clearing are all done to the caller's value.

```
fn set_first(b: Box) { b.items[0].val = 7; }
fn add_one(b: Box) { b.items += [Item { val: 1 }]; }
fn empty_it(b: Box) { b.items.clear(); }
```

33.0.3. What & adds: REPLACEMENT

Assigning the whole binding — `b = ...` — is the one operation that does NOT reach the caller through a plain parameter. It rebinds the callee's own name and the caller keeps what it had. Declaring the parameter & turns that assignment into a write-back, and that is all & does. The compiler reports the plain spelling below: the value `b` came in with is never read (`warning\[never-read\]`), and neither is the one it is given (`warning\[dead-assignment\]`), each with `b: &Box` as the cure.

```
fn replace_plain(b: Box) { b = Box { items: [Item { val: 99 }] }; }
fn replace_ref(b: &Box) { b = Box { items: [Item { val: 99 }] }; }
```

33.0.4. Forwarding

A & parameter can be handed to another function that also takes &. The reference is forwarded rather than dereferenced, so the innermost function's replacement still reaches the outermost caller, at any depth.

```
fn forward_replace(b: &Box) { replace_ref(b); }
```

These two only append and write elements, so neither needs & — a plain parameter already does both to the caller's value. The compiler says so if you add one: `advice\[slow-reference-parameter\]`.

```
fn ensure(b: Box) {
  if len(b.items) == 0 { b.items += [Item { val: 0 }]; }
}
```

```
fn set_val(b: Box, val: integer) {
  ensure(b);
  for sv_item in b.items {
    if sv_item.val == 0 { sv_item.val = val; return; }
  }
}
```

```
fn main() {
```

33.0.5. A scalar is copied

```
  n = 1;
  bump(n);
  assert(n == 1, "a scalar argument is copied");
```

33.0.6. A struct or collection is shared — no & required

```
  shared = Box { items: [Item { val: 0 }] };
  set_first(shared);
  assert(shared.items[0].val == 7, "an element write reaches the caller");
  add_one(shared);
  assert(len(shared.items) == 2, "an append reaches the caller");
  empty_it(shared);
  assert(len(shared.items) == 0, "a clear reaches the caller");
```

This is the half people assume is the other way round. & is NOT the permission you need in order to append to a vector parameter — a plain parameter already grows the caller's vector.

33.0.7. Replacement is the one thing that needs &

```
  keep = Box { items: [Item { val: 1 }] };
  replace_plain(keep);
  assert(keep.items[0].val == 1, "a plain parameter cannot replace the whole
value");
  swap = Box { items: [Item { val: 1 }] };
  replace_ref(swap);
  assert(swap.items[0].val == 99, "a `&` parameter can");
```

So read & on a signature as "this function may hand you back a different value", not as "this function may modify what you give it".

33.0.8. Forwarding carries it through

The forwarder needs the & as much as the function it calls. Its own body never assigns to the parameter, so the & can look unnecessary — but it is what carries the inner replacement out. Written `fn forward\_replace(b: Box)` the assertion below fails — and the compiler says why: the replacement lands in the forwarder's own b, which nothing reads after the call (`warning\[dead-assignment\]`).

```
    deep = Box { items: [Item { val: 1 }] };
    forward_replace(deep);
    assert(deep.items[0].val == 99, "the replacement survives one forward");
```

A chain of plain parameters carries the shared value just as far, which is why `set\_val` below needs no annotation at any level.

```
    b = Box { items: [] };
    set_val(b, 42);
    assert(b.items[0].val == 42, "a plain parameter forwards its sharing too");
```

Calling the inner function directly does the same thing, and calling it twice is harmless: it only fills an empty box.

```
    b2 = Box { items: [] };
    ensure(b2);
    assert(len(b2.items) == 1, "ensure added one item");
    ensure(b2);
    assert(len(b2.items) == 1, "ensure is idempotent");
```

33.0.9. A worked example

`set\_val` fills the first zero slot, leaving anything already set alone.

```
    b3 = Box { items: [] };
    b3.items += [Item { val: 10 }];
    b3.items += [Item { val: 0 }];
    set_val(b3, 99);
    assert(b3.items[0].val == 10, "an existing value is left alone");
    assert(b3.items[1].val == 99, "the empty slot is filled");
}
```

34. Feature catalogue

Every language surface and every part of the toolchain the catalogue tracks, in one list. Each entry has a page of its own in `doc/features/` in the repository, where `\@F2` is `F2.md`.

The two halves read differently. An `\@F` page says what the feature is and how it aids you, and 76 of the 91 carry a runnable example; the rest name what demonstrates them instead, because a loft program cannot run the compiler that runs it. An `\@I` page says what the part does and where it lives in the source — it describes how loft is built, not something you write.

The catalogue is generated from the `loft-lang/features` issue tracker, which is the single source of truth: every entry is an issue, and `make features-check` regenerates this page and fails on any difference. A feature you can name and cannot find below is missing from the TRACKER — that is a gap in the catalogue rather than a gap in loft, and the chapters of this reference are the wider list.

34.0.1. Language features

- **@F1** — Nullable value semantics — in-band sentinels + null-fallback arithmetic
- **@F2** — `??` null-coalescing operator (incl. `?? return`)
- **@F3** — Primitive scalar types (integer, float, single, boolean, character)
- **@F4** — Ranged/width integer types (u8/i8/u16/i16/i32/u32)
- **@F5** — Type conversions — implicit, format-only, and explicit `as`
- **@F6** — `vector\<T\>` — append, index, slice, comprehensions, aggregates, map/filter/reduce
- **@F7** — `hash\<T\[keys\]\>` keyed collection
- **@F8** — `sorted\<T\[keys\]\>` collection
- **@F9** — `index\<T\[keys\]\>` B-tree index (asc/desc, multi-key)
- **@F10** — `iterator\<T\>` values
- **@F11** — Tuples — anonymous fixed-arity `(T1, T2, …)`
- **@F12** — Struct records — fields, `= default`, `computed`, `limit`/`not null`/`assert`
- **@F13** — Simple enums (ordered value types)
- **@F14** — Polymorphic struct-enums (per-variant fields)
- **@F15** — Enum-scoped variant names + context inference
- **@F16** — Functions & declarations (`pub`, parameters, return)
- **@F17** — Named arguments + default parameter values
- **@F18** — `const` parameters
- **@F19** — Method dispatch via `self` / `both`
- **@F20** — Variant-based dynamic dispatch
- **@F21** — References `&T` — parameters + write-back bindings
- **@F22** — Closures & lambdas (value capture, cross-scope)
- **@F23** — Function references as first-class values
- **@F24** — Higher-order functions (map / filter / reduce)
- **@F25** — Generics — type variables `\<T\>`, `\<K, V\>`, inferred; generic structs
- **@F26** — Interfaces & bounded generics (`\<T: A + B\>`, operator interfaces)
- **@F27** — `if` / `else` as an expression
- **@F28** — `for`-in loops — ranges, loop attributes, filtered, `rev()`
- **@F29** — Pattern matching — enum/scalar/tuple, guards, or-patterns, exhaustiveness
- **@F30** — `is` variant check (+ field capture)

- **@F31** — `break` / `continue` + labelled forms
- **@F32** — Custom iterators via `operator next(self) -\> T?`
- **@F33** — `par(...)` parallel for-loop
- **@F34** — Coroutines / generators — `yield`, `yield from`
- **@F35** — String literals — `{expr}` interpolation + backtick multiline
- **@F36** — String formatting / format specifiers (+ for-expressions)
- **@F37** — Operator set — arithmetic/comparison/logical/bitwise/unary, `\*\*`
- **@F38** — Arithmetic safety — integer overflow/÷0 ⟶ null; null propagates through arithmetic
- **@F39** — Math & trigonometry library
- **@F40** — File & directory I/O (+ durable-store binding)
- **@F41** — Environment & arguments (env vars, arguments(), program dirs, path resolution)
- **@F42** — JSON — json_parse, JsonValue, Type.parse, to_json
- **@F43** — Random numbers (rand_seed / rand / rand_indices)
- **@F44** — Logging & diagnostics API (log_*, print, assert, panic)
- **@F45** — `sizeof()`
- **@F46** — Type aliases (`type X = …`)
- **@F47** — Library imports / module system (`use` forms, `pub`)
- **@F48** — The `loft` CLI — run a program, `--interpret` / `--native`, `--timeout`, `--help`
- **@F49** — REPL — interactive sessions
- **@F50** — Introspection — `loft introspect` (bytecode / native Rust / slot tables)
- **@F51** — Debugger — breakpoints, frame capture, scripted RPC
- **@F52** — Source formatter — canonical `.loft` output
- **@F53** — Native-binary backend (`--native` ⟶ rustc)
- **@F54** — Browser / WASM target (`--html` / `--native-wasm`)
- **@F55** — Package management (`loft install`, `loft.toml`, lockfile)
- **@F56** — Live code reload — patch a running program
- **@F88** — Stack traces (stack_trace)
- **@F89** — Test runner (fn test_*, loft –tests / loft test)
- **@F92** — Direct C binding — `\#c "symbol" "signature"`, no rustc and no glue crate
- **@F93** — Paged & remote store loading — read part of a dataset over HTTP range
- **@F94** — Type-directed interpolation — a type receives the literal/hole boundary
- **@F95** — Value structs (`value struct`) — copy semantics, stored inline
- **@F96** — `x?` default-fallback operator — discharge a nullable to its type default
- **@F97** — `len` vs `size` — logical count vs bytes occupied
- **@F98** — `spatial\<T\[x,y\]\>` — position-keyed collection for proximity queries
- **@F99** — Sequence match patterns — alternation, optionals, repetition, capture
- **@F100** — Dead-code lint — a local never read, and a write whose value is lost
- **@F101** — The build phase — `loft build`, declared targets, zero-config defaults
- **@F102** — Android target — `--native-android`, APK packaging, GL surface and touch input
- **@F103** — `deliver` / `expose` — hand a loft value to JavaScript with no copy
- **@F104** — `store\_reclaim()` — give a store's unused space back
- **@F105** — Custom `to\_text` format hook — a type owns how it prints
- **@F106** — Copy and move semantics — when two names share data, and when they do not

- **@F107** — Type reflection — the declared shape of a type, as data
- **@F108** — Lazy store binding — a collection fetches on a miss
- **@F109** — `\#superseded` — steer callers to a newer form without breaking the old one
- **@F110** — Diagnostic suggestions — what to write instead, checked by running it
- **@F111** — `for f in s\#fields` — a compile-time loop over a struct's scalar fields
- **@F113** — Associated types — an interface names a companion type
- **@F114** — A library type reads an element through a named method — `\[…\]` is the built-in collections'
- **@F115** — `OpDrop` — a type runs code when its scope lets it die
- **@F118** — Time (now / ticks)
- **@F119** — Store locks (#lock)
- **@F120** — Lexer in loft (lib/lexer.loft) — part of the loft-in-loft front end, not a user library yet
- **@F121** — Parser in loft (lib/parser.loft) — part of the loft-in-loft front end, not a user library yet
- **@F122** — Multiple dispatch — one name, a definition per combination of parameter types
- **@F123** — Foreign stores — a mapped file or a library's buffer read in place, a slice of it a view
- **@F124** — Operators on a program's own type — `operator compare`, `plus`, `minus`, `times`, `negate`, `divided\_by`, `remainder`, `to\_\<type\>`
- **@F125** — Text slicing — `s\[a..b\]`, `s\[a..\]`, `s\[..b\]`, `s\[i\]` (byte offsets; `char\_slice` for characters)
- **@F126** — A store's failure arm — `place = value else { … }` runs when the write did not land
- **@F127** — Store upgrade — a changed struct converts its stored data, or says why it cannot

34.0.2. Tooling and infrastructure

- **@I57** — Lexer
- **@I58** — Parser (two-pass recursive descent)
- **@I59** — Type resolver
- **@I60** — Scope & dependency/lifetime tracker (deps)
- **@I61** — Stack slot allocator
- **@I62** — IR data model (Value/Type/Data)
- **@I63** — Store-resident IR (reader + handle + materializer)
- **@I64** — Bytecode compiler (IR -> bytecode)
- **@I65** — Bytecode code generator
- **@I66** — Bytecode VM / executor
- **@I67** — Opcode implementations
- **@I68** — Native Rust generator
- **@I69** — Word-addressed store
- **@I70** — Database — alloc / persistence / journal / snapshot / schema
- **@I71** — DbRef pointers & collection keys
- **@I72** — Parallel execution runtime
- **@I73** — Native function registry
- **@I74** — CDylib extension loader
- **@I75** — Diagnostics collector
- **@I76** — Logger runtime
- **@I77** — Registry / manifest / lockfile resolution

- **@I78** — Live-reload dispatch
- **@I79** — Documentation generator (maintainer tooling)
- **@I80** — Tracker indexer (maintainer tooling)
- **@I81** — Feature-catalogue sync generator (issues ⟶ in-project mirror + example tests)
- **@I82** — Sandbox policy & enforcement
- **@I83** — Dev server (–serve / –rpc)
- **@I84** — Coroutine runtime / yield codec
- **@I85** — Engine-host kernel natives
- **@I86** — Startup cache & embedded stdlib
- **@I87** — Auto-use / lazy library triggers
- **@I90** — Shared utilities & data structures
- **@I91** — Editor tooling: language server (LSP), debug adapter (DAP), resolution index
- **@I116** — Library placement — in-process, a worker, or another machine
- **@I117** — Git query natives — a repository as a typed library, not a subprocess

```
fn main() {
  println("the feature catalogue is generated from the issue tracker");
}
```

35. Running your program

The pages before this one show how to WRITE loft. This one shows how to RUN it. You need one command to start, and everything else is optional.

35.0.1. Run a file

Put your program in a file ending in '.loft' and pass it to loft:

```
$ loft hello.loft
hello, world!
```

That is the whole thing. There is no build step to run first and no project to set up. A single file is a complete program.

35.0.2. Give your program some arguments

Put them after the file name, and declare one `vector\<text\>` parameter on 'main' to read them:

```
fn main(args: vector<text>) {
  for who in args { println("hello, {who}!"); }
}
```

```
$ loft greet.loft Ada Grace
hello, Ada!
hello, Grace!
```

That one parameter is the only shape 'main' accepts. Any other — a plain 'text', two of them — is refused, because nothing would fill it.

35.0.3. Try something without making a file

Type 'loft' on its own and you get a prompt where you can type one line at a time and see the answer straight away:

```
loft
loft REPL — :help for commands, :quit to exit
loft> 2 + 3
5
```

This is called a REPL, which is short for "read, evaluate, print, loop". It is the quickest way to check what a function does or try an idea.

Started by hand like that, the REPL remembers your session: the values you bound and the functions and structs you defined come back next time, and it says so as it starts.

```
loft
loft REPL — :help for commands, :quit to exit
restored 2 statement(s) from last session
loft>
```

'–fresh' asks for a clean one instead, and ':reset' clears it mid-session.

You can also feed it from a pipe, which is handy in a script. That is a different mode: it never reads or writes the saved session, so it always starts empty. The banner and the ‘loft>’ prompts go to standard error and the answers to standard output, so a script can read just the answers:

```
$ printf '2 + 3\n' | loft repl --fresh 2>/dev/null
5
```

35.0.4. Two ways to run, and why you usually ignore this

loft can run your program in two ways. It compiles your program to a fast program first when it can, and otherwise it reads and runs your program directly. The second way is called interpreting.

You do not have to choose. loft picks for you, and the ANSWER is the same either way — that is a rule the language holds itself to, not a hope. You can ask for one on purpose when you want to:

```
$ loft --interpret hello.loft
hello, world!
```

A downloaded loft always interprets. That is normal and not something to fix.

35.0.5. Stop a program that runs too long

A loop with a mistake in it can run forever. Give loft a time limit and it stops the program for you:

```
$ loft --timeout 10 hello.loft
hello, world!
```

This is worth using whenever you run something for the first time.

35.0.6. Check your work without running it

Sometimes you only want to know whether the program is correct so far:

```
$ loft check hello.loft
ok
```

This reports mistakes and runs nothing. It is fast, and it is a good habit while you are still writing. It leaves a ‘.loft’ folder beside your file to remember what it already worked out, which is why the second run is quicker.

35.0.7. When loft tells you something is wrong

A message from loft names the file, the line, and what to do. Say unused.loft binds a name it never uses:

```
$ loft --interpret unused.loft
warning[never-read]: Variable unused is never read
note: 1 diagnostic above suggests what to write instead — re-run with `--
explain`
hello, world!
```

Between those first two lines it also prints the file and line it means, the source line itself, and a caret under the exact spot.

That closing note means loft has a replacement in mind. Ask for it:

```
  $ loft --explain --interpret unused.loft
  fix  delete `unused`   needs: computing it has no effect you are relying on
[dead-code lint · @F100]
```

The ‘fix’ line is the suggested replacement and ‘needs’ is the condition under which it applies — loft cannot know whether you meant to keep the name, so it tells you what it is assuming. The bracket names the topic to read up on; the feature catalogue has a page per tag. ‘–explain’ only SHOWS you the fix. It changes nothing, so it is always safe to run.

```
fn main() {
```

A program is just a file with a `main` function in it, like this one. Everything above is how you would run this file.

```
  greeting = "hello";
  who = "world";
  message = "{greeting}, {who}!";
  assert(message == "hello, world!", "string interpolation builds the message");
  println(message);
}
```

36. Testing your code

A test is a function that checks your code still does what you meant. You do not install anything to write one, and there is no special syntax.

36.0.1. Write a test

Give a function a name starting with ‘test_’ and put an ‘assert’ in it. That is all a test is:

```
fn double(n: integer) -> integer { n * 2 }
```

```
fn test_double_doubles() {
  assert(double(4) == 8, "double(4) should be 8");
}
```

‘assert’ takes something that should be true, and a message. If it is true, nothing happens. If it is not, the message is what you will read, so write the message for the person who has to fix it — usually you, later.

36.0.2. Run your tests

Point loft at the file:

```
$ loft --tests calc.loft
  ok    calc.loft  (2 fns: test_double_doubles, test_double_of_zero)
test result: ok. 2 passed; 1 file
```

It finds every function whose name starts with ‘test_’, runs it, and names the file and the functions it found. That result line does not end where it does above — there is a bracket after it, which the last section on this page is about.

The underscore is part of the rule. ‘test_double’ is a test; ‘testify’, or a function called exactly ‘test’, is not — and nothing will tell you, because the run says ok having never called it. (A camel-case ‘testDouble’ cannot catch you out: loft refuses that as a function name.)

There is one exception, and it is what makes a plain script runnable. If a file names NO ‘test_’ function at all, loft treats every function in it that takes no parameters as something to run. That is how the pages of this reference are checked: they have a ‘main’ and no tests. Once one ‘test_’ appears in a file, it and its siblings are the whole set, and the helpers beside them go back to being helpers.

Give it a directory instead of a file and it looks in every ‘.loft’ file underneath. Give it nothing and it starts from where you are:

```
$ loft --tests greeter
  ok    greeter/tests/greet.loft  (1 fn: test_greet_names_the_person)
coverage: all 1 functions were entered by these tests
test result: ok. 1 passed; 1 file
```

Over a whole directory it also reports coverage — how many of your functions any of the tests actually entered. One file on its own gets no such line, because a file is not the set of functions you were trying to cover.

36.0.3. Run just one test

While you are fixing one thing, run only that one. Put '::' and the test's name after the file:

```
  $ loft --tests calc.loft::test_double_of_zero
    ok    calc.loft  (1 fn: test_double_of_zero)
  test result: ok. 1 passed; 1 file
```

36.0.4. When a test fails

A failure names the test and shows your message, once per test and once per file, and the run's exit status stops being zero — which is how a build script finds out:

```
  $ loft --tests broken/failing.loft ; echo "exit $?"
    FAIL  broken/failing.loft::test_double_of_two  –  assertion failed: double(2)
should be 5
    FAIL  broken/failing.loft  (1 failed, 0 passed)
  test result: FAILED. 1 failed; 0 passed; 1 total; 1 file
  exit 1
```

The message is the part you wrote, which is why a vague message like "it works" costs you time and a specific one saves it.

36.0.5. Read the line after the result

The result line ends with a bracket saying what the run did NOT do:

```
  $ loft --tests calc.loft
  test result: ok. 2 passed; 1 file  [ran on the interpreter only – native not
exercised: loft test --native; no [sandbox] policy – admission not exercised]
```

That is deliberate. A run that says only "ok" cannot tell you whether the other half was checked or never ran at all. Run them the compiled way and the bracket says the opposite:

```
  $ loft --tests --native calc.loft
  test result: ok. 2 passed; 1 file  [ran on native only – the interpreter not
exercised: loft test; no [sandbox] policy – admission not exercised]
```

Both halves matter, because the two ways of running your program are meant to give the same answer and a test only checks the one it ran.

36.0.6. Once you have a project

When your code grows into a project with a 'loft.toml' file (the next page), tests live in a 'tests/' folder and you run them with:

```
  $ cd greeter && loft test
  test result: ok. 1 passed; 1 file
```

```
fn double(n: integer) -> integer { n * 2 }
```

This page is itself run as a test, so the example below really does pass.

```
fn main() {
  assert(double(4) == 8, "double(4) should be 8");
  assert(double(0) == 0, "double(0) should be 0");
```

An assert that holds produces no output at all — silence is success.

```
  println("both checks passed");
}
```

37. Debugging

When a program does the wrong thing, the usual reaction is to add printing to it, run it, read the output, and take the printing out again. loft gives you a better tool: stop the program on the line you care about and look around.

The program used below is ‘count.loft’. It adds 1, 2 and 3 into a total.

37.0.1. Stop on a line

Say ‘debug’, then your file, then a colon and a line number:

```
$ printf ':continue\n' | loft debug count.loft:4
⏸ paused in main | total = 0, i = 1   (+2 compiler temp(s) — `:vars all`)
```

The program runs until it reaches line 4 and then waits for you. The line it prints tells you where you are and what your locals hold right now. The note at the end counts the variables the compiler made for itself — the loop’s hidden index and a scratch slot — and ‘:vars all’ shows those too:

```
$ printf ':vars all\n:quit\n' | loft debug count.loft:4
⏸ paused in main | total = 0, i#index = 1, i = 1, __work_1 = ""
```

Line 4 is inside a loop that goes round three times, so the program stops there three times. ‘:continue’ means “carry on until you reach this line again”, not “run to the end” — which is why the examples below say ‘:continue’ more than once when they want the program to finish.

37.0.2. Look at a value

Type the name of a variable and press enter. Any expression works, not just a name — it is worked out where the program is paused, so it can combine the locals you are looking at:

```
$ printf 'total + i * 111\n:quit\n' | loft debug count.loft:4
111
```

This is the part that replaces printing. You do not have to guess in advance which values you will want; ask for them when you are there.

37.0.3. Move one step at a time

Four commands move the program forward:

- ‘:step’ runs the next line, going INTO any function it calls
- ‘:next’ runs the next line, going OVER any function it calls
- ‘:finish’ runs until the current function returns
- ‘:continue’ carries on to the next time this line is reached

The first three only differ when there IS a call, so the program here is ‘steps.loft’, whose loop calls ‘add_up(i * 2)’ on line 9. Stopping on line 9 and stepping goes into it, and the paused line changes function:

```
$ printf ':step\n:quit\n' | loft debug steps.loft:9
⏸ paused in add_up | n = 2
```

‘:next’ at the same place runs the whole call and stays where you are:

```
$ printf ':next\n:quit\n' | loft debug steps.loft:9
⏸ paused in main | total = 2, i = 1
```

The program is now on the ‘for’ line, about to start the next turn: ‘total’ already holds this turn’s result, and ‘i’ still holds this turn’s value.

And from inside ‘add_up’, ‘:finish’ runs it out and lands back in the caller:

```
$ printf ':finish\n:quit\n' | loft debug steps.loft:2
⏸ paused in main | total = 0, i = 1
```

37.0.4. Watch a value instead of watching for it

Re-reading the paused line every time round the loop works, and ‘:watch’ does it for you: carry on, and the program stops when that value changes, saying what it changed from and to:

```
$ printf ':watch total\n:continue\n:quit\n' | loft debug count.loft:4
watching total — :continue and the run stops when it changes
⏯ watchpoint: total changed 0 → 1
```

37.0.5. Change a value while it is running

You can write to a local, not only read it. This answers “would it work if this were right?” without editing the file and starting again:

```
  $ printf 'total = 100\n:continue\n:continue\n:continue\n' | loft debug
count.loft:4
  total=106
  run finished
```

The program carries on with the value you gave it. It finished with 106 instead of 6, because the loop still had 2 and 3 to add after the change.

Changing the loop variable moves the loop itself. Set ‘i’ to 3 on the first turn and that turn adds 3 and is the last one, because the loop counts on from the value you gave it:

```
$ printf 'i = 3\n:continue\n' | loft debug count.loft:4
total=3
```

‘:undo’ takes an edit back, and ‘:redo’ puts it on again:

```
$ printf 'total = 100\n:undo\ntotal + 55\n:quit\n' | loft debug count.loft:4
55
```

37.0.6. Getting out

‘:continue’ carries on. ‘:quit’ stops right away. ‘:help’ lists every command if you forget one — including the short forms ‘:s’, ‘:n’, ‘:o’ and ‘:c’.

37.0.7. Typing, or feeding it a script

Every example here pipes its commands in, which is how they are checked automatically — the output you see above is compared against what the command really prints. Run ‘loft debug count.loft:4’ on its own and you get the ‘(dbg)’ prompt to type at instead. The commands are the same either way.

```
fn main() {
```

This is what `count.loft` does — the program the examples above debug.

```
  total = 0;
  for i in 1..4 {
    total += i;
  }
  assert(total == 6, "1 + 2 + 3 is 6, which is what the debugger shows you");
  println("total={total}");
}
```

38. Projects and libraries

One file is a complete loft program, and for a long time that is all you need. When you want to share code between programs, or keep tests beside it, you turn the folder into a package.

38.0.1. What a package looks like

A package is a folder with a ‘loft.toml’ file in it and two sub-folders:

```
greeter/
  loft.toml          what this package is called
  src/greeter.loft   the code
  tests/greet.loft   the tests
```

‘tests’ is where your tests live and the name matters — ‘loft test’ looks in exactly that folder, and a test file anywhere else is not found. ‘src’ is a habit rather than a rule: the manifest below names the entry file, and ‘src/<name>.loft’ is only what loft assumes when nothing says otherwise.

38.0.2. The manifest

‘loft.toml’ names the package and says which file is its front door:

```
[package]
name    = "greeter"
version = "0.1.0"
```

```
[library]
entry = "src/greeter.loft"
```

The ‘entry’ file is the one that other code sees. Anything you want to share from it is marked ‘pub’:

```
pub fn greet(who: text) -> text { "hello, {who}!" }
```

Without ‘pub’ a function is not private — it is un-imported. ‘pub’ is what lets a caller write the bare name; everything else in the entry file stays reachable as ‘greeter::name’. That is the difference between “you may use this” and “you cannot see this”, and loft only offers the first.

38.0.3. Testing a package

Inside the package folder, ‘loft test’ finds and runs everything in ‘tests’:

```
$ cd greeter && loft test
test result: ok. 1 passed; 1 file
```

The tests reach your code by importing the package by name:

```
use greeter::*;
fn test_greet_names_the_person() {
  assert(greet("Ada") == "hello, Ada!", "greet builds the sentence");
}
```

‘use greeter::*;’ brings in every ‘pub’ name, which is why ‘greet(“Ada”)’ works bare. A bare ‘use greeter;’ brings in only the prefix, so there you would write ‘greeter::greet(“Ada”)’. The selective forms give you a say in WHICH names arrive: ‘use sums::add’ takes one name from a package, and ‘use sums::(add, subtract)’ takes a group. Taking only what you name is the stronger position, because a package that later GROWS a name cannot then collide with one of yours.

The ‘greeter::’ prefix is available whichever form you wrote, and it is the spelling that says where a name came from — useful once you import several packages, and the only way to reach something the package did not mark ‘pub’.

38.0.4. Running one test while you work

Give ‘loft test’ the file, or the file and one function:

```
$ cd greeter && loft test greet
test result: ok. 1 passed; 1 file
```

38.0.5. Check it compiles, without running anything

Inside the package, ‘loft check’ compiles everything and reports problems:

```
$ cd greeter && loft check
loft build: src/greeter.loft → target(s): native
loft build: building `native` (Native) …
ok
loft build: `native` ✓
```

A package that is only a library has no program to start, so there is nothing to run — ‘check’ still tells you the code compiles, which is what you wanted to know.

38.0.6. Coverage — what the tests did not reach

Just above its result line, ‘loft test’ tells you which of your functions no test ever entered. The ‘sums’ package next door has two functions and a test for one of them, so it names the other:

```
$ cd sums && loft test
coverage: 1 of 2 functions were never entered by these tests
  src/sums.loft:3  subtract
```

When nothing is missing it says so in one line instead:

```
$ cd greeter && loft test
coverage: all 1 functions were entered by these tests
```

It is a list, never a percentage and never a gate. A percentage becomes a target, and tests written to move a number check nothing.

38.0.7. Using someone else’s package

Add it to the manifest under ‘[dependencies]’ and install:

```
  [dependencies]
  math = ">=0.1"
```

```
  loft install
```

Then ‘use math;’ in your code. ‘loft install’ also writes a lock file that records the exact versions, so the same code builds the same way later.

```
fn main() {
```

The package these examples describe is `greeter`, and this is its one function — a `pub fn` in `src/greeter.loft` that its tests import.

```
  greeting = "hello, Ada!";
  assert(greeting == "hello, Ada!", "what greet(\"Ada\") returns");
  println(greeting);
}
```

39. Call it yourself

Every page here has a panel at the bottom that runs its code; this is the page whose code was chosen for you to call. The functions below are compiled and live in that panel: type a call, press enter, and loft answers. Nothing is a transcript — the answer you see is the one your browser just computed.

39.0.1. What to try first

`fib(10)` is the shortest way to be sure the panel is real. Change the 10 and the answer changes; ask for `fib(30)` and you will wait a moment, because this is the naive recursion and it really does make that many calls.

```
fn fib(n: integer) -> integer {
  if n < 2 {
    return n;
  }
  fib(n - 1) + fib(n - 2)
}
```

39.0.2. Numbers with a shape

`nth\_prime(1)` is 2, `nth\_prime(10)` is 29. Try to guess `nth\_prime(100)` before you ask for it — the primes thin out faster than most people expect.

```
fn is_prime(n: integer) -> boolean {
  if n < 2 {
    return false;
  }
  d = 2;
  while d * d <= n {
    if n % d == 0 {
      return false;
    }
    d += 1;
  }
  true
}
```

```
fn nth_prime(n: integer) -> integer {
  found = 0;
  candidate = 1;
  while found < n {
    candidate += 1;
    if is_prime(candidate) {
      found += 1;
    }
  }
  candidate
}
```

39.0.3. Text goes in, and text comes back

`vowels("your name here")` counts the vowels in whatever you type, and `shout("your name here")` hands the text back changed. A text answer arrives quoted — `shout("hi")` shows `"HI!"` — which is how you can tell an empty text from nothing at all.

```
fn shout(s: text) -> text {
  "{s.to_uppercase()}!"
}
```

```
fn vowels(s: text) -> integer {
  n = 0;
  for c in s {
    if c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' {
      n += 1;
    }
  }
  n
}
```

39.0.4. A struct comes back

`stats(3, 17)` answers `{"lo":3,"hi":17,"span":14}` — a whole record, rendered as JSON, from one call. Give it the arguments in the wrong order and it still answers correctly, which is the point of it.

```
struct Span {
  lo: integer,
  hi: integer,
  span: integer,
}
```

```
fn stats(a: integer, b: integer) -> Span {
  lo = if a < b { a } else { b };
  hi = if a < b { b } else { a };
  Span { lo: lo, hi: hi, span: hi - lo }
}
```

39.0.5. Pausing inside a run

The panel can also stop the program mid-flight. Click a line number in the code above to set a breakpoint, press Run, and the panel lists the variables that line can see — then the same prompt evaluates against THAT frame, so a call like `fib(step)` uses the `step` the paused function is holding.

```
fn walk_to(limit: integer) -> integer {
  total = 0;
  step = 0;
  while step < limit {
    step += 1;
    total += fib(step);
  }
```

```
  total
}
```

39.0.6. When the panel says \<unavailable\>

Every value shape answers — a number, a character, a boolean, a text, a vector, a struct. \<unavailable\> is what you get when the expression itself cannot be compiled where the program is paused: a typo, a name that is not in scope there, an unfinished expression. It is the panel declining to guess rather than a value it could not carry, so read it as “say that again” and not as “this shape is not supported”. One \<unavailable\> does not end the session; the next expression is evaluated normally.

```
fn main() {
```

The demonstration comes first and the checks follow it, so a reader who presses Run sees an answer straight away — the panel stops on main’s LAST line, and a program whose only print IS that line would pause with nothing shown yet.

```
  print("fib(10)={fib(10)} nth_prime(10)={nth_prime(10)}
shout(\"hi\")={shout(\"hi\")}\n");
```

The page is a real program, so these run on both backends like every other page here, and an example that stops being true stops compiling.

```
  assert(fib(10) == 55, "fib(10)");
  assert(fib(1) == 1 && fib(0) == 0, "fib base cases");
  assert(nth_prime(1) == 2, "the first prime is 2");
  assert(nth_prime(10) == 29, "the tenth prime is 29");
  assert(!is_prime(1) && is_prime(2) && !is_prime(9), "is_prime edges");
  assert(vowels("hello") == 2, "hello has two vowels");
  assert(vowels("") == 0, "no text, no vowels");
  assert(shout("hi") == "HI!", "shout answers a text, which the panel renders");
  s = stats(17, 3);
  assert(s.lo == 3 && s.hi == 17 && s.span == 14, "stats orders its arguments");
```

fib(1..4) is 1 + 1 + 2 + 3.

```
  assert(walk_to(4) == 7, "walk_to sums the first four");
}
```

40. Standard Library

40.1. Types

```
pub type boolean size(1)
```

Primitive types built into loft. True or false value.

```
pub type integer size(8)
```

64-bit signed integer.

```
pub type single size(4)
```

32-bit floating-point. Good for graphics and performance-sensitive math.

```
pub type float size(8)
```

64-bit floating-point. Use when precision matters.

```
pub type text size(4)
```

UTF-8 string.

```
pub type character size(4)
```

A single Unicode code point.

```
pub type u8 = integer limit(0, 255) size(1)
```

Integer subtypes for compact storage in struct fields. Behave as integer in expressions. 0 – 255, 1 byte.

```
pub type i8 = integer limit(-128, 127) size(1)
```

–128 – 127, 1 byte.

```
pub type u16 = integer limit(0, 65535) size(2)
```

0 – 65535, 2 bytes.

```
pub type i16 = integer limit(-32768, 32767) size(2)
```

–32768 – 32767, 2 bytes.

```
pub type i32 = integer size(4)
```

4-byte signed: −2_147_483_647 – 2_147_483_647. The bottom of the 32-bit range (`-2147483648`) is the null sentinel, so it is not a value an `i32` can hold. `u32` reserves a code too, at the top of its range, and the two reservations LOOK alike and do not behave alike: an `i32` overflow reads `null`, a `u32` overflow reads `0`, the type's default. That is not a claim that the reservations differ — it is C127's pragmatic exemption, allowed because losing one value out of 2^32 is cheap and being the one narrow type whose overflow is detectable is worth more (DESIGN_DECISIONS.md C127 § Decision). Pinned across both aliases by `tests/scripts/1615-every-narrow-slot-answers-the-types-default-for-an-unfitting-value`.

```
pub type u32 = integer limit(0, 4294967294) size(4)
```

4-byte unsigned: 0 – 4_294_967_294. The top code is reserved to buy the same refusal `i32` gets at the bottom (`u32 = 4294967295` does not compile) — and, unlike `i32`, it is NOT a null: an overflow here reads `0`, the type's default (C127, and the note on `i32` above says why the two differ). size(4) forces 4-byte storage and serialisation symmetric with `i32`, and a value in the upper half (2^31..2^32-2) reads back as the same positive integer. `u32` is for file / wire-format fields (magic numbers, counts, tick fields) where the byte width is the load-bearing property; plain `integer` holds the whole 0..2^32-1 range and more.

```
pub type i64 = integer size(8)
```

8-byte signed: the full `integer`, spelled so that it stays 64-bit in a file under the 32-bit regime (`\#width 32`), where a plain `integer` is an `i32`. Elsewhere it is `integer` by another name.

```
pub type f64 = float
```

The 64-bit `float`, spelled so that it stays a double in a file under the 32-bit regime (`\#width 32`), where a plain `float` is a `single`. Elsewhere it is `float`.

```
pub enum Ordering {
  Less,
  Equal,
  Greater
}
```

Where one value stands against another: what `compare` answers. An enum rather than a signed number, so a comparison cannot be written as a subtraction that overflows.

40.2. Interfaces

```
pub interface Ordered {
  operator compare(self: Self, other: Self) -> Ordering
}
```

Standard interfaces for bounded generic functions. A type satisfies an interface by defining the `operator` method or the method it names — a built-in type by the stdlib's own, a program's type by its own (`formal/operators.md`). Types that support the `\<` comparison operator. Satisfied by integer, single, float, text, and any type defining `operator compare`. `boolean` is NOT among them, deliberately:

it satisfies Equatable below and has no ordering, so `false \< true` is a refusal rather than a convention the language picks for you. A program that wants it says so — `(a as integer) \< (b as integer)`. The same line bounds how null is ordered: that applies to the ordered types only, and a boolean null still compares with == like every other scalar. ONE method is all a type has to define: inside a generic bounded by this, \<, \<=, \> and \>= all read it, each evaluating its operands exactly once.

```
pub interface Equatable {
}
```

Types that support the == equality operator: every type, compared by content at every depth (@C91). == is always structural and no type redefines it, so this declares nothing; a deep comparison of a type's own is a named method. A tuple compares element by element, as its concrete == does. `!=` derives from it (`a != b` is `!(a == b)`).

```
pub interface Addable {
  operator plus(self: Self, other: Self) -> Self
}
```

Types that support the + addition operator, returning the same type. Satisfied by integer, single, float, and any type defining `operator plus`.

```
pub interface Numeric {
  operator times(self: Self, other: Self) -> Self
  operator negate(self: Self) -> Self
}
```

Types that support * and - (unary negation). Separate from `Addable` so a generic can ask for the fewest operators it needs. Satisfied by integer, single, float, and any type defining `operator times` and `operator negate`. Binary subtraction is `Subtractable` below and deliberately NOT here. One interface CAN declare both arities of -, but adding a requirement to `Numeric` would take satisfaction away from every type that meets it, which a compatible release may not do.

```
pub interface Subtractable {
  operator minus(self: Self, other: Self) -> Self
}
```

Types that support binary - (subtraction), returning the same type. Satisfied by integer, single, float, and any type defining `operator minus`. A bound of its own rather than a third requirement on `Numeric`, so `\<T: Numeric + Subtractable\>` gets negation and subtraction together from two interfaces. The built-in number types satisfy both.

```
pub interface Scalable {
  fn scale(self: Self, factor: integer) -> integer
}
```

Types that support integer scaling via a `scale` method. Uses a method (not `operator times`): inside a `\<T: Numeric + …\>` body, * already means Numeric's `Self \* Self`, so `x \* 2` there is refused

("operator '*' requires a concrete type") even when another bound declares `operator times(self: Self, f: integer)`. Two SEPARATE generics may each bound their own `T` by an interface declaring the same method differently — a header binds its own type variable. User types satisfy Scalable by defining `fn scale(self: T, factor: integer) -\> integer`.

```
pub interface Printable {
  operator to_text(self: Self) -> text
}
```

Types that can be converted to text via a `to\_text` method. User types satisfy Printable by defining `operator to\_text(self: T) -\> text`.

40.3. Math

Functions for numeric computation. All trigonometric functions work in radians. Both single and float variants exist for every floating-point function — choose single for speed, float for precision.

```
pub fn abs(self: integer) -> integer
```

Absolute value. Removes the sign from a negative integer.

```
pub fn floor_mod(self: integer, divisor: integer) -> integer?
```

Floor modulo — the remainder that takes the sign of the DIVISOR, so `x.floor\_mod(n)` lands in `\[0, n)` for a positive `n`. The `%` operator truncates toward zero and keeps the DIVIDEND's sign (`-1 % 3 == -1`); `floor\_mod` wraps (`(-1).floor\_mod(3) == 2`), which is what circular indexing / wrap-around wants (`grid\[(i - 1).floor\_mod(w)\]`). `x.floor\_mod(0)` is null, like `%` (C80 — an undefined value is a null, never a fault).

```
pub fn abs(self: single) -> single
```

Absolute value for single-precision floats.

```
pub fn cos(self: single) -> single
```

Cosine. Use for circular motion: x = r * cos(angle).

```
pub fn sin(self: single) -> single
```

Sine. Use for circular motion: y = r * sin(angle).

```
pub fn tan(self: single) -> single
```

Tangent. Use for slopes and perspective projection.

```
pub fn acos(self: single) -> single?
```

Arc cosine. Returns the angle (in radians) whose cosine is the value; null outside [−1, 1].

```
pub fn asin(self: single) -> single?
```

Arc sine. Returns the angle whose sine is the value; null outside [−1, 1].

```
pub fn atan(self: single) -> single
```

Arc tangent of a single value. Returns angle in (-PI/2, PI/2).

```
pub fn ceil(self: single) -> single
```

Round up to the nearest integer value. Use to compute required buffer sizes from fractional counts.

```
pub fn floor(self: single) -> single
```

Round down to the nearest integer value. Use to convert a float position to a tile index.

```
pub fn round(self: single) -> single
```

Round to the nearest integer value (half rounds away from zero).

```
pub fn sqrt(self: single) -> single?
```

Square root. Use for distances and normalization. Null for a negative value.

```
pub fn atan2(self: single, v2: single) -> single
```

Arc tangent of y/x as `y.atan2(x)`, preserving the correct quadrant: `atan2(1.0, 0.0)` is PI/2. Use instead of atan when you have separate x/y components.

```
pub fn log(self: single, v2: single) -> single?
```

Logarithm in the base given second: `log(8.0, 2.0)` is 3. Null for a negative value, -inf at 0. Use for converting between scales (e.g., decibels).

```
pub fn pow(self: integer, v2: integer) -> integer?
```

Raises the integer to the power given second: `pow(3, 4)` is 81, and `3 \*\* 20` is 3486784401 exactly. Null when the exponent is negative (no integer answers it) or the answer does not fit an integer (`pow(2, 63)`), as `\*` answers null past the range.

```
pub fn pow(self: single, v2: single) -> single?
```

Raises the value to the power given second: `pow(2.0, 10.0)` is 1024. Null when the answer is not a real number (`pow(-8.0, 0.5)`). Use for exponential growth curves and scaling.

```
pub fn abs(self: float) -> float
```

Absolute value for double-precision floats.

```
pub PI = OpMathPiFloat()
```

The ratio of a circle's circumference to its diameter (3.14159...).

```
pub E = OpMathEFloat()
```

Euler's number, the base of natural logarithms (2.71828...).

```
pub fn cos(self: float) -> float
```

Cosine. Use for circular motion: x = r * cos(angle).

```
pub fn sin(self: float) -> float
```

Sine. Use for circular motion: y = r * sin(angle).

```
pub fn tan(self: float) -> float
```

Tangent. Use for slopes and perspective projection.

```
pub fn acos(self: float) -> float?
```

Arc cosine. Returns the angle (in radians) whose cosine is the value; null outside [−1, 1].

```
pub fn asin(self: float) -> float?
```

Arc sine. Returns the angle whose sine is the value; null outside [−1, 1].

```
pub fn atan(self: float) -> float
```

Arc tangent of a single value. Returns angle in (-PI/2, PI/2).

```
pub fn ceil(self: float) -> float
```

Round up to the nearest integer value. Use to compute required buffer sizes from fractional counts.

```
pub fn floor(self: float) -> float
```

Round down to the nearest integer value. Use to convert a float position to a tile index.

```
pub fn round(self: float) -> float
```

Round to the nearest integer value (half rounds away from zero).

```
pub fn sqrt(self: float) -> float?
```

Square root. Use for distances and normalization. Null for a negative value.

```
pub fn atan2(self: float, v2: float) -> float
```

Arc tangent of y/x as `y.atan2(x)`, preserving the correct quadrant: `atan2(1.0, 0.0)` is PI/2. Use instead of atan when you have separate x/y components.

```
pub fn log(self: float, v2: float) -> float?
```

Logarithm in the base given second: `log(8.0, 2.0)` is 3. Null for a negative value, -inf at 0. Use for converting between scales (e.g., decibels).

```
pub fn pow(self: float, v2: float) -> float?
```

Raises the value to the power given second: `pow(2.0, 10.0)` is 1024. Null when the answer is not a real number (`pow(-8.0, 0.5)`). Use for exponential growth curves and scaling.

40.4. exp / ln / log2 / log10

```
pub fn exp(self: single) -> single
```

Raises E (2.71828...) to the power v. Use for exponential growth models.

```
pub fn exp(self: float) -> float
```

Double-precision exp (e^v).

```
pub fn ln(self: single) -> single?
```

Natural logarithm (base E). Use for growth rates and information entropy.

```
pub fn ln(self: float) -> float?
```

Double-precision natural logarithm.

```
pub fn log2(self: single) -> single?
```

Base-2 logarithm. Use for bit-count calculations and information theory.

```
pub fn log2(self: float) -> float?
```

Double-precision base-2 logarithm.

```
pub fn log10(self: single) -> single?
```

Base-10 logarithm. Use for decibels, orders of magnitude, and display scales.

```
pub fn log10(self: float) -> float?
```

Double-precision base-10 logarithm.

40.5. min / max / clamp

```
pub fn min(self: integer, b: integer) -> integer
```

Each of these has two overloads: one over non-null values (`-\> τ`) and one over nullable ones (`-\> τ?`). You get the nullable overload whenever ANY argument is statically nullable — an `integer?`/`single?`/`float?`, or a division result such as `1 / z` — and it propagates: the answer is null if either argument is null. Otherwise you get the non-null overload and a non-null answer, so a plain `min(a, b)` needs no discharge. Smallest of two integer values.

```
pub fn max(self: integer, b: integer) -> integer
```

Largest of two integer values.

```
pub fn clamp(self: integer, lo: integer, hi: integer) -> integer
```

Clamps v into the inclusive range [lo, hi]. Returns null if any argument is null.

```
pub fn min(self: single, b: single) -> single
```

Smallest of two single-precision float values.

```
pub fn max(self: single, b: single) -> single
```

Largest of two single-precision float values.

```
pub fn clamp(self: single, lo: single, hi: single) -> single
```

Clamps v into the inclusive range [lo, hi]. Returns null if any argument is null.

```
pub fn min(self: float, b: float) -> float
```

Smallest of two double-precision float values.

```
pub fn max(self: float, b: float) -> float
```

Largest of two double-precision float values.

```
pub fn clamp(self: float, lo: float, hi: float) -> float
```

Clamps v into the inclusive range [lo, hi]. Returns null if any argument is null.

```
pub fn then(self: Ordering, next: Ordering) -> Ordering
```

This answer, or `next` when this one is `Equal`: compares by several fields in turn, `a.year.compare(b.year).then(a.month.compare(b.month))`.

```
pub operator compare(self: integer, other: integer) -> Ordering
```

Where `self` stands against `other` — `Less`, `Equal` or `Greater` — in the order `\<` uses.

```
pub operator compare(self: single, other: single) -> Ordering
```

Where `self` stands against `other`, in the order `\<` uses.

```
pub operator compare(self: float, other: float) -> Ordering
```

Where `self` stands against `other`, in the order `\<` uses.

```
pub operator plus(self: integer, other: integer) -> integer
```

`+` as the method `Addable` names: `a.plus(b)` is `a + b`.

```
pub operator minus(self: integer, other: integer) -> integer
```

`-` as the method `Subtractable` names: `a.minus(b)` is `a - b`.

```
pub operator times(self: integer, other: integer) -> integer
```

`\*` as the method `Numeric` names: `a.times(b)` is `a \* b`.

```
pub operator negate(self: integer) -> integer
```

The unary `-` as the method `Numeric` names: `a.negate()` is `-a`.

```
pub operator divided_by(self: integer, other: integer) -> integer?
```

`/` as a method: `a.divided\_by(b)` is `a / b`, null on a zero divisor.

```
pub operator remainder(self: integer, other: integer) -> integer?
```

`%` as a method: `a.remainder(b)` is `a % b`, null on a zero divisor.

```
pub operator plus(self: single, other: single) -> single
```

`+` as the method `Addable` names: `a.plus(b)` is `a + b`.

```
pub operator minus(self: single, other: single) -> single
```

`-` as the method `Subtractable` names: `a.minus(b)` is `a - b`.

```
pub operator times(self: single, other: single) -> single
```

* as the method Numeric names: a.times(b) is a * b.

```
pub operator negate(self: single) -> single
```

The unary - as the method Numeric names: a.negate() is -a.

```
pub operator divided_by(self: single, other: single) -> single?
```

/ as a method: a.divided_by(b) is a / b, null on a zero divisor.

```
pub operator remainder(self: single, other: single) -> single?
```

% as a method: a.remainder(b) is a % b, null on a zero divisor.

```
pub operator plus(self: float, other: float) -> float
```

+ as the method Addable names: a.plus(b) is a + b.

```
pub operator minus(self: float, other: float) -> float
```

- as the method Subtractable names: a.minus(b) is a - b.

```
pub operator times(self: float, other: float) -> float
```

* as the method Numeric names: a.times(b) is a * b.

```
pub operator negate(self: float) -> float
```

The unary - as the method Numeric names: a.negate() is -a.

```
pub operator divided_by(self: float, other: float) -> float?
```

/ as a method: a.divided_by(b) is a / b, null on a zero divisor.

```
pub operator remainder(self: float, other: float) -> float?
```

% as a method: a.remainder(b) is a % b, null on a zero divisor.

```
pub fn approx(self: float, b: float, eps: float) -> boolean
```

Approximate equality: true when a and b differ by at most eps (inclusive). == on float/single is EXACT IEEE equality — use approx when you want a tolerance, e.g. comparing computed transcendentals: approx(sqrt(2.0) * sqrt(2.0), 2.0, 1e-9). A null (NaN) operand is not approximately equal to anything, so the result is false.

```
pub fn approx(self: single, b: single, eps: single) -> boolean
```

Approximate equality for single-precision floats (see the float overload).

40.6. Text

Functions for working with text (UTF-8 strings) and character values.

```
pub fn len(self: text) -> integer
```

Number of characters (Unicode code points) in the text — the human count, as in every mainstream language. For a byte length (bounds checks, the limit of byte-positioned indexing / slicing) use `size`.

```
pub fn size(self: text) -> integer
```

Number of bytes in the text. This is the bound for byte-positioned operations: `s\[i\]`, slices `s\[a..b\]`, and `find`/`rfind` all use byte offsets. For the human character count use `len`.

```
pub fn len(self: character) -> integer
```

Byte length of the character's UTF-8 encoding (1–4).

```
pub operator compare(self: text, other: text) -> Ordering
```

Where `self` stands against `other`, in the order `\<` uses: by bytes, as `\<` compares text.

```
pub operator compare(self: character, other: character) -> Ordering
```

Where `self` stands against `other`, by code point, in the order `\<` uses.

```
pub fn split(self: text, separator: character) -> vector<text>
```

Splits self on every occurrence of separator and returns the parts as a vector. Use to parse CSV lines or space-separated tokens. Every separator ends a part, so a trailing separator leaves an empty last part ("a,".split(',') is ["a", ""]); an empty text has no parts at all.

```
pub fn split_text(self: text, separator: text) -> vector<text>
```

Split on a multi-character text separator. `"a, b, c".split\_text(", ")` returns `\["a", "b", "c"\]`. Edge cases: - empty self ⟶ empty result - empty separator ⟶ returns `\[self\]` unchanged (no infinite-split) - separator never matches ⟶ returns `\[self\]` - separator at start / end ⟶ produces empty boundary entries

```
pub fn starts_with_at(self: text, pos: integer, prefix: text) -> boolean
```

Functions for searching, transforming, and classifying text and character values. Character classification functions return true only if every character in the text satisfies the condition. The single-character

variants test one code point. Returns true if self contains `prefix` starting at byte position `pos`. Sugar over `self\[pos..pos + prefix.size()\] == prefix` for the common "is this token at this offset?" pattern in scanners / parsers. Returns false (not an error) when pos + prefix.size() exceeds self.size() — same shape as `starts\_with` for the "prefix too long for input" case. Faster than comparing characters one at a time for a known prefix at `pos`.

```
pub fn char_slice(self: text, from: integer, to: integer) -> text
```

The characters of `self` from `from` up to (not including) `to` — a slice that counts CHARACTERS where `self\[a..b\]` counts BYTES. ⚠️ **This is the cure for the commonest text bug in this stack.** `len(text)` counts characters while `self\[a..b\]` slices bytes, and loft snaps a byte cut outward to a character boundary — so a count computed from `len` and applied as a byte range fits FEWER characters than it measured. The failure is not mojibake but something quieter: the text comes back SHORT, silently, and only when it is not ASCII, so every ASCII test stays green. `"héllo"\[0..4\]` is `"hél"`; `"héllo".char\_slice(0, 4)` is `"héll"`. Use it wherever a character COUNT becomes a cut: fitting a label to a width, wrapping a line, a caret or a selection in a text field, truncating with an ellipsis. Reach for `self\[a..b\]` only when the numbers came from a byte source — `find`, `size`, `byte\_at`. Negative bounds count from the end, as a byte slice's do (`char\_slice(-2, n)` is the last two characters); both ends clamp, and a reversed or empty range answers `""` rather than failing.

```
pub fn trim(self: text) -> text[self]
```

Removes leading and trailing whitespace. Use when processing user input or file content.

```
pub fn trim_start(self: text) -> text[self]
```

Removes leading whitespace only.

```
pub fn trim_end(self: text) -> text[self]
```

Removes trailing whitespace only.

```
pub fn find(self: text, value: text) -> integer?
```

Returns the byte index of the first occurrence of value, or null if not found. Use to locate substrings before slicing. Nullable: null when value is absent (the type is honest about the not-found case).

```
pub fn rfind(self: text, value: text) -> integer?
```

Returns the byte index of the last occurrence of value, or null if not found. Use to find file extensions or the last path separator. Nullable: null when value is absent (the type is honest about not-found).

```
pub fn contains(self: text, value: text) -> boolean
```

Returns true if value appears anywhere in self. `s.contains(v)` is `s.find(v) != null`: `find` answers the position, `contains` only whether there is one — the clearer spelling for a membership test.

```
pub fn replace(self: text, value: text, with: text) -> text
```

Returns a copy of self with every occurrence of value replaced by with.

40.7. Collections

Operations on vector<T> — the primary ordered collection type. Vectors are grown by appending with += and elements are accessed by index. A vector or struct passed to a function is passed by reference: the callee's appends and element writes reach the caller. Binding one to a new local (`w = v`) copies it.

```
pub fn len(self: const vector) -> integer
```

Number of elements in the vector. Use in loop bounds: for i in 0..v.len().

```
pub fn len(self: const sorted) -> integer
```

Number of elements in a sorted collection.

```
pub fn clear(self: vector)
```

Remove all elements from the vector, setting its length to 0.

```
pub fn len(self: const hash) -> integer
```

Number of elements in a hash collection.

```
pub fn size(self: const hash) -> integer
```

The byte footprint of a hash: its full bucket table, holes included. The table is the hash's own allocation — `elms` slots (an empty slot is a hole and still counts, because open addressing's spare capacity IS the format). A slot is 8 bytes; a hash that owns its records keeps each in its slot too, so a slot then costs 8 bytes plus the record. Records kept in separate allocations are not counted. Grows in steps as the table rehashes (it is rebuilt at half full). 0 for an empty (unallocated) hash.

```
pub fn size(self: const index) -> integer
```

The byte footprint of an index: its B+tree's node arena, room included. Allocation-local, as `size` of a hash: the element records live in separate allocations and are not counted. Grows in steps as the arena doubles. 0 for an empty (unallocated) index.

40.8. Output and Diagnostics

```
pub fn assert(test: boolean, message: text, file: text, line: integer)
```

Panics with message if test is false. Use to verify invariants during development. In production mode (–production), logs an error instead of aborting. The file and line are injected by the compiler; do not pass them manually. Safe to call from `par` workers: failures write to the log/stderr sink, which the host serialises across threads.

```
pub fn panic(message: text, file: text, line: integer)
```

Immediately terminates execution with message. Use for unrecoverable error states. In production mode (–production), logs a fatal entry and execution continues. The file and line are injected by the compiler; do not pass them manually.

```
pub fn exit(code: integer) process#update
```

Ends the program at once with exit status `code`, as a shell's `exit` does: what was printed is flushed, nothing more is printed, and no diagnostic names the line — a script's verdict is its own, where `assert` and `panic` answer loft's (status 1 and a message). Under `loft test` it ends the test instead, as a failure that names the code.

```
pub fn log_info(message: text, file: text, line: integer) io#append
```

Write a structured log record at the chosen severity. The file and line are injected by the compiler; do not pass them manually. Safe to call from `par` workers: the host serialises log writes across threads.

```
pub fn log_warn(message: text, file: text, line: integer) io#append
```

Like log_info, at warning severity.

```
pub fn log_error(message: text, file: text, line: integer) io#append
```

Like log_info, at error severity.

```
pub fn log_fatal(message: text, file: text, line: integer) io#append
```

Like log_info, at fatal severity.

```
pub fn print(v1: text) io#append
```

Writes v to standard output without a newline. Use for progress output and building up a line incrementally.

```
pub fn println(v1: text) io#append
```

Writes v followed by a newline. The standard choice for line-oriented output.

```
pub fn eprint(v1: text) io#append
```

Writes v to standard ERROR without a newline. Companion to print() — use to separate human-readable status / progress / summary lines from machine-readable stdout (JSON, structured data piped to downstream consumers).

```
pub fn eprintln(v1: text) io#append
```

Writes v followed by a newline to standard ERROR.

```
pub fn len(self: const spatial) -> integer
```

Number of elements in a spatial (radix / Morton tree) collection.

```
pub fn size(self: const spatial) -> integer
```

The byte footprint of a spatial index: its tree's node array, room included. Allocation-local, as `size` of a hash: the element records are not counted. 0 for an empty (unallocated) one.

```
pub fn len(self: const trie) -> integer
```

Number of elements in the trie.

```
pub fn size(self: const trie) -> integer
```

The byte footprint of a trie: its tree's node array, room included. Allocation-local, as `size` of a hash: the element records are not counted. 0 for an empty (unallocated) one.

40.9. Vector aggregates

```
pub fn contains < T: Equatable > (self: const vector<T>, value: T) -> boolean
```

Does the vector hold `value`? Compared by content, as == compares every type; the empty vector holds nothing. The membership test a script asks of a list of names — `changed.contains(path)` — beside `text`'s own `contains`.

```
pub fn min_of < T: Ordered > (v: const vector<T>) -> T?
```

Smallest element in a vector, or null when the vector is empty (the type is honest about the empty case). Works on any Ordered type (op <).

```
pub fn max_of < T: Ordered > (v: const vector<T>) -> T?
```

Largest element in a vector, or null when the vector is empty (the type is honest about the empty case). Works on any Ordered type (op <).

```
pub fn sum < T: Addable > (v: const vector<T>, init: T? = null) -> T
```

Sum of vector elements. Works on any Addable type. `init` is the identity to start from; leave it out and the element type's own zero is used (0, 0.0). Example: sum([10, 20, 12], 0) == 42 Example: sum([10, 20, 12]) == 42

```
pub fn reverse < T > (self: vector<T>)
```

Reverse a vector's elements in place: `reverse(v)` or `v.reverse()`. A method on `vector`, so a program may define its own `reverse` for its own types beside it.

```
pub fn reserve < T > (self: vector<T>, n: integer)
```

Give a vector room for `n` elements, so filling it does not grow it step by step: `reserve(v, n)` or `v.reserve(n)`. Changes neither `len(v)` nor what is in it, and a count the vector already covers does nothing. A `hash` takes `reserve(h, n)` too.

```
pub fn insert < T > (self: vector<T>, index: integer, elem: T)
```

Insert `elem` at `index`, moving the elements from there one place up: `insert(v, i, x)` or `v.insert(i, x)`. Both arguments are values before the vector grows, so an element read from the vector itself (`v.insert(0, v\[1\])`) is the one it was.

```
pub fn sort < T: Ordered > (self: vector<T>)
```

Sort a vector in place, ascending: `sort(v)` or `v.sort()`. Takes any element with a `\<` (`Ordered`) — a struct defining `op \<` sorts by it — and is stable: elements that compare equal keep their order. A null element sorts first.

```
pub fn sort_by < T, K: Ordered > (self: vector<T>, key: fn(T) -> K)
```

Sort a vector in place by a key, ascending: `v.sort\_by(fn(d: Date) -\> integer { d.ms })`. The key is any `Ordered` value — a number or a `text` — so this is how a struct sorts (C132). Stable, like `sort`: elements with equal keys keep their order. `key` is called once per element.

```
pub fn min_by < T, K: Ordered > (self: vector<T>, key: fn(T) -> K) -> T?
```

The element with the smallest key, or null when the vector is empty: `v.min\_by(fn(d: Date) -\> integer { d.ms })`. Of several with that key, the FIRST.

```
pub fn max_by < T, K: Ordered > (self: vector<T>, key: fn(T) -> K) -> T?
```

The element with the largest key, or null when the vector is empty: `v.max\_by(fn(d: Date) -\> integer { d.ms })`. Of several with that key, the FIRST.

```
pub fn sum_by < T, K: Addable > (self: vector<T>, key: fn(T) -> K, init: K? =
null) -> K
```

The sum of a key over the elements: `v.sum\_by(fn(o: Order) -\> float { o.price })`. The key is any `Addable` value; `init` is the identity to start from, and left out it is the key type's own zero (0, 0.0), so an empty vector sums to it.

```
pub fn filter < T > (self: vector<T>, f: fn(T) -> boolean) -> vector<T>
```

Keep the elements `f` answers true for, in order: `filter(v, f)` or `v.filter(f)`.

```
pub fn map < T, U > (self: vector<T>, f: fn(T) -> U) -> vector<U>
```

What `f` answers for each element, in order: `map(v, f)` or `v.map(f)` — the result's element is what `f` returns, so a map can change the element type.

```
pub fn reduce < T, U > (self: vector<T>, init: U, f: fn(U, T) -> U) -> U
```

Fold the elements into one value, left to right, from `init`: `reduce(v, init, f)` or `v.reduce(init, f)`. Any accumulator: a number, a `text`, a struct, any collection.

```
pub fn sum_of(v: const vector<integer>) -> integer
```

Sum of all integer elements. Returns 0 for an empty vector. Superseded by the general `sum(v, init)`; kept as a shim over it (the old form keeps working). `sum\_of(v) == sum(v, 0)`.

```
pub interface Walkable {
  fn children(self: Self) -> vector<Self>
}
```

A type that can be walked as a tree. Declaring `fn children(self: T) -\> vector\<T\>` is the whole contract — that bare function is what makes `T` walkable, with nothing to register and no base type to inherit from. `tree\_walk` is what consumes it.

```
pub fn tree_walk < T: Walkable > (wk_root: T, cap: integer) -> vector<T>
```

Every node reachable from `wk\_root` in breadth-first order: the root, then its children, then theirs. `T` is any type declaring `fn children(self: T) -\> vector\<T\>` — writing that bare function is what makes a type `Walkable`, with nothing to register. `cap` bounds the ANSWER, not just the effort: at most `cap` nodes come back (the root alone when `cap` is 0 or 1) and the walk stops there. That bound is what makes this total on a structure that has a cycle or no end, so pass a count you would accept as an answer rather than a number picked to be large. Nodes are not de-duplicated — the walk carries no visited set, so a node reached by two parents is returned twice.

40.10. Assertions that report both sides

```
pub fn assert_eq < AssertValue: Equatable + Printable > (got: AssertValue, want:
AssertValue, what: text, file: text, line: integer)
```

Assert that two values are equal, naming BOTH of them when they are not. `assert(got == want, "…")` puts the expected value in the CONDITION, so a failure says what was got and leaves the reader to recover what was wanted by reading the expression. This says both. Works on any type that is `Equatable` (op ==) and `Printable` (to_text) — every built-in scalar, and a user type defining both. Failure behaves exactly as `assert` does, because it IS `assert`: the run aborts (and in `--production` logs an error instead), and the reported position is the CALLER's, not this function's. The file and line are injected by the compiler; do not pass them manually.

```
pub fn assert_ne < AssertValue: Equatable + Printable > (got: AssertValue, want:
AssertValue, what: text, file: text, line: integer)
```

Assert that two values DIFFER, naming the value both sides share when they do not. The mirror of assert_eq; same bounds, same failure behaviour, same position injection.

40.11. Field iteration support types

```
pub enum FieldValue {
  FvBool { v: boolean },
  FvInt { v: integer },
  FvLong { v: integer },
  FvFloat { v: float },
  FvSingle { v: single },
  FvChar { v: character },
  FvText { v: text },
}
```

Wraps a field value in a type-discriminated enum for compile-time field iteration.

```
pub struct StructField {
  pub name: text,
  pub value: FieldValue,
  // Was the field DECLARED nullable (`text?` rather than `text`)?
  //
  // The payload variants above are typed non-null, and `τ?` shares `τ`'s
  // runtime layout — a nullable field spells absence with a SENTINEL
  // rather than a wrapper. So a null field's value arrives inside `FvText` /
  // `FvInt` / … as that sentinel, and this flag is what says so; without it a
  // reader would have to discover the possibility by being surprised.
  //
  // Mirrors `FieldInfo.nullable` from `type_of` (07_reflect.loft) on purpose:
  // the two field lists describe the same declaration and must agree.
  pub nullable: boolean,
}
```

Represents a single struct field during compile-time iteration.

```
pub operator to_text(self: integer) -> text
```

Converts an integer to its decimal text. Built-in types define `to\_text` so they satisfy the `Printable` interface (used by `print`, format strings, ...).

```
pub operator to_text(self: float) -> text
```

Converts a float to its text representation.

```
pub operator to_text(self: single) -> text
```

Converts a single-precision float to its text representation.

```
pub operator to_text(self: boolean) -> text
```

Converts a boolean to “true” or “false”.

```
pub operator to_text(self: character) -> text
```

Converts a character to a one-character text.

```
pub operator to_text(self: text) -> text
```

Returns the text unchanged — the identity case, so `text` satisfies `Printable` too.

40.12. File System

Types and functions for reading and writing files. A File value is obtained via file() and carries the path, format, and an internal reference. Capability groups — the fs/env split a sandbox profile grants via `fs\#read` / `fs\#update` / `env\#read`, the host clock (`time\#read`) and the embedding host’s message channel (`host\#read` / `host\#append`); the functions below link in their signature.

```
pub struct EnvVariable {
  pub name: text,
  pub value: text,
}
```

One entry of the process environment, as `env\_variables()` answers them: the variable’s name and its value, both as plain text.

```
pub enum Format {
  TextFile,
  LittleEndian,
  BigEndian,
  Directory,
  NotExists,
}
```

Describes how a file is opened. TextFile: read or write as UTF-8 text (default). LittleEndian: binary mode, least-significant byte first. BigEndian: binary mode, most-significant byte first. Directory: represents a directory path. NotExists: no file or directory exist.

```
pub enum FileResult {
  Ok,
  NotFound,
  PermissionDenied,
  IsDirectory,
  NotEmpty,
  Other,
}
```

Result of a filesystem-mutating operation (delete, move, mkdir). Use ok() to get a simple boolean, or match on specific variants for detailed error handling. Every variant can actually be produced: Ok on success, NotFound for a missing / out-of-project path, PermissionDenied when the OS refuses access, IsDirectory when a file op targets a directory (e.g. deleting a directory with delete()), NotEmpty when rmdir() is given a directory that still holds entries, and Other for anything else. (–native and –interpret classify from the OS error; the wasm host reports only Ok / Other, and NotFound still comes from the loft-level existence check.)

```
pub fn ok(self: FileResult) -> boolean
```

True when the result is `FileResult.Ok`. Use to test a file op for success.

```
pub struct File {
  pub path: text,

  pub size: integer,

  // A fresh File reads as `NotExists` until `file()` has determined the real
format for
  // the path, so this default is a placeholder rather than a claim about the
path — it is
  // overwritten before any read.
  pub format: Format = NotExists,

  ref: i32?,

  current: integer,

  next: integer,
}
```

A handle to a filesystem entry. Fields: path (full path), size (file size in bytes), format (open mode), current (byte position after last read), next (byte position to read next).

```
pub fn content(self: const File) -> text?fs#read
```

Reads the entire file as a UTF-8 text value. Use for small configuration files or scripts.

```
pub fn lines(self: const File) -> vector<text> fs#read
```

Reads the file and splits it into lines. Strips trailing '\r' so CRLF files (Windows) and LF files (Unix) produce identical results. Use when processing line-by-line (logs, CSV, etc.).

40.13. Files and directories

```
pub fn path_sep() -> character
```

The separator of a loft path: '/' on every platform. A path a program hands loft may also use '\' (it reads as '/'), and every path loft hands back uses '/'.

```
pub fn file(path: text) -> File fs#read
```

Use as the entry point for all file I/O. A relative path resolves against the program's own directory (or against the working directory under `\#cwd`), so `../data.txt` names the file above the script — the same file an absolute path would name, and it answers the same either way. A path means the same on every platform: `/` and `\\` both separate; a name no platform could hold (`a:b`, `q?`, `aux.txt`, a trailing dot or space) is refused everywhere; a name means its exact spelling, so `B.txt` beside `b.txt` is refused. A refused path answers like a missing one, and is logged once.

```
pub fn exists(path: text) -> boolean fs#read
```

Use to check whether a path is accessible before reading or writing it. A RELATIVE path resolves against the program's own directory, or against the working directory under `\#cwd`; an absolute path is used as given, including one outside the project. This is not an access boundary — the boundary is the `fs` capability a `\[sandbox\]` profile grants or withholds.

```
pub fn exists(self: const File) -> boolean fs#read
```

Method form: f = file("path"); if f.exists() { .. } Also callable as exists(file_obj): a `self` first parameter gives a function both call spellings, the method one and the free one.

```
pub fn delete(path: text) -> FileResult fs#update
```

Use to remove a file after processing or as a cleanup step. Returns FileResult.Ok on success and FileResult.NotFound when the file did not exist. A path outside the project is deleted like any other; withhold the `fs` capability in a `\[sandbox\]` profile to prevent that.

```
pub fn move(from: text, to: text) -> FileResult fs#update
```

Use to rename or relocate a file. The destination must not already exist (FileResult.Other if it does), and a missing source is FileResult.NotFound. Neither path is confined to the project directory.

```
pub fn mkdir(path: text) -> FileResult fs#update
```

Create a single directory level; the parent must already exist (FileResult.NotFound when it does not). FileResult.Other when the directory is already there. rmdir() removes an empty directory again.

```
pub fn mkdir_all(path: text) -> FileResult fs#update
```

Create a directory and all missing parents (like Unix mkdir -p). Idempotent: a directory that already exists is FileResult.Ok, which is what makes it safe on the way into a run. rmdir() removes an empty directory again.

```
pub fn rmdir(path: text) -> FileResult fs#update
```

Remove the EMPTY directory `path`. Returns FileResult.NotEmpty when it still holds entries, NotFound when it does not exist, and Other when `path` is a file rather than a directory. Empty directories only: a recursive removal is a separate decision, and the walk is writable here — list_dir() the children, delete() the files, rmdir() the directory, deepest first. Use to undo a mkdir(), or to clean up a scratch directory a program made.

```
pub fn is_dir(path: text) -> boolean fs#read
```

Returns true if `path` exists and is a directory. Use to branch on a path's kind before descending into / reading it.

```
pub fn is_file(path: text) -> boolean fs#read
```

Returns true if `path` exists and is a regular file. Use to confirm a directory entry is a readable file before opening it.

```
pub fn is_symlink(path: text) -> boolean fs#read
```

Returns true if `path` itself is a symbolic link, whatever it points at — is_dir and is_file answer for the TARGET, so a walk that must not follow a link asks this first. False on a browser host, which has no links.

```
pub fn list_dir(path: text) -> vector<text> ?fs#read
```

Lists the entry NAMES of directory `path` (base names, not full paths), sorted. A MISSING / non-directory path lists as NULL (distinct from an EMPTY directory, `\[\]`); discharge with `?? \[\]` to keep the old shape. Use to enumerate a directory; join with `path` to build full child paths.

```
pub fn files(self: const File) -> vector<File> fs#read
```

Returns the entries inside a directory, sorted by path — the same order as `list\_dir`, so an index into either listing means the same entry. The File must have format == Format.Directory; anything else lists as `\[\]` (where `list\_dir` answers null, because it has no format to check first). Use to iterate over all files in a folder.

```
pub fn write(self: File, v: text) -> FileResult fs#update
```

Writes v as UTF-8 text at the file's position: the first write through a File replaces the file's content, later ones follow it. Returns FileResult.Ok on success and FileResult.Other on an OS write failure (disk full, permission denied, a bad path) — a failed write is OBSERVABLE, not silently swallowed. Discarding callers (`f.write(s)` as a statement) are unaffected; check with `f.write(s).ok()` or match the result.

40.14. Binary file I/O

```
pub fn read_bytes(path: text) -> vector<u8> ?fs#read
```

Reads the whole file `path` as raw bytes. A MISSING / unreadable file reads as NULL (distinct from an EMPTY file, `\[\]`); discharge with `?? \[\]` to keep the old shape. Binary-exact (round-trips with write_bytes); use for non-UTF-8 data — for text prefer `file(path).content()`.

```
pub fn file_map(path: text) -> const vector<u8> ?fs#read
```

Maps the whole file `path` READ-ONLY, without copying it: the vector's bytes ARE the file's, kept mapped for as long as the vector lives. A MISSING / unreadable file maps as NULL, like read_bytes. Every read works as on any vector<u8>; a slice is a view. The vector is `const`: a WRITE, to it or a slice, is refused when the program is compiled — copy first (`w = v`, a bind copies) to change bytes. For a file read once and whole, read_bytes gives the same bytes at the cost of the copy.

```
pub fn write_bytes(path: text, bytes: const vector<u8>) -> boolean fs#update
```

Writes `bytes` to file `path`, truncating any existing content. Returns true on success. The inverse of read_bytes; the pair round-trips a non-UTF-8 blob byte-for-byte.

```
pub fn mtime(path: text) -> i64 fs#read
```

Modification time of `path` as Unix epoch SECONDS (integer — same i64 representation as file.size). Returns `0` on missing file / IO error / pre-epoch dates — treat `0` as "unknown". Use for date-window filters: convert the returned seconds to YYYY-MM-DD and compare lexicographically against `ymd\_days\_ago(N)`.

```
pub fn set_file_size(self: File, size: integer) -> FileResult fs#update
```

Truncates or extends the file to `size` bytes; extending fills with zero bytes. Returns FileResult.Ok, FileResult.NotFound when the file does not exist, FileResult.IsDirectory for a directory, and FileResult.Other for a negative `size` or a failed OS call.

```
pub fn seek(self: File, pos: integer) -> boolean fs#update
```

Moves the read/write position to `pos` bytes from the start, for random access into a binary file: read an index, jump to the record it names, read that. Equivalent to `self\#next = pos`, the operator form. Returns false — a no-op — when there is nothing to seek in: a directory, an absent file, or a negative `pos`. A seek before the first read or write is remembered and applies to it. Seeking PAST the end is allowed: the position is remembered and a following write extends the file, which is how a free-list or an update-in-place walk lands.

```
pub fn position(self: const File) -> i64
```

The current read/write position in bytes, i.e. where the next read or write will land. The read side of [seek]; `self\#next` is the operator form. Distinct from `self\#index`, which is where the LAST read STARTED — after `x = f\#read as i32` on a fresh file, `position` is 4 and `f\#index` is 0. A file this process has not read from or written to yet has no position, and reports null rather than 0 — 0 is a legitimate position, so returning it would make "not opened" indistinguishable from "at the start". Discharge with `?? 0` when the distinction does not matter.

```
pub fn sync(self: File) -> boolean fs#update
```

Flushes buffered bytes for self to the underlying storage so that the preceding writes are durable. Use between log records or block boundaries to guarantee that earlier appends have landed on disk before later ones are issued.

40.15. Persistent stores

```
pub fn store_persist_copy(r: const reference, path: text) -> boolean fs#update
```

Write an image of `r` to `path`, laid out for PAGING, and keep the live collection as it is. Use this to save a collection and for a file another program (or a browser) will READ — a vocabulary, a map, any shipped dataset — where what matters is how few pages a query touches. The image is a rebuilt copy, with each record placed in its collection's own order — key order for a `trie` — so one prefix is one run of pages. Records in the live store sit where they were INSERTED; written as they are, a 20-record prefix query on a 74,692-word vocabulary read 22 requests and 1.25 MB, against 4 and 0.26 MB from the rebuilt image. The live collection is untouched: nothing moves, so every reference you hold stays valid, and writes after this do not reach the file. Write it again to save a later state; `store\_load` reads it back. words: trie<Word[w]> = [] for w in vocabulary() { words += [Word { w: w }] } store_persist_copy(words, "vocab.store") A `reference\<T\>` to a record in the same collection is carried: it names the same record in the image. Returns `false` on an I/O or format error, and for a collection the image cannot carry — one holding a reference into another store. The image is sized to its content, with none of the growth slack the live store keeps.

```
pub fn store_load(r: reference, path: text) -> boolean fs#read
```

Load a persisted store IMAGE fully into memory, populating the empty store-rooted collection `r` so it can be queried like any in-memory collection — the reading counterpart of `store\_persist\_copy`. It copies the file into memory, so it works on EVERY backend, wasm included, and writes after it stay in memory: write the collection again with `store\_persist\_copy` to save them. A file whose `.dschema` records an older layout than `r`'s declared type is converted in memory: every added field takes its default (which may read the record through `\$`), and the file itself is left as it is. Returns `false` on a missing / truncated / wrong-format file, and on a change the conversion cannot carry, such as one that looks like a rename (no panic; the reason is printed on stderr). This is the whole-file load that the browser's working-set path builds on. h: hash<Rec[id]> = [] store_load(h, "world.store")

```
pub struct Release { pub major: integer, pub minor: integer, pub patch: integer }
```

A release number, `major.minor.patch`: what a store records as the release of the program that wrote it, and what an `upgrade` routine is told its stored data came from. Releases are ordered by number, so `10.0` comes after `9.0`, and a text compares as the release it spells: fn upgrade(from: Release, old: ItemBefore2, item: Item) { if from < "2.0" { item.count = len(old.label) } }

```
pub operator compare(self: Release, other: Release) -> Ordering
```

Releases in the order they were made: by major, then minor, then patch.

```
pub operator compare(self: Release, other: text) -> Ordering
```

A release against the release a text spells (`"2"`, `"2.1"`, `"2.1.3"`; a part that is missing or not a number counts as 0): `from \< "2.0"`.

```
pub operator to_text(self: Release) -> text
```

`major.minor.patch`.

```
pub fn store_load_url(r: reference, url: text, sha256: text) -> boolean fs#read
```

Load a persisted store IMAGE over HTTP(S) from a TRUSTED source, establishing authenticity BEFORE the bytes are adopted: fetch the whole image at `url`, verify its SHA-256 against the caller-pinned `sha256` (lowercase hex), and only on a match heap-load it into `r` — the bytes never touch disk. A fetch error or a hash mismatch REFUSES the load (returns false, loads nothing). The fetch⟶verify⟶trust discipline the registry install uses, bridged onto the store loader; `url` may be `http(s)://` or `file://`. Like `store\_load`, it refuses an image whose `\<url\>.dschema` records a different layout than `r`'s type, and an image with no `.dschema` at all. Whole-file counterpart of the paged `store\_load\_key(s)`/ `store\_load\_range` loaders. h: hash<Rec[id]> = [] store_load_url(h, "https://cdn.example/world.store", "<sha256-hex>")

```
pub fn store_load_url_trusted(r: reference, url: text) -> boolean fs#read
```

Load a whole store IMAGE over HTTP(S)/file:

```
pub fn store_load_untrusted(r: reference, path: text) -> boolean fs#read
```

Load a store IMAGE from a local file that may be UNTRUSTED into `r` — the structurally-validated counterpart of store_load. Reads the file and validates its block structure BEFORE adoption, so a crafted or corrupt file cannot hang (0-size block) or drive a heap over-read; it is rejected (false). store_load is faster (validates only in debug) for a file you produced yourself; use this for a file whose provenance you don't control. h: hash<Rec[id]> = [] if !store_load_untrusted(h, "downloaded.store") { /* rejected — malformed */ }

```
pub fn store_verify(r: const reference) -> boolean
```

Structural integrity check of a store-rooted collection's heap graph: verify every internal pointer targets a live record — no dangling / out-of-bounds / cyclic edge. Returns true when sound, false (with a reason on stderr) when not. The verifier behind the working-set loader's confidence: after any `store\_load\*`, `store\_verify(local)` proves the (partial) copy produced a structurally valid heap, not one with a pointer left aimed at the source. h: hash<Rec[id]> = [] store_load_key(h, "block.store", 42) assert(store_verify(h))

```
pub fn store_reclaim(r: reference) -> integer fs#update
```

Give back the free space at the END of a store-rooted collection's store, and return the BYTES handed back to the allocator (0 when there was nothing to give). Records are never moved, so every reference into the collection stays valid. You say when, because only the program knows whether a drop is permanent: a collection that shrinks and grows again would just pay to re-grow. Read `store\_memory()` first, and read it as a RANKING rather than a quantity: its `tail%` says which store is worth reclaiming, and it is NOT the size of the return. A reclaimed store lands at `tail  11%` — a growth reserve the allocator keeps — so what comes back is `tail%  -  11%` of the resulting capacity, never the whole tail. Measured across eight shapes with tails from 13% to 60%: at a 13% tail the report suggests 36 KB and the call hands back 5832 bytes. Treat a reading near 11% as nothing to get back, not a little. Its `inner%` is the space BETWEEN records, which this does not touch — though the number RISES after a reclaim, because the capacity it is a percentage of has shrunk. WHERE the drop was matters as much as how big it was. `mergeable` counts adjacent free neighbours that never coalesced, so it measures how CONTIGUOUS the drop was, and that is exactly the part this call can fix: the same 2000 records dropped contiguously merge 2004 free blocks down to 7 and hand back 25400 bytes, while dropped alternately they leave 1997 blocks standing forever — the live records between them are what keeps them apart — and hand back 16312. You do NOT need this before writing an image: `store\_persist\_copy` writes the collection rebuilt, sized by what it holds. Call it MID-RUN, when a live set has dropped for good and the memory is wanted back before the end.

40.16. Lazy stores

```
pub fn store_bind_lazy(local: reference, source: text) -> boolean fs#read
```

Bind a COLLECTION to a lazy source. After this, a lookup that MISSES fetches that one entry and inserts it, so the next lookup is an ordinary resident hit; a lookup that hits never leaves the process. The collection is therefore automatically the cached data set — there is no separate cache. Per COLLECTION, not per store: `persons` and `companies` are different sources, and two collections of one type can bind differently. Binding replaces, and may be done before the collection holds anything. `source` is either an IMAGE — what `store\_load\_key` accepts: a local `.store` file or an `http(s)://` URL served with Range — or a DATABASE, named by a driver prefix. Returns false for a null collection. persons: hash<Person[id]> = [] store_bind_lazy(persons, "people.store") p = persons[42] `sqlite:\<path\>` binds to a table instead, and the query is DERIVED from the collection's own type: the table is the element type's name lowercased, the columns are its fields, and the `WHERE` is the collection's key. Nothing is written down twice. persons: hash<Person[id]> = [] Read-only, and the connection enforces it. A binding that cannot be served — a field that is not a column, a collection whose KIND the source cannot read — is REFUSED rather than served wrongly, and says so through `store\_lazy\_error`.

sqlite is opened on the first fault, so a program that binds no database loads nothing. FALSE means the binding was not made, and it is worth checking. A `.store` IMAGE is read a page at a time, which only a `hash`, a `trie` or a `spatial` supports: a `sorted` or `index` bound to one is refused HERE, at the call that is wrong, rather than answering `null` at every later lookup. Those kinds load whole — `store\_load` / `store\_load\_url\_trusted` carry all of them. A DATABASE source judges its own schema on the first fault instead, since what it can serve is a fact about the other end. if !store_bind_lazy(tiles, "tiles.store") { store_load(tiles, "tiles.store");

```
pub fn store_lazy_query(local: reference, condition: text) -> integer fs#read
```

Run an explicit condition against this collection's bound DATABASE source and pull every matching row into the collection. Answers how many records the collection gained. The escape hatch for what the collection's KEY cannot express — a predicate on another column, a pattern, a range nobody declared an index for. A keyed lookup derives its own query and needs no call; this one cannot be derived, so it is written down and visible rather than happening behind a lookup. found = store_lazy_query(persons, "name LIKE 'Ada%'"); p = persons[42] The rows land IN the collection, not in a separate result: a person reached by this query and the same person reached by a later lookup are ONE record, and a row already resident is left alone rather than fetched twice. So `len` and iteration keep answering "what have I got" — after this, more. `condition` is SQL, sent as written (the connection is read-only). Answers 0 both when nothing matched and when the query could not run; `store\_lazy\_error` tells those apart.

```
pub fn store_lazy_range(local: reference, lo: integer, hi: integer) -> integer
fs#read
```

Pull a whole KEY RANGE from this collection's bound DATABASE source in ONE query. Answers how many records the collection gained. This is what keeps lazy reading usable rather than merely correct. Fetching 500 records one lookup at a time is 500 round trips; the same 500 as a range is one. So when you know the span you want, ask for the span: store_lazy_range(events, 100, 199); The collection must be ORDERED (`sorted` or `index`) — a `hash` has no order to range over — and keyed on ONE column, since two numbers cannot say which value pins a composite key's leading column; use `store\_lazy\_query` for that. Both bounds are inclusive, in the collection's own key order. A record already resident is left alone. Answers 0 when nothing matched AND when the query could not run; `store\_lazy\_error` tells those apart.

```
pub fn store_lazy_error(local: const reference) -> text
```

Why a lazy fetch for this collection could not REACH its source, or "" when it is healthy. A lookup cannot tell you this. C80 says a value read never raises, so a miss answers `null` whether the key is genuinely absent or the source is unreachable — and those are different facts, one stable and one not. Ask this after a null to tell them apart: p = persons[42]; if p == null { why = store_lazy_error(persons); if why == "" { /* really no such person */ } else { /* could not reach: {why} */ } } The FIRST failure's reason, kept — not the last: it names the original cause, and later ones are usually the same failure repeating. Nothing clears it but `store\_lazy\_clear`. An absence does NOT, and neither does a later

success: reaching the source now says nothing about what an earlier failure already lost, and answering "healthy" over a traversal that missed data is the silent wrong answer this channel exists to prevent.

```
pub fn store_lazy_faults(local: const reference) -> integer
```

How many fetches could not REACH this collection's source. 0 is healthy. The magnitude behind store_lazy_error: after a traversal it answers "how incomplete am I".

```
pub fn store_lazy_clear(local: reference) -> boolean
```

Acknowledge this collection's fetch failures, returning whether there was anything to acknowledge. The ONLY thing that clears them. A later fetch happening to succeed does NOT: a traversal whose first lookup could not reach the source and whose second could is MISSING data, and answering "healthy" afterwards would be exactly the silent wrong answer this channel exists to prevent. Clearing is a caller saying "I have seen this", which is a different event entirely.

```
pub fn store_lazy_fail(local: reference, why: text) fs#read
```

A loft DRIVER reporting that it could not reach its source. The writing end of the channel store_lazy_error reads. A driver written in loft (fn lazy_fetch(...)) has the same three answers a Rust source has, and two of them are an integer: 1 inserted, 0 absent. The third is not — "the source is down" carries a REASON, and answering 0 for it would be a silent wrong answer, because a caller cannot tell it from "no such person". fn lazy_fetch(coll: hash<Person[id]>, source: text, key_int: integer, key_text: text) -> integer { if !db.db_open(source) { store_lazy_fail(coll, "cannot open {source}: {db.db_last_error()}"); return 0; } .. } Sticky and counted exactly like a Rust source's failure: the FIRST reason is kept, every failure is counted, and only store_lazy_clear clears them.

40.17. Paged store loading

```
pub fn store_load_key(local: reference, path: text, key: integer) -> boolean
fs#read
```

Working-set load: fetch ONE integer-keyed entry from a persisted HASH image into the empty local hash local, reading only the pages the lookup touches — not the whole file. The bounded-fetch counterpart of store_load, for when local should hold only the entries actually asked for (a phone pulling the few map tiles a route needs from a large block). The singular of store_load_keys and the integer form of store_load_key_text. path is a local file or an http(s):// URL served with Range, on every target including the browser (--html), where the fetch goes through the same bridge store_load_url_trusted uses. Returns false when the key is absent, the file is unreadable, or the collection is not an integer-keyed hash. The entry's own fields are RELOCATED into the local store, so text, nested structs and flat vectors all come across; only a vector\<text\> or a vector\<vector\> is refused (its element pointers would dangle), and a refusal says so on stderr rather than looking like an absent key. tiles: hash<Tile[id]> = [] store_load_key(tiles, "block.store", 42)

```
pub fn store_load_key_text(local: reference, path: text, key: text) -> boolean
fs#read
```

Text-keyed form of store_load_key: fetch ONE entry from a persisted hash\<T\[textkey\]\> or trie\<T\[textkey\]\> (a place-name / string-id index) into local, reading only the pages the lookup touches. Returns false when the key is absent or the collection isn't a copyable text-keyed hash or trie. places: hash<Place[name]> = [] store_load_key_text(places, "gazetteer.store", "Amsterdam")

```
pub fn store_load_prefix(local: reference, path: text, pre: text, limit: integer)
-> integer fs#read
```

Prefix form: fetch every entry whose text key begins with pre from a persisted trie\<T\[k\]\> into local, reading only the pages the prefix walk touches — what a search box needs, and what a sorted range cannot express without a hand-built successor string. Returns the count loaded. limit caps the WALK, not just the answer: with limit 8 the ninth record is never stepped to, so its pages are never fetched. A negative limit means no cap, which on a common prefix reads the whole run. words: trie<Word[w]> = [] store_load_prefix(words, "vocab.store", "kerk", 20)

```
pub fn store_load_box(local: reference, path: text, from: const vector<integer>,
till: const vector<integer>, limit: integer) -> integer fs#read
```

Box form: fetch every entry inside the closed bounding box from..till from a persisted spatial\<T\[x, y\]\> into local, reading only the pages the box walk touches — what a map viewport needs. Returns the count loaded. The corners are vectors so the same call serves 1, 2 or 3 axes, and writing them the other way round names the same box. TWO bounds, and a map needs both. limit caps the WALK, not just the answer: with limit 200 the 201st marker is never stepped to, so its pages are never fetched (a negative limit means no cap). And the BOX bounds it — the Morton interval between two corners is a superset the Z-order curve threads in and out of, so a wide, shallow viewport would otherwise read 1.46 M records to return 4 k. Measured on a 3.2 M-point map index: 5.3 pages of 64 KB for the first viewport, 1.5 for each pan after it, against a 158 MB whole-image download. Those numbers assume a dataset written with locality in every axis, which is the one thing this call cannot do for you. The Morton walk is symmetric; a file laid out along ONE axis is not, so a box crossing that axis pays for every stride it crosses. On a 62 500-point grid returning the same 500 records, a 250x2 box read 107 pages of 64 KiB against 7 for its 2x250 mirror, and the two swapped when the identical data was written in the other axis order — 81 % of the image to return 0.8 % of the records. Write such a dataset TILED: it has no bad orientation, and for the square-ish boxes a viewport uses it beats either linear order. pins: spatial<Pin[x, y]> = [] store_load_box(pins, "map.store", [x1, y1], [x2, y2], 200)

```
pub fn store_load_keys(local: reference, path: text, keys: const vector<integer>)
-> integer fs#read
```

Plural form of store_load_key: fetch the given integer keys' entries into local in one call (the paged reader is opened once and its cache reused), returning how many were found. Keys absent from the remote are skipped. Same relocation rules as store_load_key. tiles: hash<Tile[id]> = [] got = store_load_keys(tiles, "block.store", [7, 13, 42])

```
pub fn store_load_keys_text(local: reference, path: text, keys: const
vector<text>) -> integer fs#read
```

Plural form of `store\_load\_key\_text`, and the text twin of `store\_load\_keys`: fetch the given text keys' entries into `local` in ONE call, returning how many were found. Keys absent from the remote are skipped. Serves a `hash\<T\[k\]\>` and a `trie\<T\[k\]\>` alike, like the single-key form. Looping `store\_load\_key\_text` is not the same call: each one opens its own reader, so a ring of twenty asset names re-fetches the same 64 KiB bucket-table page twenty times. This opens the reader once and shares its page cache — which is what makes it the PREFETCH call: ask for the ring at a load or level boundary, so no frame is ever the thing that discovers it needs an asset. art: hash<Blob[bl_key]> = [] got = store_load_keys_text(art, "pack.store", ["page/mobs", "hit.ogg"])

```
pub fn store_load_range(local: reference, path: text, lo: integer, hi: integer) -
> integer fs#read
```

Range form: fetch every entry whose integer key is in [lo, hi] from a persisted `sorted\<T\[k\]\>` into `local`, reading only the pages the range walk touches — the ordered-collection counterpart of `store\_load\_keys` (a phone pulling the corridor of map tiles a route crosses). Returns the count loaded. tiles: sorted<Tile[tkey]> = [] store_load_range(tiles, "block.store", lo_cell, hi_cell)

40.18. Environment

```
pub fn env_variables() -> vector<EnvVariable> env#read
```

Returns all environment variables as a vector of EnvVariable records (fields: name, value). Use to inspect or forward the full environment.

```
pub fn env_variable(name: text) -> text env#read
```

Returns the value of the environment variable `name`, or `null` when it is not set. A variable set to the empty string answers `""`, so `== null` tells the two apart. Use to read configuration from the shell environment.

```
pub fn arguments() -> vector<text> env#read
```

Functions for interacting with the host operating system. Returns the script-level arguments passed after the script path. Does not include the loft binary name or loft CLI flags.

```
pub fn ymd_days_ago(days: integer) -> text time#read
```

Returns the UTC calendar date `days` days before today as `YYYY-MM-DD`. Use for cutoff dates in time-window filters; negative `days` clamps to today.

```
pub fn directory(v: text = "") -> text env#read
```

Returns the current working directory, optionally with v appended as a subpath. loft sets the working directory to the program's own directory, so this is that directory — unless the file declares `\#cwd`, which keeps the directory the program was launched from.

```
pub fn user_directory(v: text = "") -> text env#read
```

Returns the current user's home directory, optionally with v appended. Use for storing user-specific data or configuration.

```
pub fn program_directory(v: text = "") -> text env#read
```

Returns the directory containing the running executable, optionally with v appended. Use to locate assets shipped beside a compiled program. Under the interpreter the executable is the `loft` binary itself, and a `--native` run executes from a temporary build directory, so neither names the script's directory — use `source\_dir()` for that.

```
pub fn source_dir() -> text env#read
```

Returns the directory containing the main source file being executed. Use to locate data files relative to the script, regardless of working directory.

40.19. Optional C libraries

```
pub fn c_library_available(soname: text) -> boolean env#read
```

Returns whether the C library soname is usable right now: it loads, and every `\#c` binding declared against it resolves in it. Ask this before calling into a library declared with `\[c\] optional-libs`, which is not installed on every machine and is loaded only when a binding needs it. Both halves of the answer matter — a library of the wrong version loads and then exports only some of the symbols, so "the file is there" would say yes where the call still fails. A library the program never declared answers false.

40.20. System directories

Where this machine keeps temporary files and loft's per-user cache — each answering null on a target with no filesystem (the browser). The host input and output channels live here too.

```
pub fn temp_dir() -> text?env#read
```

Returns the system temporary directory, or null on a target with no filesystem (the browser). Prefer this over reading TMPDIR / TEMP / TMP directly: those differ by platform (Unix uses TMPDIR, Windows TEMP/TMP) and this also applies the OS fallback (e.g. /tmp) when none is set.

```
pub fn cache_dir() -> text?env#read
```

Returns loft's per-user cache directory ($XDG_CACHE_HOME/loft, else $HOME/.cache/loft) — the same root the engine caches into — or null on a target with no filesystem (the browser).

```
pub fn host_input(wait_ms: integer = -1) -> text host#read
```

Reads pending host input as one text; empty when there is none. The source is per-target: stdin on native and –native-wasm (WASI); the JS host's input QUEUE on –html — seed it with globalThis.loftInput before loft_start, and push live messages any time with globalThis.loftPush(msg) (e.g. fetch() completions). Use for a headless compute module or a polled input loop — the inbound mirror of host_output. The engine only moves opaque bytes; the program parses them. wait_ms says how long to wait for input that has not arrived yet. The default –1 reads the WHOLE stream, waiting for stdin to end; 0 takes what is already there and returns at once; a positive value waits that many milliseconds for the first byte. Ask with a bound whenever the answer might be that nobody is listening: an absent host never closes stdin, so the default form waits forever for a reply that is not coming. host_output("MODE?"); mode = host_input(200); On –html every read already polls (a page cannot wait for a message its own thread has to deliver), and on –native-wasm, which has no threads, a bounded read falls back to waiting for the whole stream.

```
pub fn host_output(msg: text) host#append
```

Sends one STRUCTURED message to the host shell — the outbound mirror of host_input, distinct from user-facing print. Per target: a line on stderr (native and WASI — the invoking process can script it); the page's globalThis.loftOutput(msg) handler on –html. The browser pattern for loft-initiated work: host_output a request (e.g. "fetch <url>"), the JS shell acts on it, and pushes the completion back via loftPush for host_input to pop — JS owns the network, loft stays pure compute.

40.21. Time

```
pub fn now() -> i64 time#read
```

Returns the current wall-clock time as milliseconds since the Unix epoch (00:00 UTC on 1 Jan 1970). Use for timestamps and logging.

```
pub fn ticks() -> i64 time#read
```

Returns microseconds elapsed since program start (monotonic clock). Unaffected by system clock adjustments. Use for benchmarks and frame timing.

40.22. Memory diagnostics

```
pub fn store_memory() -> text
```

Returns a multi-line snapshot of all LIVE stores' internal memory utilisation: total capacity vs actual claimed data vs free space, record + free-block counts, mergeable adjacent-free pairs (free neighbours that should have coalesced), and the largest stores by capacity with their creation site (`bc:\<pos\>` — a bytecode position on the interpreter; 0 on –native) and type name. Use to watch memory growth in a running program.

40.23. Path helpers

```
pub fn starts_with(self: text, value: text) -> boolean
```

Returns true if self begins with value. Use for prefix matching (e.g., protocol detection).

```
pub fn ends_with(self: text, value: text) -> boolean
```

Returns true if self ends with value. Use for suffix matching (e.g., file extension checks).

```
pub fn dir(self: text) -> text
```

Directory part of a path — strips the trailing `/\<segment\>`. Paths are plain slash-separated text: a `\\` is not a separator here, on any platform. “doc/claude/PROBLEMS.md”.dir() ⟶ “doc/claude” “CLAUDE.md”.dir() ⟶ “” (no `/`) “/etc/passwd”.dir() ⟶ “/etc” “”.dir() ⟶ “”

```
pub fn basename(self: text) -> text
```

Last segment of a path — the part after the trailing `/`. “doc/claude/PROBLEMS.md”.basename() ⟶ “PROBLEMS.md” “CLAUDE.md”.basename() ⟶ “CLAUDE.md” “/foo/”.basename() ⟶ “” (trailing slash) “”.basename() ⟶ “”

```
pub fn join(self: text, other: text) -> text
```

Join two paths — `self/other` with a single `/` separator. Special cases: - other starts with `/` (absolute) ⟶ return other unchanged - self is empty ⟶ return other - self ends with `/` ⟶ no extra separator inserted

```
pub fn resolve(self: text, target: text) -> text
```

Resolve `target` against `self` (where self is a base directory). Strips leading `./` repeatedly; each `../` segment trims the last component from `self`. Mirrors scan.loft’s `resolve\_link\_path`. “doc/claude”.resolve(“../README.md”) ⟶ “doc/README.md” “doc/claude”.resolve(“./PROBLEMS.md”) ⟶ “doc/claude/PROBLEMS.md” “a/b/c”.resolve(“../../x”) ⟶ “a/x” “”.resolve(“foo”) ⟶ “foo”

40.24. Text: case and classification

```
pub fn to_lowercase(self: text) -> text
```

Returns a lowercase copy. Use for case-insensitive comparisons.

```
pub fn to_uppercase(self: text) -> text
```

Returns an uppercase copy.

```
pub fn is_lowercase(self: text) -> boolean
```

True if the text is non-empty and all characters are lowercase letters.

```
pub fn is_lowercase(self: character) -> boolean
```

True if the character is a lowercase letter.

```
pub fn is_uppercase(self: text) -> boolean
```

True if the text is non-empty and all characters are uppercase letters.

```
pub fn is_uppercase(self: character) -> boolean
```

True if the character is an uppercase letter.

```
pub fn is_numeric(self: text) -> boolean
```

True if the text is non-empty and all characters are numeric digits (Unicode numeric, not just ASCII 0–9).

```
pub fn is_numeric(self: character) -> boolean
```

True if the character is a numeric digit.

```
pub fn is_alphanumeric(self: text) -> boolean
```

True if the text is non-empty and all characters are letters or digits. Use to validate identifiers or tokens.

```
pub fn is_alphanumeric(self: character) -> boolean
```

True if the character is a letter or digit.

```
pub fn is_alphabetic(self: text) -> boolean
```

True if the text is non-empty and all characters are alphabetic.

```
pub fn is_alphabetic(self: character) -> boolean
```

True if the character is alphabetic.

```
pub fn is_whitespace(self: text) -> boolean
```

True if the text is non-empty and all characters are whitespace. Use to detect blank lines.

```
pub fn is_whitespace(self: character) -> boolean
```

True if the character is whitespace.

```
pub fn is_control(self: text) -> boolean
```

True if the text is non-empty and all characters are control characters.

```
pub fn is_control(self: character) -> boolean
```

True if the character is a control character.

```
pub fn join(self: const vector<text>, sep: text) -> text
```

Joins parts with sep between each consecutive pair. Returns “” for an empty vector. Use to build comma-separated lists, path segments, or any delimited output.

40.25. Text: bytes and code points

```
pub fn byte_at(self: text, i: integer) -> integer
```

Return the BYTE at position `i` (0 up to `size()`) as integer 0-255, or 0 for out-of-bounds; a negative `i` counts from the end, as a slice bound does (`byte\_at(-1)` is the last byte). Unlike `text\[i\]` which decodes the UTF-8 codepoint containing byte `i` (walking back through continuation bytes), `byte\_at(i)` is a pure O(1) byte read. Use in ASCII-heavy scanning hot paths (tokenisers, regex- like loops) where the UTF-8 decode is wasted work — every non-ASCII byte still returns a valid 0-255 number; the caller compares against ASCII constants so byte semantics suffice. 5-10× faster than `text\[i\]` for pure-ASCII checks.

```
pub fn text_from_bytes(bytes: const vector<u8>) -> text
```

Build a text from the raw UTF-8 bytes of a vector<u8> — the inverse of byte_at. Use in binary decoders (CBOR text, HPKE byte composition) that assemble a byte buffer and need to turn it back into text. Bytes that are not valid UTF-8 yield the empty text (never a crash); validate first if you must tell “empty input” from “invalid bytes” apart.

```
pub fn text_from_byte_range(bytes: const vector<u8>, lo: integer, hi: integer) ->
text
```

The same over the byte RANGE `lo..hi` of the buffer, read in place: what a decoder that has found a text’s span writes instead of `text\_from\_bytes(bytes\[lo..hi\])`, whose slice is a fresh vector (`(Slice-Value)`) built to be read once. The bounds are the slice’s (a negative one counts from the end; both clamp); a reversed or empty range, or invalid UTF-8, is “”. Its own name, not an overload: a native is dispatched by name.

```
pub fn chr(cp: integer) -> text
```

Build a one-character text from a Unicode CODE POINT — the inverse of the `ch as integer` that text iteration already gives you, and the code-point twin of text_from_bytes’ byte route. Use when you have a number and need the character it names: decoding an escape (`\\u{...}`, an HTML entity), walking a code-point table, or reassembling text a code point at a time. chr(65) “A” chr(233) “é” chr(20013) “中” chr(128512) “😀” A code point that names no character answers the EMPTY text, never a crash: a

surrogate (D800-DFFF), anything past U+10FFFF, a negative number — and also 0, because `character` uses 0 as its null and text iteration stops there, so a NUL built here could not be read back. If you need an embedded NUL, go through the byte route: `text\_from\_bytes(\[0\])` carries one.

40.26. Stack traces

```
pub enum ArgValue {
  NullVal,
  BoolVal { b: boolean },
  IntVal { pub n: integer },
  LongVal { pub n: integer },
  FloatVal { f: float },
  SingleVal { f: single },
  CharVal { c: character },
  TextVal { pub t: text },
  RefVal { store: integer, rec: integer, pos: integer },
  FnVal { d_nr: integer },
  OtherVal { description: text },
}
```

Typed union of inspectable argument/variable values. Each variant wraps one primitive type so that match expressions can recover the concrete value from a StackFrame's argument list.

```
pub struct ArgInfo {
  pub name: text,
  pub type_name: text,
  pub value: ArgValue,
}
```

One function argument in a stack frame.

```
pub struct VarInfo {
  pub name: text,
  type_name: text,
  pub value: ArgValue,
}
```

One local variable or parameter in a stack frame (see `stack\_trace`).

```
pub struct StackFrame {
  pub function: text,
  file: text,
  pub line: integer,
  pub arguments: vector<ArgInfo>,
  pub variables: vector<VarInfo>,
}
```

One call frame in the stack trace.

```
pub fn stack_trace() -> vector<StackFrame>
```

Return the current call stack as a vector of frames, outermost first. Each frame's `variables` holds its parameters and live locals, `arguments` the parameters alone (typed via `ArgValue`). Use this to inspect not only the current function's variables but also the variables of any function further up the call stack. Under `--native` both lists are empty: a compiled frame has no reader for its locals (STACKTRACE.md § Known Limitations, ST-6). `fn debug\_dump() { for frame in stack\_trace() { println("{frame.function}:{frame.line}"); for v in frame.variables { println("{v.name} = {v.value}"); } } }`

40.27. Coroutines

```
pub enum CoroutineStatus {
  Created,
  Suspended,
  Running,
  Exhausted,
}
```

Lifecycle state of a coroutine frame. Transitions: Created -> Running -> Suspended -> Running -> ... -> Exhausted.

40.28. JSON

```
pub enum JsonValue {
  JNull,
  JBool { value: boolean },
  JNumber { value: float },
  JString { value: text },
  JArray { pub items: vector<JsonValue> },
  JObject { pub fields: vector<JsonField> },
  JInteger { value: integer },
}
```

Typed union of JSON values. The discriminant (1..7) picks the active variant; variant data lives in the variant's fields. Matches the RFC 8259 kinds, plus `JInteger` for an integer-shaped number preserved to an exact `integer` (i64). `JInteger` MUST stay the LAST variant: the store discriminant it gets (7) is hard-coded as `JV\_DISCR\_INT` in `src/native.rs`, and the existing variants' discriminants (1–6) must not shift.

```
pub struct JsonField {
  pub name: text,
  pub value: JsonValue,
}
```

One field of a `JObject`. A `JObject` keeps its fields as a `vector\<JsonField\>` in source order, so a lookup by name is a linear scan — fine for the object sizes typical in configuration / API responses.

```
pub fn json_parse(raw: text) -> JsonValue
```

Parse JSON text into a `JsonValue` tree. Malformed input returns `JNull`; the error trail is accessible via `json\_errors()`. Every variant materialises (primitives, arrays, objects, nested containers); the entire tree lives in one store and frees as one unit when the root `DbRef` leaves scope. `match json\_parse(raw) { JObject { fields } =\> for f in fields { handle(f) }, JArray { items } =\> for v in items { handle(v) }, JNull =\> println("parse error: {json\_errors()}"), \_ =\> println("unexpected root kind"), }`

```
pub fn json_errors() -> text
```

Return a pipe-separated trail of JSON parse errors from the most recent `json\_parse` call. Empty when the parse succeeded. Each entry carries an RFC 6901 path, a `line:col` location, and a context snippet.

40.29. JSON: reading values

```
pub fn field(self: const JsonValue, name: text) -> JsonValue[self]
```

JObject indexer — returns the value at `name`, or `JNull` when `self` isn't a JObject or the key is missing. Chained access like `root.field("a").field("b")` is safe — every intermediate missing produces `JNull`, never a trap.

```
pub fn item(self: const JsonValue, index: integer) -> JsonValue[self]
```

JArray indexer — returns the element at `index`, or `JNull` when `self` isn't a JArray or the index is out of bounds.

```
pub fn len(self: const JsonValue) -> integer
```

Length of a JArray's items vector or a JObject's fields vector. Returns `null` for any other variant.

```
pub fn as_text(self: const JsonValue) -> text
```

Typed extractor — returns `null` on kind mismatch. The null is NOT the empty text, and only `??` tells the two apart: on a mismatch `s == ""` is false and `len(s)` is 1, while a field that really holds `""` compares equal and measures 0. Reach for `as\_text(...) ?? fallback`, never `== ""`.

```
pub fn as_number(self: const JsonValue) -> float
```

Typed extractor — returns `null` on kind mismatch.

```
pub fn as_long(self: const JsonValue) -> i64
```

Typed extractor — returns `null` on kind mismatch. A `JInteger` answers its exact value; a `JNumber` is truncated toward zero.

```
pub fn as_bool(self: const JsonValue) -> boolean?
```

Typed extractor — returns `null` on kind mismatch. Declared `boolean?`, so a mismatch — an absent field, the string `"true"`, the number `1` — is `null` and never reads as a field that really says `false`.

```
pub fn kind(self: const JsonValue) -> text
```

Returns the variant name as text: `"JNull"`, `"JBool"`, `"JNumber"`, `"JString"`, `"JArray"`, `"JObject"` or `"JInteger"`. Cheap: reads the discriminant byte, formats a literal. Useful for logs and conditional branches that don't want to commit to a full pattern match.

```
pub fn keys(self: const JsonValue) -> vector<text>
```

Returns the field-name list of a `JObject` in insertion order, or an empty vector for any other variant. Safe idiom: `for k in v.keys() { ... }` works on any JsonValue because non-objects yield an empty walk.

```
pub fn fields(self: const JsonValue) -> vector<JsonField>
```

Returns the (name, value) entries of a `JObject` in insertion order so callers can iterate as `for entry in fields(v) { … entry.name … entry.value … }`. Values deep-copy (primitives + nested containers — full tree). Empty vector for any other variant.

```
pub fn has_field(self: const JsonValue, name: text) -> boolean
```

Returns true iff `self` is a `JObject` variant carrying a field named `name`. All other variants (including `JNull` on a parse error) return false, so the common pattern `if v.has\_field("users") { … }` is safe to write on any JsonValue without first destructuring. Distinguishes "absent" from "present-but-null" — a field whose value is `JNull` still returns `true`.

40.30. JSON: building values

```
pub fn to_json(self: const JsonValue) -> text
```

Render a JsonValue to canonical RFC 8259 JSON text. Every variant serialises; `JArray` / `JObject` recurse through their children (full tree serialisation, nested containers walk naturally). Strings escape `"`, `\\\\`, and ASCII control bytes; UTF-8 passes through verbatim. Non-finite numbers render as `null`.

```
pub fn to_json_pretty(self: const JsonValue) -> text
```

`to\_json\_pretty` produces 2-space indented, one-element-per-line output for non-empty `JArray` / `JObject` containers. Empty containers render `\[\]` / `{}` (no newline padding). Primitives are byte-identical to `to\_json` (no nested structure to indent). After object keys the colon is followed by a single space (`"k": v`). Useful for golden-file tests and log output.

```
pub fn json_null() -> JsonValue
```

Build a JsonValue set to the `JNull` variant. Useful in test fixtures and reply-construction code that needs a known-null JsonValue without going through `json\_parse("null")`.

```
pub fn json_bool(v: boolean) -> JsonValue
```

Build a JsonValue set to the `JBool` variant carrying the supplied boolean payload.

```
pub fn json_number(v: float?) -> JsonValue
```

Build a JsonValue set to the `JNumber` variant carrying the supplied float payload. Non-finite inputs (float null = NaN, or ±Inf) produce `JNull` with a diagnostic in `json\_errors()` — mirrors the RFC 8259 constraint that JSON numbers must be finite. The parameter is `float?` because handling a null/NaN input IS its contract (⟶ `JNull`); a finite `float` passes as a non-null value into the nullable slot as usual.

```
pub fn json_string(v: text) -> JsonValue
```

Build a JsonValue set to the `JString` variant carrying the supplied text payload. The text is copied into the JsonValue's own store, so the returned value owns the string independently of the argument's lifetime.

```
pub fn json_array(items: const vector<JsonValue>) -> JsonValue
```

Build a JsonValue set to the `JArray` variant carrying the supplied items. Each element is deep-copied into the new tree's arena via the shared `dbref\_to\_parsed` walker, so nested containers and arena-origin subtrees (e.g. a captured `field()` result) embed correctly. Empty input produces a real empty JArray.

```
pub fn json_object(fields: const vector<JsonField>) -> JsonValue
```

Build a JsonValue set to the `JObject` variant carrying the supplied fields. Each field's value deep-copies via the same `dbref\_to\_parsed` walker as `json\_array`, so a JObject can carry captured-subtree JArray / JObject values. Empty input produces a real empty JObject.

40.31. JSON: the struct bridge

```
pub fn struct_from_jsonvalue(v: const JsonValue, struct_kt: integer) -> JsonValue
```

Internal walker — populate a struct of the given `struct\_kt` (known-type number) from a JsonValue. Compile-time codegen for `Struct.parse(JsonValue)` emits exactly one call to this function regardless of struct shape; the runtime walker uses `stores.types\[struct\_kt\].parts` to dispatch on each field's declared type (primitive, nested struct, JsonValue passthrough, or vector). Path-qualified schema-side diagnostics on type mismatches go to `json\_errors()`. Users should not call this directly — write `MyStruct.parse(value)` instead. Return type is declared as `JsonValue` here purely because it shares the same DbRef byte layout as the actual `reference\[T\]` the walker produces — the compile-time codegen at `parse\_type\_parse` overrides the type to `reference\[T\]` for the caller while the stack accounting remains correct.

```
pub fn struct_to_json(self_ref: const JsonValue, struct_kt: integer) -> text
```

Serialise any user struct to canonical JSON (RFC 8259) — what `instance.to\_json()` calls. Text fields are escaped (" / \\ / control bytes); nested structs and vectors recurse; a `JsonValue`-typed field renders its subtree as it is.

```
pub fn struct_to_json_pretty(self_ref: const JsonValue, struct_kt: integer) ->
text
```

As `struct\_to\_json` but produces a pretty (2-space-indent, one element per line) form. Same field-type matrix.

40.32. Type reflection

```
pub enum TypeKind {
  IntegerKind,
  LongKind,
  SingleKind,
  FloatKind,
  BooleanKind,
  TextKind,
  CharacterKind,
  /// A struct — its fields are in `fields`.
  RecordKind,
  /// An enum — its variants are in `variants`.
  EnumKind,
  /// One variant of a struct-enum; it has fields of its own.
  VariantKind,
  /// A vector — `element` names what it holds.
  VectorKind,
  /// A keyed collection (hash / index / sorted / ordered / radix / trie; a
  /// `spatial` reports as radix) — walked by cursor rather than by layout, so it
  /// has no fields of its own. Which of the six it is, and on which fields, are
  /// in `collection` and `keys`.
  KeyedKind,
  /// A stored reference to another record.
  RefKind,
  /// A kind this loft version has no name for. Never guessed at.
  OtherKind,
}
```

What a type IS — the discriminant a caller matches on before reading the fields that only some kinds have. These are STORAGE kinds, because storage is what the descriptor records. A narrow `i32` field reports `IntegerKind`; the declared width is visible in `size`, not in a separate kind.

```
pub struct FieldInfo {
  pub name: text,
  /// The field type's name, as the store records it.
```

```
  pub type_name: text,
  /// Byte offset of the field within its record.
  pub position: integer,
  pub kind: TypeKind,
  /// Was the field DECLARED nullable (`text?` rather than `text`)?
  ///
  /// Not a layout fact — a nullable field occupies the same bytes and spells an
  /// absent value with a sentinel — so nothing in the stored bytes implies it,
  /// and it reaches you only because the compiler records it. It is what a
  /// generated `CREATE TABLE` needs for `NOT NULL`.
  pub nullable: boolean,
}
```

One field of a record or of a struct-enum variant.

```
pub enum CollectionKind {
  /// Not a keyed collection. Every other kind of type answers this, so a
  /// caller can read `collection` without matching `kind` first.
  NotKeyed,
  KeyedHash,
  KeyedIndex,
  KeyedSorted,
  KeyedOrdered,
  KeyedRadix,
  KeyedTrie,
}
```

WHICH keyed collection a type is — the shape of its lookup. `KeyedKind` says a type is walked by cursor; this says what by. The distinction decides what a query over it can be: a `hash` answers equality only, `sorted` / `ordered` / `index` also answer a range in their declared direction, a `radix` is a Morton-order structure that no SQL shape means the same thing as, and a `trie` answers equality, byte order, and a PREFIX — which is `LIKE 'x%'` and nothing narrower.

```
pub struct KeyInfo {
  pub name: text,
  /// Byte offset of this field within the ELEMENT record — the same number the
  /// element type's `FieldInfo.position` carries, so a caller joins the two by
  /// VALUE rather than by matching names. A name is a display fact; the
  /// position is what the collection actually keys on.
  pub position: integer,
  /// Does the collection order this key ascending?
  ///
  /// `true` for a kind that has no order of its own (`hash`, `radix`), and for a
  /// `trie`, whose single key IS ordered ascending by byte. Only a kind that can
  /// be declared descending ever answers `false`. Match `collection` first where
the
  /// difference between "ascending" and "unordered" matters.
```

```
  pub ascending: boolean,
}
```

One key field of a keyed collection.

```
pub struct VariantInfo {
  pub name: text,
  /// The discriminant this variant is stored as. Never 0 — 0 means absent.
  pub tag: integer,
}
```

One variant of an enum.

```
pub struct TypeInfo {
  pub name: text,
  pub kind: TypeKind,
  /// Bytes one record of this type occupies; 0 for a type with no record.
  pub size: integer,
  pub const fields: vector<FieldInfo>,
  pub const variants: vector<VariantInfo>,
  /// For a vector or a keyed collection, the element type's name.
  pub element: text,
  /// For a keyed collection, which of the six it is; `NotKeyed` otherwise.
  pub collection: CollectionKind,
  /// The key fields of a keyed collection, in KEY ORDER — the order a composite
  /// lookup binds them in, which is why it is a vector and not a set.
  ///
  /// Empty for every type that is not keyed, and empty for the one keyed shape
  /// whose keys are not its element's own fields: a `__nullable<S>` element
  /// keys through its `Some` payload. Empty rather than partial is deliberate —
  /// a query built from half a composite key is a WRONG query, not a narrower
  /// one, so a key list is delivered whole or not at all.
  pub const keys: vector<KeyInfo>,
}
```

The declared shape of one type. `fields` is empty for everything but a record and a struct-enum variant; `variants` for everything but an enum; `element` is `""` unless the type holds one. Empty is the honest answer for a kind that has no such thing — a caller matches `kind` first.

```
pub fn reflect_type(kt: integer) -> TypeInfo
```

The declared shape of `value`'s type. **The argument is read for its TYPE and is not evaluated** — the same contract C's `sizeof` has, and for the same reason: nothing about the answer depends on the value. Pass a variable, a field or a parameter; an expression with a side effect will not have it. `t = type\_of(row); println("{t.name}:"); for f in t.fields { println("  {f.name}: {f.type\_name} \@{f.position}") }` **Not inside a generic.** A generic body is parsed ONCE against its type variable, so `type\_of(v)` there answers a type-variable placeholder (`\_\_typevar\_T…`) — the same reason `"{v:j}"` in a generic body renders `{}`. Call it where the concrete type is known.

Describes a TYPE. To read what a VALUE holds at one of these positions, use `field\_value`. Reflection offers no way to WRITE a value by field.

```
pub fn type_named(name: text) -> TypeInfo?
```

The declared shape of the type called `name`, or `null` if this program has no such type — reflection with no value in hand. This is what an ORM or a schema check needs: the name arrives from a config file, a database catalogue or a command line, so there is nothing to call `type\_of` on. `t = type\_named("Row"); if t == null { println("no such type") } else { println("{t.name}") }` **`TypeInfo?`, and null means the name is not a type here.** A type that does not exist has no shape, and saying so in the type is what makes a caller handle it rather than read a plausible-looking empty answer. A name is matched exactly as the store records it, which is the loft type name — `Point`, `text`, `vector\<Point\>`. A composite name such as `vector\<Point\>` exists only when the program uses that type somewhere; a program that never does answers `null` for it. Unlike `type\_of`, no parser intercept: the name is a RUNTIME value, so the lookup is a runtime one. It works on `--native` because the generated `init()` replays the type registrations — names included — and `Stores::name` is a total lookup that answers "absent" rather than minting a type for a typo.

```
pub struct ValueInfo {
  /// What the type's own descriptor says lives at that position — never what
  /// the caller expected to find there.
  pub kind: TypeKind,
  /// The SCALAR at that position holds loft's NULL.
  ///
  /// Separate from the payload because a null is not a value a payload can
  /// spell: `0`, `""` and `false` are ordinary answers a field can genuinely
  /// hold, and an ORM that could not tell them from SQL NULL would write the
  /// wrong row. Test this before reading `i` / `f` / `t`.
  ///
  /// `false` for a `kind` that has no scalar reading and for `OtherKind` — a
  /// field that was never read is not a field that read as null, and those are
  /// three answers rather than two.
  pub is_null: boolean,
  pub i: integer,
  pub f: float,
  pub t: text,
}
```

One field's VALUE, read out of a record at the position reflection reported. `kind` says which payload carries the answer, so a caller matches it once rather than asking a different question per type: | `kind` | read | |—|—| | `IntegerKind` · `LongKind` | `i` | | `BooleanKind` | `i` — 1 or 0 | | `CharacterKind` | `i` — the code point | | `FloatKind` · `SingleKind` | `f` | | `TextKind` | `t` | | `RecordKind` · `VectorKind` · `KeyedKind` · `RefKind` | nothing — no scalar reading | | `OtherKind` | nothing — see `field\_value` | A boolean and a character ride in `i` for the reason a bound SQL value does: one integer path is one thing to get right, and the `kind` beside it is what keeps them apart.

```
pub fn reflect_field(value: const TypeInfo, position: integer, kt: integer) ->
ValueInfo
```

The value `value` holds at byte `position` — the VALUE half of reflection. `type\_of` says a record has a `text` at byte 16; this reads it. Together they are what a generic serialiser, an ORM write or a diff needs, and neither half is enough alone: `t = type\_of(row); for f in t.fields { v = field\_value(row, f.position); if v.is\_null { println("{f.name}=null") } else { println("{f.name}={v.t}") } }` **The position is CHECKED against the type's own descriptor, never trusted.** A number that does not begin a field — a hand-made offset, a stale one, a position from a different type — answers `OtherKind`. A field whose type has no scalar reading (a nested record, a vector, a keyed collection, a stored reference) answers its own kind with no payload. So there is no offset a caller can pass that reads bytes belonging to something else, which is what makes this ordinary loft rather than a pointer. `kind` is the DESCRIPTOR's answer, not the caller's: passing the position of an `integer` field does not make the reading an integer, it makes it whatever that field is. That is why the kind is reported rather than requested. **REFUSED inside a generic**, and refused rather than merely unsupported. A generic body is parsed once against its type variable, so there is no concrete type to read positions out of — the same limit `type\_of` has. Left to answer, every call there reports `OtherKind`, which for an ORM's write half is an EMPTY ROW rather than an error: the write succeeds and the columns are missing. So it is a compile error naming the type variable. Call it one frame out, where the type is known, and pass the values in.

```
pub fn reflect_field_path(value: const TypeInfo, path: const vector<integer>, kt:
integer) -> ValueInfo
```

The value at the end of a PATH of positions — `field\_value(x, \[8, 0\])`. `type\_of` reports a nested record's fields at positions relative to THAT record, so one number cannot name `origin.x` — and reflection hands back no handle to a nested record to call `field\_value` on again. A path closes that: each element is a position in the record the previous element landed in. **The offsets must not be added up by the caller.** `field\_value(doc, 8 + 0)` asks for a field BEGINNING at byte 8 of `Doc`, which is `origin` itself — the check that makes this ordinary loft rather than a pointer is that a position must begin a field of the type it is read against. So the walk happens inside, one descriptor step per element, each checked the same way. `// Doc.origin \@8, Point.y \@8 v = field\_value(doc, \[8, 8\]); // doc.origin.y` Every step but the last must land on an INLINE record. A step onto a scalar, a vector, a keyed collection or a stored reference answers `OtherKind` and reads nothing: a stored reference names a record with its own identity, and following it would be a pointer chase rather than a field read. An empty path answers `OtherKind` for the same reason a bad position does — nothing names a field there. A one-element path is the single-position form, and answers identically.

41. Roadmap

Loft is under active development. Every code example on the language pages is executed by the test suite on both backends, so what they show runs today. This page describes where the project is headed.

The project goal is **browser games that anyone can play via a shared link**. Native OpenGL is supported for desktop enthusiasts. Server and multiplayer features come after the single-player browser experience works.

Since **2026-06** loft ships on a calendar version: a release is named for its month, which `Cargo.toml` spells `2026.6.0`, `2026.7.0`, `2026.8.0` — `loft --version` prints the one you have. The semver milestones below were the plan before that switch; the two numbered ones have shipped, and 1.0.0 is still the name of the finish line rather than a month.

41.0.1. 0.8.4 (shipped 2026-04-24): Awesome Brick Buster

0.8.4 turns Brick Buster from a tech demo into a game someone would actually want to share with a friend, and makes sharing trivial via single-file HTML export.

41.0.1.1. Game polish

- **Hand-designed levels** — five themed layouts dispatched by `level_brick(lv, r, c)`, with a procedural fallback for deeper runs.
- **Cel-shaded sprite atlas** — outlined icons, a round ball with velocity-directional squash, hearts for lives, balloon projectiles, fireball after-images.
- **Roman-numeral level display** and **persistent high score** (`.loft/brickbuster_score.txt`).
- **Chiptune soundtrack** — three early-Capcom-style tracks (Heroic / Determined / Calm) that play once per level with silences between, plus brick/paddle/wall/pickup/life sound effects.
- **Screen shake** on brick breaks and life lost; pause menu; title and game-over screens.

41.0.1.2. Single-file HTML export

Compile any loft program to a self-contained `.html` file that runs in any browser — no install, no server.

```
$ loft --html app.loft
```

The game's WebAssembly, textures, and audio are all embedded. Host the file anywhere, send someone the URL, they play.

41.0.1.3. Infrastructure

- **Audio** — `rodio`-backed playback on desktop; WebAudio bridge in the browser.
- **Store-leak fixes** — per-frame idiomatic struct APIs now work without workarounds (P122 + P117–P131 resolved).
- **CLI hardening** — script-level arguments, file-scope constants, and release-mode coroutine-iterator fixes (P126, P128, P131, P132).
- **Bytecode cache** — `.loftc` files are invalidated on interpreter rebuild via the embedded git commit hash.

41.0.2. 0.8.5 (shipped 2026-06-07): Language maturity

The language itself got solid: closures hold a live reference to what they capture, bounded generics carry their types through methods and tuples, and native compilation went to production.

41.0.2.1. Working Moros editor

The Moros hex RPG scene editor runs end-to-end in the browser: load a map, paint hexes, place walls and items, see a live 3D preview, export to GLB. Web only — multiplayer comes in 1.0.0.

- **Map model** — `Hex`, `Chunk`, `Map` with material/wall/item palettes and spawn/NPC routines, serialised as JSON.
- **2D scene canvas** — hex coordinate math, pan/zoom/hit-test, layered rendering.
- **Editor shell** — toolbar, palettes, Paint / Height / Wall / Item tools, undo/redo, localStorage autosave.
- **3D renderer** — hex surface geometry (flat and slope), wall slabs, stairs, camera orbit, hex picking, GLB export.
- **Live 3D preview** alongside the 2D editor, powered by the loft graphics library compiled to WASM.

41.0.3. 0.9.0 — Fully working loft language

The language itself becomes feature-complete, well-documented, and tooling-friendly; every item below has shipped in the monthly releases. No "appears fixed but unverified" bugs. Anyone can write loft code in their preferred editor with syntax highlighting, decent error messages, and a REPL for experimentation.

41.0.3.1. Language polish

- **Error recovery** — the parser continues after a token-level failure and reports multiple errors in a single pass — **shipped** (a two-error program reports both in one pass).
- **REPL** — running `loft` with no arguments starts an interactive session; definitions persist across lines — **shipped** (bare `loft` starts one and restores the last session).
- **Developer warnings** — Clippy-inspired lints for common mistakes — **shipped** (a two-tier warning/advice set with `--explain` fix lines).
- **AOT library compilation** — automatically compile libraries to native shared libs for faster startup — **shipped** (a `used` library builds its own native library when a Rust toolchain is present).
- **Stdlib hygiene** — a `std::` prefix for shadowed builtins; library enums in `match` arms without qualification — **shipped**.

41.0.3.2. Compilation cache

The parsed standard library and program are cached under `.loft/` and restored on the next run, alongside the shared constant store — **shipped**.

41.0.3.3. Developer experience

- TextMate grammar for `.loft` syntax highlighting — **shipped** (`editors/vscode/syntaxes/loft.tmLanguage.json`).
- VS Code extension with snippets and a run task — **shipped** (`editors/vscode/`, plus `loft-lsp` and `loft-dap`).
- Quick-start `examples/` directory — **shipped** (`examples/`).
- CI covering package tests and native codegen — **shipped** (`make ci`).

41.0.3.4. Packaging and FFI

- **Lock file** (`loft.lock`) for reproducible builds — **shipped** (`loft install` writes it).
- **Generic FFI marshaller** — zero-boilerplate native functions from a `#native` signature; generic `cdylib` loader scans exports into a hash map — **shipped** (`loft generate` writes the stubs).

41.0.4. 1.0.0 — Totally sure everything works

The stability contract. Anyone can write, run, and share a program — terminal or browser — and trust that it will keep working.

41.0.4.1. Web IDE

A browser-based IDE running the full loft interpreter as WebAssembly — no installation, no server.

- Editor shell with CodeMirror 6 and a loft grammar.
- Symbol navigation (go-to-definition, find-usages).
- Multi-file projects stored locally (IndexedDB).
- Documentation and examples browser built in.
- Export/import ZIP; PWA offline mode.

41.0.4.2. Scene scripting

The Moros scene editor gains a loft scripting panel with in-browser compile and hot reload. Scene scripts are sandboxed, travel with the scene JSON, and can be shared alongside the map.

41.0.4.3. Multiplayer

- **Server library** — plain HTTP, HTTPS with static certs, WebSockets, CORS headers and static file serving are **shipped** (the `server` library). Still to build: automatic ACME certs, JWT / session / API-key auth, rate limiting.
- **Game loop primitives** — WebSocket polling, broadcast, connection registry — **shipped** (`server`).
- **Game client library** — the `GameEnvelope` protocol (`game_protocol`) and a fixed-timestep loop (`fixstep`) are **shipped**. Still to build: lobby + matchmaking, client-side prediction and reconciliation, WASM script loading with Ed25519 verification.
- **Moros multiplayer** — DM and players share a live scene hosted on a loft server.

41.0.4.4. Stability contract

Version 1.0 means: any program that works on 1.0 will compile and run identically on all future 1.x releases. The language syntax, type system, standard library, and command-line flags are frozen. Until then, what a released version promises is kept, and a breaking change needs a documented reason and a migration.

Tagging 1.0 requires every stability gate to be satisfied without shortcuts: zero open high-severity bugs, valgrind-clean debug builds of the full Brick Buster and tuple tests, green CI on Linux, macOS Intel, macOS ARM, and Windows, pre-built binaries on the GitHub release, and the reference + PDF linked from the release page.

41.0.5. 1.1 and beyond

Post-1.0 items include route decorators (`@get` / `@post` / `@ws`), iterator protocol (`for msg in ws`), interface factory methods, lazy work-variable initialisation, stack raw-pointer cache, and additional native codegen optimisations. Browser-side `par()` on Web Workers (`loft --html --threads`) and the `spatial<T>` collection have already shipped.

41.0.6. Following progress

Development is tracked in the GitHub repository. The full internal roadmap with effort estimates, dependencies, and design pointers is in the repository's `doc/claude/ROADMAP.md`.